ProcureFlow Local Enterprise DevOps Modernization

A connected-reasoning, copy-paste implementation manual for the GEP Senior DevOps interview

Prepared for: Yasir Malik

Target environment: Windows 11 + WSL 2 Ubuntu + Docker Desktop + kind Kubernetes

Document date: 24 June 2026

```
LEGACY PROCUREMENT PLATFORM
        |
        |  assess -> containerize -> automate -> migrate -> observe -> secure
        v
LOCAL ENTERPRISE DEVOPS PLATFORM
        |
        +-- Node.js + Java microservices
        +-- Jenkins CI + local registry
        +-- Helm + Argo CD GitOps
        +-- kind Kubernetes + Gateway API
        +-- PostgreSQL + MySQL migration
        +-- Kafka + RabbitMQ + Elasticsearch
        +-- Prometheus + Grafana + Loki
        +-- Security, load tests, incidents, runbooks
```

The complete project story in one connected chain

Interview objective
Aap sirf tools install nahi karenge. Aap ek business-critical procurement platform ko legacy baseline se controlled Kubernetes target tak le jane ki complete engineering story demonstrate karenge: problem, evidence, decision, command, expected result, rollback aur business impact.

How to use this manual

1. Commands ko WSL 2 Ubuntu terminal mein exactly run karein, jab tak Windows PowerShell explicitly na likha ho.
2. Har step ke Expected evidence ko verify kiye baghair next step par na jayen.
3. Failure aaye to Decision table se branch select karein; random commands na chalayein.
4. Screenshots, command outputs aur incident notes ko docs/evidence folder mein save karein.
5. Interview mein har component ka naam bolne ke bajaye uska business reason aur preceding evidence explain karein.

Resource reality
Kafka, Elasticsearch, Jenkins, SonarQube, Prometheus, Grafana aur three application environments ek saath laptop par heavy ho sakte hain. 16 GB RAM par staged profiles use hongi; 32 GB+ par fuller profile use ki ja sakti hai. Project ka senior-level value controlled rollout mein hai, laptop ko freeze karne mein nahi.

Table of Contents

Use this document map to jump to the required implementation phase. PDF search and heading bookmarks provide the fastest navigation.

Part I - Business context, architecture, and execution contract

1. Project definition: what exactly are we building?

ProcureFlow ek fictional lekin realistic procurement platform hai. Is project ka purpose GEP ke business domain ko copy karna nahi, balke procurement workload ke through Senior DevOps ownership demonstrate karna hai. Platform purchase requests, suppliers, order approvals, asynchronous events, notification tasks aur search operations handle karega.

Previous evidence	JD mein Node.js, Java, Jenkins, Helm, Argo CD, Kubernetes, Azure-style networking, stateful migration, Kafka, RabbitMQ, MySQL, PostgreSQL, Elasticsearch aur Prometheus demanded hain.
Why this step now	Agar hum har tool ko separate lab mein install karenge to interview story fragmented lagegi. Isliye ek single business transaction ko sab components se logically connect karna zaroori hai.
Action	Ek purchase-order flow define karein aur har platform component ko is flow mein explicit responsibility dein.
Expected evidence	Har technology ke liye ek clear producer, consumer, data ownership aur operational failure mode available hoga.
Next connection	Ab architecture ko transaction flow se derive karenge, na ke random tool list se.

```
User submits purchase request
              |
              v
Node.js Frontend / Supplier API
              |
              v
Java Order Service -----> PostgreSQL
              |
              | publishes OrderCreated
              v
Kafka Topic
              |
              v
Java Approval Service
              |
              | creates notification work item
              v
RabbitMQ Queue
              |
              v
Node.js Notification Worker
              |
              +---- status/event indexing ----> Elasticsearch
              |
              v
Prometheus metrics + Loki logs + Grafana dashboards
```

One business transaction creates the complete platform dependency chain

Why each component exists

Observation	Meaning	Next decision
Node.js Supplier API	Fast API layer for supplier operations and a direct match to the JD.	Expose health, metrics and supplier CRUD endpoints.
Java Order Service	Represents business-critical Java microservice and JVM operations.	Persist order and publish OrderCreated event.
PostgreSQL	Transactional storage for modern services.	Demonstrate schema, connections, health and backup.
Kafka	Durable business-event stream consumed by multiple services.	Demonstrate consumer lag and event-driven scaling.
RabbitMQ	Task queue for retryable notification work.	Demonstrate acknowledgements, retries and dead-letter queue.
Elasticsearch	Operational search/indexing workload.	Demonstrate cluster health and disk pressure incident.
MySQL legacy workload	Source stateful workload to migrate.	Demonstrate backup, restore, cutover and

Observation	Meaning	Next decision
		rollback.
Jenkins	Continuous Integration orchestration using Groovy.	Test, build, scan and push immutable images.
Argo CD	Continuous Delivery and desired-state reconciliation.	Separate CI from CD and enable Git-based rollback.
Prometheus/Grafana/Loki	Evidence for reliability decisions.	Connect application symptoms to infrastructure signals.

The interview sentence to memorize

Core explanation

I built ProcureFlow as a local modernization lab for a business-critical procurement platform. I first established a measurable legacy baseline, then containerized Java and Node.js services, introduced CI, immutable artifacts, Helm packaging and Argo CD GitOps, migrated a MySQL stateful workload with validation and rollback, and finally added observability, security, scaling and incident runbooks. Every migration decision was driven by evidence from the previous stage.

2. Connected reasoning model used in every phase

Har technical step ka same reasoning pattern hoga. Yeh pattern aapko implementation aur interview dono mein disconnected answers se bachata hai.

```
Previous evidence
      |
      | tells us what is missing or risky
      v
Reason for next step
      |
      | defines a controlled action
      v
Exact command / configuration
      |
      | produces measurable output
      v
Expected evidence
      |
      +---- unexpected ---> troubleshoot / rollback
      |
      +---- expected ------> next dependent step
```

The evidence-driven implementation loop

Example: why registry comes before Kubernetes deployment

Previous evidence	Docker image local machine par build ho gayi, lekin kind node ek separate Docker container hai.
Why this step now	Kind node host Docker image ko automatically nahi dekh sakta. Deployment se pehle image-distribution path prove karna zaroori hai.
Action	Local registry create karein, kind containerd ko registry trust karwayen, test image push karein aur pod se pull verify karein.
Expected evidence	Pod Running state mein ho aur image localhost:5001 se pull ho.
Next connection	Ab Helm deployment reliable image source use kar sakti hai.

3. Scope boundaries: what local lab proves and what it does not

Local implementation production Azure ka substitute nahi hai. Iska purpose repeatable engineering patterns prove karna hai. Interview mein honest boundary seniority dikhati hai.

Local capability	What it proves	What remains cloud-specific
kind multi-node cluster	Kubernetes scheduling, rollout, probes, HPA and GitOps behavior	Real cloud availability zones, managed control plane SLA
Local registry	Immutable image tagging and CI artifact flow	ACR identity, geo-replication and private endpoints

Local capability	What it proves	What remains cloud-specific
Docker/Kind volumes	StatefulSet, PVC, backup and restore logic	Azure Disk failure domains and managed database SLA
Gateway API + Envoy	Role-based traffic routing and route ownership	Cloud load balancer and public DNS integration
Local Prometheus	Metrics, alerts, SLI/SLO reasoning	Enterprise retention and managed monitoring scale
Controlled failure injection	Triage, rollback and postmortem discipline	Actual customer blast radius

Interview honesty
Kabhi yeh claim na karein ke local kind cluster production AKS ke equal hai. Kahen: I validated the delivery and operational patterns locally, documented cloud-specific gaps, and designed a clean migration path to AKS.

4. Resource profile and staged execution plan

Heavy stack ko blindly start karne se Out Of Memory, disk pressure aur false application failures aa sakte hain. Isliye pehle resource budget define hota hai, phir profile select hoti hai.

Machine capacity	Recommended mode	Execution rule
16 GB RAM, 4-8 CPU	Core profile	Registry, kind, app services, PostgreSQL, Argo CD, Prometheus/Grafana. Kafka/RabbitMQ/Elasticsearch/SonarQube ko phase-wise run karein.
24 GB RAM, 8 CPU	Expanded profile	Core + one messaging stack + Jenkins/SonarQube; Elasticsearch separately validate karein.
32 GB+ RAM, 8-12 CPU	Fuller demo profile	Most components together, lekin resource requests/limits phir bhi enforce karein.
Less than 16 GB	Minimal profile	One worker node, one environment namespace, no Elasticsearch/SonarQube during normal development.

Suggested Docker Desktop limits

Docker Desktop ke WSL backend ko enough memory milni chahiye, lekin Windows host ko bhi breathing room chahiye. Start with 10-12 GB for a 16 GB laptop only if Windows remains responsive; otherwise run smaller profiles. Docker Desktop settings vary by version, isliye actual resource screen verify karein.

Staged profiles

```
# Profile A - foundation
registry + kind + gateway + app + PostgreSQL

# Profile B - delivery
Profile A + Jenkins + SonarQube + Trivy

# Profile C - messaging
Profile A + Kafka + RabbitMQ

# Profile D - search and observability
Profile A + Elasticsearch + Prometheus + Grafana + Loki

# Profile E - migration demo
legacy MySQL + target MySQL StatefulSet + application validation
```

5. Project repository as the single source of truth

Previous evidence	Architecture aur scope defined hain, lekin commands abhi scattered ho sakti hain.
Why this step now	Repeatability tab aati hai jab code, infrastructure, documentation, evidence aur rollback scripts ek predictable repository structure mein hon.
Action	Repository skeleton create karein aur har future artifact ko correct ownership directory mein place karein.
Expected evidence	git status sirf intentional files dikhaye aur tree structure project

	architecture ko reflect kare.
Next connection	Ab preflight script isi repository ke scripts folder mein add hogi.

Step 5.1 - Repository skeleton create karein

Previous evidence	Project architecture approved hai.
Why this step now	Ab har component ka stable location chahiye taa ke later Jenkins, Helm aur Argo CD relative paths reliably use kar saken.
Action	Run the copy-paste block below.
Expected evidence	Directory list applications, platform, ci, observability, messaging, databases, migration, incidents aur docs ko show kare.
Next connection	Stable structure ke baad version control baseline create kar sakte hain.

```
mkdir -p ~/projects/procureflow-devops-modernization
cd ~/projects/procureflow-devops-modernization

mkdir -p \
  applications/frontend-nodejs \
  applications/supplier-service-nodejs \
  applications/notification-worker-nodejs \
  applications/order-service-java \
  applications/approval-service-java \
  applications/integration-runtime-java \
  legacy-platform/docker-compose \
  legacy-platform/ansible \
  platform/kind \
  platform/terraform/local \
  platform/helm/procureflow/templates \
  platform/argocd/apps \
  platform/kubernetes/base \
  platform/kubernetes/gateway \
  platform/security \
  ci/jenkins \
  ci/scripts \
  observability/prometheus \
  observability/grafana/dashboards \
  observability/loki \
  messaging/kafka \
  messaging/rabbitmq \
  databases/postgresql \
  databases/mysql \
  databases/elasticsearch \
  migration/mysql-to-kubernetes \
  load-testing/k6 \
  incidents \
  docs/architecture \
  docs/adr \
  docs/runbooks \
  docs/evidence \
  scripts

find . -maxdepth 3 -type d | sort
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
All expected folders appear	Skeleton correct hai.	Git initialization karein.
Permission denied	Repository Windows-mounted path ya wrong owner par ho sakti hai.	Project ko WSL home, e.g. ~/projects, mein rakhein; sudo chown -R "$USER:$USER" only on your own project.
Folders created under /mnt/c	Performance aur file-permission issues ho sakte hain.	Repository ko WSL ext4 filesystem mein move karein.

Step 5.2 - Git baseline create karein

Previous evidence	Repository folders successfully create ho gaye.
Why this step now	Migration aur troubleshooting mein known-good checkpoints critical hote hain. Isliye first technical change se pehle baseline commit chahiye.
Action	Run the copy-paste block below.

Expected evidence	One initial commit show ho aur git status empty ho.
Next connection	Version-control checkpoint prove hone ke baad environment validation safely start hogi.

```
cd ~/projects/procureflow-devops-modernization

git init
git branch -M main

cat > .gitignore <<'EOF'
.env
*.log
*.tmp
*.swp
.DS_Store
.terraform/
*.tfstate
*.tfstate.*
terraform.tfvars
secrets/
coverage/
target/
node_modules/
.vscode/
.idea/
artifacts/
EOF

cat > README.md <<'EOF'
# ProcureFlow DevOps Modernization

Local enterprise modernization lab for Java and Node.js procurement workloads.

## Delivery chain
Source -> Jenkins CI -> Local Registry -> GitOps -> Argo CD -> kind Kubernetes
EOF

git add .
git commit -m "chore: initialize ProcureFlow modernization repository"
git log --oneline -1
git status --short
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Commit created and status empty	Clean baseline ready.	Preflight checks add karein.
Author identity unknown	Git user.name/user.email configured nahi.	git config --global user.name "Yasir Malik" and git config --global user.email "your-email" then repeat commit.
Untracked secret file	Sensitive value accidentally repository mein hai.	File remove karein, .gitignore update karein, phir commit karein.

Part II - Workstation, WSL 2, Docker, and toolchain foundation

6. Why environment preflight comes before installation

Agar Docker unavailable ho, disk full ho, WSL integration disabled ho, ya project Windows filesystem par ho, to later Kubernetes errors misleading honge. Preflight root cause ko earliest layer par isolate karta hai.

```
Windows host healthy?
      |
      v
WSL 2 distribution healthy?
      |
      v
Docker engine reachable?
      |
      v
CPU / memory / disk sufficient?
      |
      v
CLI versions pinned?
      |
      v
Only then create registry and cluster
```

Step 6.1 - Windows-side WSL verification

Previous evidence	Git baseline exists; no platform processes have started.
Why this step now	WSL and Docker Desktop integration are the substrate. Agar substrate broken ho to every later container command fail karega.
Action	Run the copy-paste block below.
Expected evidence	Ubuntu distribution VERSION column mein 2 show kare aur WSL command errors na de.
Next connection	WSL 2 confirmed hone ke baad Linux-side Docker socket verify hoga.

```
# Run this block in Windows PowerShell, not inside Ubuntu
wsl --status
wsl --version
wsl --list --verbose
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Ubuntu VERSION = 2	Correct WSL generation active.	Ubuntu terminal open karke Docker test karein.
Ubuntu VERSION = 1	Docker Desktop WSL 2 workflow ready nahi.	PowerShell: wsl --set-version Ubuntu 2, then re-check.
WSL command missing or old	WSL installation/update required.	PowerShell as administrator: wsl --update; reboot if requested.

Step 6.2 - Docker integration verify karein

Previous evidence	WSL distribution VERSION 2 par running hai.
Why this step now	Ab prove karna hai ke Docker CLI WSL se Docker Desktop engine tak reach karti hai. Sirf docker binary ka present hona enough nahi.
Action	Run the copy-paste block below.
Expected evidence	docker version Client aur Server dono show kare; hello-world successful message de.
Next connection	Docker engine proven hone ke baad capacity test meaningful hoga.

```
cd ~/projects/procureflow-devops-modernization

docker version
docker info --format 'ServerVersion={{.ServerVersion}} Driver={{.Driver}} CPUs={{.NCPU}}
Memory={{.MemTotal}}'
docker run --rm hello-world
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Client and Server shown	WSL integration correct.	Resource and disk checks karein.
docker: command not found	Docker Desktop WSL integration disabled ya CLI path unavailable.	Docker Desktop -> Settings -> Resources -> WSL Integration -> Ubuntu enable; Apply and restart.
Cannot connect to Docker daemon	Docker Desktop stopped ya integration stale.	Docker Desktop start/restart karein; wsl --shutdown; reopen Ubuntu.
hello-world pull timeout	Network, proxy, DNS or registry access issue.	docker pull hello-world separately run; proxy/DNS troubleshoot before continuing.

Step 6.3 - Capacity and filesystem preflight

Previous evidence	Docker engine reachable hai.
Why this step now	Cluster creation se pehle CPU, RAM aur disk headroom verify karna zaroori hai. Otherwise later pod failures actual code issue nahi, resource starvation hongi.
Action	Run the copy-paste block below.
Expected evidence	At least 4 CPU, preferably 8; several GB free RAM; at least 25-40 GB free disk for full project; project filesystem ext4/WSL filesystem ho.
Next connection	Capacity determines which components can be activated, then CLI installation can be pinned.

```
printf '
== CPU ==
'
nproc
lscpu | grep -E 'Model name|CPU\(s\)'

printf '
== MEMORY ==
'
free -h

printf '
== DISK ==
'
df -h / /var/lib/docker 2>/dev/null || df -h /

printf '
== PROJECT FILESYSTEM ==
'
df -T "$HOME/projects/procureflow-devops-modernization"

printf '
== DOCKER USAGE ==
'
docker system df
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Free disk > 40 GB and RAM adequate	Expanded/full profiles possible.	Install CLI tools.
Free disk 15-25 GB	Core profile possible but image/cache cleanup needed.	Use staged profiles and docker builder prune only when safe.
Project filesystem drvfs under /mnt/c	Bind-mount performance and Linux permission issues likely.	Move repository to ~/projects.
Memory pressure already high	Full stack unsafe.	Close heavy apps and use core profile.

7. Tool version policy

Latest floating versions repeatability ko break kar sakti hain. Hum versions ko environment file mein pin karenge, lekin install se pehle official release page ke against verify bhi karenge. Document ke reference build mein kind v0.32.0 aur Envoy Gateway v1.8.1 use kiye gaye hain.

Step 7.1 - Version contract file banayein

Previous evidence	Machine capacity known hai.
Why this step now	Same commands kal aur interview se pehle same behavior dein, isliye tool versions central file mein hon.
Action	Run the copy-paste block below.
Expected evidence	All version variables clearly visible hon.
Next connection	Version contract package installation scripts ko deterministic banayega.

```
cd ~/projects/procureflow-devops-modernization

cat > .tool-versions.env <<'EOF'
KIND_VERSION=v0.32.0
KUBECTL_MINOR=v1.35
HELM_MAJOR=3
ENVOY_GATEWAY_VERSION=v1.8.1
ARGOCD_CHANNEL=stable
TERRAFORM_VERSION=1.13.5
TRIVY_VERSION=0.67.2
EOF

cat .tool-versions.env
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
File prints expected versions	Version contract ready.	Install prerequisite packages.
A version later becomes unavailable	Upstream release changed or asset removed.	Update one central file and record ADR; do not silently use latest.

Step 7.2 - Base Linux packages install karein

Previous evidence	Versions pinned hain.
Why this step now	curl, jq, git, unzip aur certificates later installers aur API parsing ke dependencies hain. Inko pehle verify karna error chain ko short rakhta hai.
Action	Run the copy-paste block below.
Expected evidence	Loop har command ka path print kare aur exit code 0 ho.
Next connection	Base packages ke baad Kubernetes clients reliably download aur verify ho sakte hain.

```
sudo apt-get update
sudo apt-get install -y \
  ca-certificates \
  curl \
  git \
  gnupg \
  jq \
  make \
  unzip \
  zip \
  gettext-base \
  openssl \
  netcat-openbsd \
  dnsutils \
  iproute2 \
  python3 \
  python3-pip \
  python3-venv

for cmd in curl git jq make openssl python3; do
  command -v "$cmd" || exit 1
done
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
All command paths printed	Base dependencies ready.	kubectl install karein.
apt lock error	Another package process running.	Process finish hone dein; lock files manually delete na karein.
Repository/DNS failure	Network layer issue.	nslookup archive.ubuntu.com and curl -I https://archive.ubuntu.com run karein.

Step 7.3 - kubectl install aur validate karein

Previous evidence	curl and certificates working hain.
Why this step now	kind cluster banane ke baad control plane se baat karne ke liye kubectl required hai. Client pehle install hoga taa ke cluster creation immediately validate ho.
Action	Run the copy-paste block below.
Expected evidence	Checksum kubectl: OK show kare aur clientVersion output aaye.
Next connection	kubectl available hone ke baad cluster lifecycle tool kind install hoga.

```
cd /tmp
KUBECTL_VERSION="$(curl -L -s https://dl.k8s.io/release/stable.txt)"
echo "Installing kubectl ${KUBECTL_VERSION}"

curl -fsSLO "https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/amd64/kubectl"
curl -fsSLO "https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/amd64/kubectl.sha256"

echo "$(cat kubectl.sha256)  kubectl" | sha256sum --check
sudo install -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client --output=yaml
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Checksum OK	Binary authentic download se match karti hai.	kind install karein.
Checksum failed	Corrupt or unexpected download.	Files delete karke official URL se re-download; install na karein.
Exec format error	Wrong CPU architecture asset.	uname -m check karke arm64 asset use karein if applicable.

Step 7.4 - kind v0.32.0 install karein

Previous evidence	kubectl client valid hai.
Why this step now	kind local Kubernetes nodes Docker containers mein banata hai. Cluster lifecycle tool ke baghair Kubernetes target available nahi.
Action	Run the copy-paste block below.
Expected evidence	kind v0.32.0 print ho.
Next connection	kind cluster packages ko deploy karne ke liye Helm next dependency hai.

```
cd /tmp
source ~/projects/procureflow-devops-modernization/.tool-versions.env
ARCH="$(uname -m)"
case "$ARCH" in
  x86_64) KIND_ARCH=amd64 ;;
  aarch64|arm64) KIND_ARCH=arm64 ;;
  *) echo "Unsupported architecture: $ARCH"; exit 1 ;;
esac

curl -fsSLo kind "https://kind.sigs.k8s.io/dl/${KIND_VERSION}/kind-linux-${KIND_ARCH}"
chmod +x kind
sudo mv kind /usr/local/bin/kind
kind version
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Correct version printed	Cluster tool ready.	Helm install karein.
HTML/error text downloaded	URL/version wrong or network interception.	file /usr/local/bin/kind; remove invalid file; verify release URL.
Permission denied on /usr/local/bin	sudo rights required.	Use sudo install or place in ~/.local/bin and update PATH.

Step 7.5 - Helm install karein

Previous evidence	kind installed hai.
Why this step now	Envoy Gateway, monitoring, Argo CD and operators Helm charts se manage honge. Helm pehle install hoga taa ke add-on lifecycle reproducible rahe.
Action	Run the copy-paste block below.
Expected evidence	helm version v3.x aur environment output show ho.
Next connection	Core CLIs installed hone ke baad one-command preflight create ki ja sakti hai.

```
cd /tmp
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
helm env
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Helm v3 shown	Package manager ready.	Unified preflight script create karein.
Download blocked	GitHub raw access blocked.	Use official Helm release tarball and checksum.
Helm v2 shown	Old binary path shadowing.	which -a helm; remove/update old binary.

Step 7.6 - Reusable preflight script create karein

Previous evidence	Docker, kubectl, kind and Helm individually validated hain.
Why this step now	Interview/demo se pehle manual checks repeat karna error-prone hai. Script same minimum contract every time enforce karegi.
Action	Run the copy-paste block below.
Expected evidence	All mandatory checks PASS hon; only resource warning acceptable hai if minimal profile intentionally selected.
Next connection	Preflight success registry and cluster creation ka entry gate hai.

```
cd ~/projects/procureflow-devops-modernization

cat > scripts/00-preflight.sh <<'EOF'
#!/usr/bin/env bash
set -Eeuo pipefail

fail() { echo "FAIL: $*" >&2; exit 1; }
pass() { echo "PASS: $*"; }

for cmd in docker kubectl kind helm git curl jq; do
  command -v "$cmd" >/dev/null 2>&1 || fail "$cmd is missing"
  pass "$cmd found at $(command -v "$cmd")"
done

docker info >/dev/null 2>&1 || fail "Docker daemon is not reachable"
pass "Docker daemon reachable"

AVAILABLE_KB=$(df -Pk "$HOME" | awk 'NR==2 {print $4}')
if (( AVAILABLE_KB < 15728640 )); then
  fail "Less than 15 GB free space in WSL home filesystem"
fi
pass "At least 15 GB free disk available"

MEM_KB=$(awk '/MemTotal/ {print $2}' /proc/meminfo)
if (( MEM_KB < 7500000 )); then
  echo "WARN: Less than about 8 GB visible to WSL; use minimal profile"
else
  pass "WSL memory is adequate for core profile"
fi

echo
printf 'Docker: '; docker version --format '{{.Server.Version}}'
printf 'kubectl: '; kubectl version --client --output=json | jq -r '.clientVersion.gitVersion'
printf 'kind: '; kind version
printf 'Helm: '; helm version --short
EOF

chmod +x scripts/00-preflight.sh
./scripts/00-preflight.sh
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
All PASS	Foundation ready for registry.	Commit checkpoint create karein.
Disk FAIL	Cluster images may fill disk.	docker system df inspect; remove only unused resources; expand disk if needed.
Docker FAIL	Cluster cannot be created.	Return to Docker integration step.
CLI missing	Tool installation incomplete.	Install missing command before moving forward.

Step 7.7 - Foundation checkpoint commit

Previous evidence	Preflight script passes.
Why this step now	Known-good workstation contract ko Git mein capture karna rollback aur peer review ke liye necessary hai.
Action	Run the copy-paste block below.
Expected evidence	Latest commit preflight/version contract show kare.
Next connection	Ab artifact-distribution layer build karenge.

```
cd ~/projects/procureflow-devops-modernization
git add .tool-versions.env scripts/00-preflight.sh README.md .gitignore
git commit -m "chore: add local toolchain version contract and preflight"
git log --oneline -2
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Commit visible	Foundation reproducible.	Registry creation start karein.
Nothing to commit	Changes already committed.	Continue; do not create empty commit.

Part III - Local registry and kind Kubernetes foundation

8. Why image distribution comes before application deployment

Docker image host par build hona aur Kubernetes node ka us image ko pull kar pana do different facts hain. kind nodes Docker containers hain; unka containerd runtime host image cache automatically share nahi karta. Isliye registry path ko isolated test image se prove karna pehla Kubernetes engineering task hai.

```
Developer / Jenkins
       |
       | docker push localhost:5001/app:tag
       v
Local Registry container
       |
       | kind containerd mirror configuration
       v
kind worker node
       |
       | image pull
       v
Kubernetes Pod
```

9. Local registry with deterministic port handling

Step 9.1 - Check whether port 5001 is already occupied

Previous evidence	Preflight passes and Docker is reachable.
Why this step now	Registry container bind karne se pehle existing listener identify karna zaroori hai. Blind docker run duplicate registry aur port conflict create karta hai.
Action	Run the copy-paste block below.
Expected evidence	Either no listener ho, ya clearly existing procureflow-registry container identify ho.
Next connection	Port ownership known hone ke baad idempotent registry script safely run ho sakti hai.

```
cd ~/projects/procureflow-devops-modernization

printf '== Host listener ==
'
ss -ltnp | grep ':5001' || true

printf '
== Docker port users ==
'
docker ps --format '{{.ID}}   {{.Names}}      {{.Ports}}' | grep '5001' || true

printf '
== Existing registry container ==
'
docker ps -a --filter 'name=^/procureflow-registry$' \
  --format '{{.ID}}   {{.Names}}      {{.Status}}     {{.Ports}}' || true
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
No output for port 5001	Port free hai.	Create registry container.
procureflow-registry already running	Reusable registry exists.	Do not create duplicate; validate /v2/ endpoint.
Different container/process owns 5001	Port collision real hai.	Stop/remove only if safe, or choose a documented alternative port consistently.
Stopped procureflow-registry exists	Name reserved but service inactive.	docker start procureflow-registry; recreate only if configuration wrong.

Step 9.2 - Idempotent registry creation script

Previous evidence	Port conflict status known hai.
Why this step now	Manual docker run repeat karne se duplicate-name aur duplicate-port errors aati hain. Script existing state ko inspect

	karke create/start/reuse decision leti hai.
Action	Run the copy-paste block below.
Expected evidence	HTTP 200 OK aur empty JSON object {} return ho.
Next connection	Registry API healthy hone ke baad kind nodes ko is registry ka mirror path configure karenge.

```
cd ~/projects/procureflow-devops-modernization

cat > scripts/10-registry-up.sh <<'EOF'
#!/usr/bin/env bash
set -Eeuo pipefail

REG_NAME=procureflow-registry
REG_PORT=5001

if docker ps --format '{{.Names}}' | grep -qx "$REG_NAME"; then
  echo "Registry already running"
elif docker ps -a --format '{{.Names}}' | grep -qx "$REG_NAME"; then
  echo "Starting existing registry"
  docker start "$REG_NAME" >/dev/null
else
  if ss -ltn | awk '{print $4}' | grep -q ":${REG_PORT}$"; then
    echo "Port ${REG_PORT} is already occupied by another process" >&2
    exit 1
  fi
  echo "Creating registry"
  docker run -d \
    --restart=always \
    -p "127.0.0.1:${REG_PORT}:5000" \
    --name "$REG_NAME" \
    registry:2 >/dev/null
fi

curl -fsS "http://127.0.0.1:${REG_PORT}/v2/" >/dev/null
echo "Registry healthy at 127.0.0.1:${REG_PORT}"
EOF

chmod +x scripts/10-registry-up.sh
./scripts/10-registry-up.sh
curl -i http://127.0.0.1:5001/v2/
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
HTTP 200	Registry API healthy.	Create kind config and registry mirror.
Connection refused	Container stopped or port mapping missing.	docker logs procureflow-registry; docker inspect port mappings.
Port occupied exit	Safety gate worked.	Identify owner; do not bypass randomly.
HTTP 500	Registry internal storage/problem.	Inspect docker logs and recreate registry volume/container if disposable.

10. kind cluster design

Two-node local cluster ek control plane aur ek worker use karega. Yeh production high availability nahi, lekin scheduling role separation, node inspection aur worker failure simulation ke liye enough hai. 16 GB se kam available memory ho to one-node config use karein.

Step 10.1 - kind cluster configuration create karein

Previous evidence	Local registry healthy hai.
Why this step now	Cluster creation se pehle topology aur registry mirror configuration file version-controlled honi chahiye. CLI-only flags later reproduce karna mushkil hota hai.
Action	Run the copy-paste block below.
Expected evidence	Valid YAML show ho with control-plane, worker and containerd config_path.
Next connection	Topology file ready hone ke baad cluster lifecycle script registry network and mirror configuration apply karegi.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/kind/kind-config.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    extraPortMappings:
      - containerPort: 30080
        hostPort: 8080
        listenAddress: "127.0.0.1"
        protocol: TCP
      - containerPort: 30443
        hostPort: 8443
        listenAddress: "127.0.0.1"
        protocol: TCP
  - role: worker
containerdConfigPatches:
  - |-
    [plugins."io.containerd.grpc.v1.cri".registry]
      config_path = "/etc/containerd/certs.d"
EOF

cat platform/kind/kind-config.yaml
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
YAML correct	Topology declared.	Create cluster script.
Need minimal profile	Two nodes may consume extra resources.	Remove worker role and use one control-plane node for learning; document limitation.
Host ports already used	8080/8443 conflict.	Identify process or choose consistent alternate ports; update Gateway exposure plan.

Step 10.2 - Create cluster and connect registry

Previous evidence	Registry is healthy and kind config exists.
Why this step now	kind cluster ko registry container ke same Docker network par connect karna aur node containerd ko localhost:5001 requests registry container tak redirect karna hoga.
Action	Run the copy-paste block below.
Expected evidence	Two nodes Ready hon, current context kind-procureflow ho, aur each node hosts.toml registry redirect show kare.
Next connection	Cluster existence alone sufficient nahi; next isolated test proves artifact travels from host through registry into pod.

```
cd ~/projects/procureflow-devops-modernization

cat > scripts/20-kind-up.sh <<'EOF'
#!/usr/bin/env bash
set -Eeuo pipefail

CLUSTER_NAME=procureflow
REG_NAME=procureflow-registry
REG_PORT=5001
CONFIG=platform/kind/kind-config.yaml

./scripts/00-preflight.sh
./scripts/10-registry-up.sh

if kind get clusters | grep -qx "$CLUSTER_NAME"; then
  echo "kind cluster ${CLUSTER_NAME} already exists"
else
  kind create cluster \
    --name "$CLUSTER_NAME" \
    --config "$CONFIG" \
    --wait 5m
fi

if ! docker inspect "$REG_NAME" \
  --format '{{json .NetworkSettings.Networks.kind}}' 2>/dev/null \
  | grep -q 'IPAddress'; then
  docker network connect kind "$REG_NAME" 2>/dev/null || true
fi

for node in $(kind get nodes --name "$CLUSTER_NAME"); do
  REGISTRY_DIR="/etc/containerd/certs.d/localhost:${REG_PORT}"
  docker exec "$node" mkdir -p "$REGISTRY_DIR"
  cat <<HOSTS | docker exec -i "$node" tee "$REGISTRY_DIR/hosts.toml" >/dev/null
[host."http://${REG_NAME}:5000"]
  capabilities = ["pull", "resolve", "push"]
HOSTS
  docker exec "$node" cat "$REGISTRY_DIR/hosts.toml"
done

cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: local-registry-hosting
  namespace: kube-public
data:
  localRegistryHosting.v1: |
    host: "localhost:${REG_PORT}"
    help: "https://kind.sigs.k8s.io/docs/user/local-registry/"
EOF

kubectl cluster-info --context "kind-${CLUSTER_NAME}"
kubectl get nodes -o wide
EOF

chmod +x scripts/20-kind-up.sh
./scripts/20-kind-up.sh
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Both nodes Ready	Cluster and registry network ready.	Prove image push/pull.
Cluster creation timeout	Docker resources, image download or kubeadm issue.	kind export logs ./artifacts/kind-create; inspect disk/memory/network.
Node NotReady	CNI/kubelet initialization incomplete.	kubectl describe node; docker logs node; wait briefly then investigate conditions.
Registry network connect error already exists	Container already attached.	Safe to continue if docker network inspect kind shows registry.

Step 10.3 - Registry end-to-end smoke test

Previous evidence	kind nodes Ready hain and registry mirror configured hai.
Why this step now	Application build se pehle tiny known image use karke push/pull path isolate karenge. Failure aaye to code or Helm ko blame nahi karenge.
Action	Run the copy-paste block below.
Expected evidence	Pod log REGISTRY_PATH_OK show kare; events mein

	successful pull/start ho.
Next connection	Artifact path proven hone ke baad Kubernetes platform namespaces and storage safely configure ho sakte hain.

```
cd ~/projects/procureflow-devops-modernization

TEST_TAG="localhost:5001/procureflow/hello:registry-smoke"

docker pull busybox:1.36.1
docker tag busybox:1.36.1 "$TEST_TAG"
docker push "$TEST_TAG"

kubectl delete pod registry-smoke --ignore-not-found
kubectl run registry-smoke \
  --image="$TEST_TAG" \
  --restart=Never \
  --command -- sh -c 'echo REGISTRY_PATH_OK && sleep 5'

kubectl wait --for=condition=Ready pod/registry-smoke --timeout=90s
kubectl logs registry-smoke
kubectl describe pod registry-smoke | sed -n '/Events:/,$p'
kubectl delete pod registry-smoke
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
REGISTRY_PATH_OK	Host -> registry -> kind image path proven.	Create namespaces and baseline policies.
ImagePullBackOff	Mirror file, network, tag, or registry health issue.	curl registry catalog; docker exec node cat hosts.toml; docker network inspect kind.
ErrImagePull localhost connection refused	Node tried its own loopback without mirror.	Verify config_path and exact directory /etc/containerd/certs.d/localhost:5001.
Pod Completed before Ready wait	Short command exited normally.	Use kubectl wait --for=jsonpath or inspect phase/log; success remains valid if exit code 0.

11. Namespace and environment model

Development, staging aur production ko separate clusters ke bajaye local namespaces mein model karenge. Yeh blast-radius and configuration separation demonstrate karta hai, lekin interview mein explicitly batayen ke actual production normally stronger account/subscription/cluster isolation demand kar sakti hai.

Step 11.1 - Environment namespaces create karein

Previous evidence	Registry pull path works.
Why this step now	Application deployment se pehle environment boundaries create karni hain taa ke Helm releases, quotas aur policies predictable scope mein hon.
Action	Run the copy-paste block below.
Expected evidence	Three ProcureFlow namespaces and platform-system Active show hon.
Next connection	Boundaries exist; ab resource starvation ko limit karne ke liye quotas define hongi.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/kubernetes/base/namespaces.yaml <<'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: procureflow-dev
  labels:
    app.kubernetes.io/part-of: procureflow
    environment: dev
---
apiVersion: v1
kind: Namespace
metadata:
  name: procureflow-staging
  labels:
    app.kubernetes.io/part-of: procureflow
    environment: staging
---
apiVersion: v1
kind: Namespace
metadata:
  name: procureflow-prod
  labels:
    app.kubernetes.io/part-of: procureflow
    environment: prod
---
apiVersion: v1
kind: Namespace
metadata:
  name: platform-system
  labels:
    app.kubernetes.io/part-of: procureflow-platform
EOF

kubectl apply -f platform/kubernetes/base/namespaces.yaml
kubectl get namespaces -L environment
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Namespaces Active	Environment boundaries ready.	Add resource quotas.
Namespace Terminating	Old finalizer/resource issue.	Inspect namespace finalizers; do not force-delete without understanding.
AlreadyExists/apply unchanged	Idempotent desired state working.	Continue.

Step 11.2 - Local resource quotas add karein

Previous evidence	Environment namespaces exist.
Why this step now	One faulty environment poore laptop ki memory consume na kare. Quota failure also teaches capacity planning and request/limit discipline.
Action	Run the copy-paste block below.
Expected evidence	Quota and defaults display hon with Used values initially low/zero.
Next connection	Compute boundaries set hone ke baad stateful workloads ke liye storage path verify hoga.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/kubernetes/base/resource-quotas.yaml <<'EOF'
apiVersion: v1
kind: ResourceQuota
metadata:
  name: environment-budget
  namespace: procureflow-dev
spec:
  hard:
    requests.cpu: "3"
    requests.memory: 4Gi
    limits.cpu: "6"
    limits.memory: 8Gi
    pods: "40"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: container-defaults
  namespace: procureflow-dev
spec:
  limits:
    - type: Container
      defaultRequest:
        cpu: 100m
        memory: 128Mi
      default:
        cpu: 500m
        memory: 512Mi
EOF

kubectl apply -f platform/kubernetes/base/resource-quotas.yaml
kubectl describe resourcequota environment-budget -n procureflow-dev
kubectl describe limitrange container-defaults -n procureflow-dev
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Quota shown	Resource budget enforced.	Verify storage class.
Pod later Forbidden: exceeded quota	Requests/limits or namespace capacity insufficient.	Right-size workload; do not delete quota as first reaction.
No LimitRange in staging/prod	Intentional initial dev-only scope.	Copy tuned policy after dev evidence.

Step 11.3 - Default storage class verify karein

Previous evidence	Namespace resource policies exist.
Why this step now	PostgreSQL/MySQL deployment se pehle dynamic volume provisioning prove karna hoga. Storage unavailable ho to stateful pods Pending rahenge.
Action	Run the copy-paste block below.
Expected evidence	STORAGE_OK log ho; PVC Bound and dynamically created PV visible ho.
Next connection	Storage proven hone ke baad HPA prerequisite Metrics API install karenge.

```
kubectl get storageclass
kubectl get storageclass -o jsonpath='{range .items[*]}{.metadata.name}{" default="}
{.metadata.annotations.storageclass\.kubernetes\.io/is-default-class}{" provisioner="}{.provisioner}{"
"}{end}'

cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: storage-smoke
  namespace: procureflow-dev
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 128Mi
EOF

cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: storage-smoke
  namespace: procureflow-dev
spec:
  containers:
    - name: writer
      image: busybox:1.36.1
      command: ["sh", "-c", "echo STORAGE_OK > /data/result && cat /data/result && sleep 30"]
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: storage-smoke
EOF

kubectl wait --for=condition=Ready pod/storage-smoke -n procureflow-dev --timeout=120s
kubectl logs storage-smoke -n procureflow-dev
kubectl get pvc,pv -n procureflow-dev
kubectl delete pod storage-smoke -n procureflow-dev
kubectl delete pvc storage-smoke -n procureflow-dev
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
PVC Bound	Dynamic storage path works.	Install Metrics Server.
PVC Pending	No/default storage class or provisioner problem.	kubectl describe pvc; inspect local-path provisioner pods.
Pod mount error	Volume provisioning or node path issue.	Inspect events and provisioner logs before databases.

12. Metrics Server for HPA evidence

Step 12.1 - Metrics Server install karein

Previous evidence	Cluster compute and storage foundations work.
Why this step now	HPA CPU metrics ke baghair scale decision nahi le sakta. Application se pehle Metrics API install karna later autoscaling errors ko isolate karta hai.
Action	Run the copy-paste block below.
Expected evidence	kubectl top nodes CPU and memory values return kare.
Next connection	HPA prerequisite complete hai; now north-south traffic entry build karenge.

```
cd ~/projects/procureflow-devops-modernization

helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm repo update

cat > platform/kubernetes/base/metrics-server-values.yaml <<'EOF'
args:
  - --kubelet-insecure-tls
  - --kubelet-preferred-address-types=InternalIP,Hostname,ExternalIP
resources:
  requests:
    cpu: 50m
    memory: 100Mi
  limits:
    cpu: 200m
    memory: 300Mi
EOF

helm upgrade --install metrics-server metrics-server/metrics-server \
  --namespace kube-system \
  --values platform/kubernetes/base/metrics-server-values.yaml \
  --wait \
  --timeout 5m

kubectl rollout status deployment/metrics-server -n kube-system --timeout=180s
kubectl top nodes
kubectl top pods -A | head -20
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Metrics values shown	resource metrics API works.	Install Gateway API/Envoy.
Metrics not available yet	Scrape warm-up or kubelet TLS/address issue.	Wait 30-60 seconds; inspect metrics-server logs.
Helm install timeout	Pod readiness or image pull issue.	kubectl describe pod -n kube-system -l app.kubernetes.io/name=metrics-server.

Part IV - Gateway API and Envoy Gateway traffic entry

13. Why Gateway API instead of a legacy Ingress-only design

Gateway API routing infrastructure aur application routes ko separate resources mein model karta hai. Cluster/platform team Gateway own kar sakti hai aur application team HTTPRoute own kar sakti hai. Yeh role separation GEP jaisi virtual/global team environment mein clearer ownership demonstrate karti hai.

```
Platform engineer owns
GatewayClass + Gateway
          |
          | permits route attachment
          v
Application team owns
HTTPRoute
          |
          v
Kubernetes Service
          |
          v
Application Pods
```

Previous evidence	Cluster health, storage and metrics proven hain; applications abhi external entry ke baghair honge.
Why this step now	App deploy karne se pehle routing API and controller install karenge taa ke service exposure pattern consistent ho.
Action	Envoy Gateway Helm chart install karein, GatewayClass acceptance verify karein, then a smoke service route karein.
Expected evidence	Gateway Programmed condition True aur HTTPRoute Accepted/ResolvedRefs True show kare.
Next connection	Only then ProcureFlow frontend/API routes add honge.

Step 13.1 - Envoy Gateway v1.8.1 install karein

Previous evidence	Metrics API works.
Why this step now	Gateway API CRDs sirf resource schema dete hain; Envoy Gateway controller resources ko actual proxy deployment mein reconcile karta hai.
Action	Run the copy-paste block below.
Expected evidence	Envoy Gateway deployment Available ho aur Gateway API CRDs listed hon.
Next connection	Controller ready hone ke baad listener and route ownership resources define karenge.

```
cd ~/projects/procureflow-devops-modernization
source .tool-versions.env

helm upgrade --install eg \
  oci://docker.io/envoyproxy/gateway-helm \
  --version "$ENVOY_GATEWAY_VERSION" \
  --namespace envoy-gateway-system \
  --create-namespace \
  --wait \
  --timeout 10m

kubectl wait \
  --timeout=5m \
  --namespace envoy-gateway-system \
  deployment/envoy-gateway \
  --for=condition=Available

kubectl get pods -n envoy-gateway-system
kubectl get crd | grep gateway.networking.k8s.io
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Controller Available and CRDs present	Gateway controller ready.	Create GatewayClass/Gateway.
OCI pull error	Docker Hub/OCI access or version issue.	helm pull command test; verify pinned

Observation	Meaning	Next decision
		version from official release.
CRD conflict	Existing incompatible Gateway API version.	helm status and CRD versions inspect; do not force overwrite blindly.
Pod CrashLoopBackOff	Controller configuration/runtime issue.	kubectl logs deployment/envoy-gateway -n envoy-gateway-system.

Step 13.2 - GatewayClass and local Gateway create karein

Previous evidence	Envoy Gateway controller Available hai.
Why this step now	Controller ke baghair GatewayClass accepted nahi hoti; ab platform entry point declare kar sakte hain. NodePort listener kind host port 8080 se expose hoga.
Action	Run the copy-paste block below.
Expected evidence	GatewayClass Accepted=True; Gateway Programmed=True; generated Envoy service exists.
Next connection	Gateway exists; now its generated service must match kind extraPortMappings.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/kubernetes/gateway/gateway.yaml <<'EOF'
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: envoy-gateway
spec:
  controllerName: gateway.envoyproxy.io/gatewayclass-controller
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: procureflow-gateway
  namespace: procureflow-dev
spec:
  gatewayClassName: envoy-gateway
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      allowedRoutes:
        namespaces:
          from: Same
EOF

kubectl apply -f platform/kubernetes/gateway/gateway.yaml
kubectl wait gatewayclass/envoy-gateway \
  --for=condition=Accepted \
  --timeout=180s
kubectl wait gateway/procureflow-gateway \
  -n procureflow-dev \
  --for=condition=Programmed \
  --timeout=300s

kubectl get gatewayclass envoy-gateway -o yaml
kubectl get gateway procureflow-gateway -n procureflow-dev -o wide
kubectl get svc -A | grep -E 'envoy|procureflow-gateway' || true
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Accepted and Programmed True	Proxy infrastructure reconciled.	Patch generated service to required NodePorts.
Gateway Accepted but not Programmed	Infrastructure deployment/service pending.	Describe Gateway conditions; inspect generated Envoy pods/services.
Invalid controllerName	GatewayClass not claimed.	Compare official Envoy controller name and installed version docs.
No generated service	Controller reconciliation failed.	Check Envoy Gateway logs and Gateway events.

Step 13.3 - Map Envoy service to kind host ports

Previous evidence	Gateway is Programmed and generated Envoy service exists.
Why this step now	kind host port 8080 maps to control-plane container port 30080. Generated Envoy service ko stable NodePort 30080 dena hoga taa ke browser/curl localhost:8080 reach kar sake.
Action	Run the copy-paste block below.
Expected evidence	Service type NodePort and port 30080 show. Initial curl may return 404 because no HTTPRoute exists; network reachability is still proven.
Next connection	Proxy reachability proven; next route acceptance and backend resolution prove complete Layer 7 flow.

```
GATEWAY_SVC=$(kubectl get svc -n envoy-gateway-system \
  -l gateway.envoyproxy.io/owning-gateway-name=procureflow-gateway \
  -o jsonpath='{.items[0].metadata.name}')

echo "Generated service: ${GATEWAY_SVC}"

kubectl patch service "$GATEWAY_SVC" \
  -n envoy-gateway-system \
  --type='json' \
  -p='[
    {"op":"replace","path":"/spec/type","value":"NodePort"},
    {"op":"add","path":"/spec/ports/0/nodePort","value":30080}
  ]'

kubectl get service "$GATEWAY_SVC" -n envoy-gateway-system -o wide
curl -i --max-time 5 http://127.0.0.1:8080/ || true
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
HTTP 404 from Envoy	Host -> kind port -> Envoy path works; route missing as expected.	Create smoke backend and HTTPRoute.
Connection refused	NodePort patch or kind port mapping not effective.	docker port procureflow-control-plane; inspect service nodePort.
Label returns no service	Generated labels changed/version mismatch.	kubectl get svc -n envoy-gateway-system --show-labels and select correct owning Gateway.
JSON patch path error	Generated service port structure differs.	kubectl get svc -o yaml, then patch exact port index/name.

Step 13.4 - Gateway smoke application and HTTPRoute

Previous evidence	Envoy responds on localhost:8080 but no application route exists.
Why this step now	A tiny backend isolates Gateway API routing before ProcureFlow code is introduced. This prevents application bugs from hiding routing errors.
Action	Run the copy-paste block below.
Expected evidence	curl exactly GATEWAY_ROUTE_OK return kare; HTTPRoute Accepted True and ResolvedRefs True ho.
Next connection	Traffic platform now stable; application code can be added without mixing infrastructure uncertainty.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/kubernetes/gateway/smoke-route.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gateway-smoke
  namespace: procureflow-dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: gateway-smoke
  template:
    metadata:
      labels:
        app: gateway-smoke
    spec:
      containers:
        - name: echo
          image: hashicorp/http-echo:1.0
          args: ["-text=GATEWAY_ROUTE_OK"]
          ports:
            - containerPort: 5678
---
apiVersion: v1
kind: Service
metadata:
  name: gateway-smoke
  namespace: procureflow-dev
spec:
  selector:
    app: gateway-smoke
  ports:
    - port: 80
      targetPort: 5678
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: gateway-smoke
  namespace: procureflow-dev
spec:
  parentRefs:
    - name: procureflow-gateway
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /smoke
      backendRefs:
        - name: gateway-smoke
          port: 80
EOF

kubectl apply -f platform/kubernetes/gateway/smoke-route.yaml
kubectl rollout status deployment/gateway-smoke -n procureflow-dev --timeout=180s
kubectl wait httproute/gateway-smoke -n procureflow-dev \
  --for=condition=Accepted \
  --timeout=180s

kubectl get httproute gateway-smoke -n procureflow-dev -o yaml
curl -fsS http://127.0.0.1:8080/smoke
echo
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
GATEWAY_ROUTE_OK	Complete north-south traffic path proven.	Build ProcureFlow services.
HTTP 404	Route not attached/match mismatch.	Inspect HTTPRoute status parents and Gateway allowedRoutes.
HTTP 503	Route matched but backend endpoints unavailable.	kubectl get endpoints; check pod readiness and service selector.
ResolvedRefs False	Backend service/port reference invalid.	Correct service name/port in HTTPRoute.

Step 13.5 - Platform checkpoint commit

Previous evidence	Registry, cluster, storage, metrics and Gateway smoke tests pass.
Why this step now	This is first complete platform baseline. Application

	development se pehle commit creates a clean rollback boundary.
Action	Run the copy-paste block below.
Expected evidence	Commit visible and working tree clean.
Next connection	Application layer ab proven platform par build hogi.

```
cd ~/projects/procureflow-devops-modernization
git add scripts platform/kind platform/kubernetes
git commit -m "feat(platform): add local registry kind cluster metrics and Gateway API"
git log --oneline -3
git status --short
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Clean commit	Platform baseline recoverable.	Start Supplier Service.
Untracked evidence files	Evidence may need intentional location.	Move safe outputs to docs/evidence; never commit secrets.

Part V - Build the application services from health to business flow

14. Why health and metrics come before business features

Kubernetes application ko sirf process running nahi, readiness, liveness and metrics contracts chahiye. Isliye first service minimal business endpoint ke saath operational endpoints expose karegi. Yeh pattern baad ke Java aur Node.js services ke liye template banega.

```
Container process starts
        |
        v
/startup or /health/live succeeds
        |
        v
/health/ready verifies dependencies
        |
        v
Service receives traffic
        |
        v
/metrics exposes operational evidence
```

15. Node.js Supplier Service

Step 15.1 - Supplier Service source create karein

Previous evidence	Gateway and cluster platform are proven.
Why this step now	First app intentionally simple hai: process health, in-memory supplier endpoint, Prometheus metric and controlled CPU work. Isse container, probes, route, metrics and HPA later test ho sakte hain.
Action	Run the copy-paste block below.
Expected evidence	npm test pass ho and package-lock.json create ho.
Next connection	Source contract validated; now immutable container image banegi.

```
cd ~/projects/procureflow-devops-modernization/applications/supplier-service-nodejs

cat > package.json <<'EOF'
{
  "name": "procureflow-supplier-service",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "start": "node src/server.js",
    "test": "node --test"
  },
  "dependencies": {
    "express": "4.21.2",
    "prom-client": "15.1.3"
  }
}
EOF

mkdir -p src test

cat > src/server.js <<'EOF'
import express from 'express';
import client from 'prom-client';

const app = express();
const port = Number(process.env.PORT || 8080);
const serviceVersion = process.env.SERVICE_VERSION || 'dev';

app.use(express.json());
client.collectDefaultMetrics({ prefix: 'procureflow_supplier_' });

const requestCounter = new client.Counter({
  name: 'procureflow_supplier_http_requests_total',
  help: 'Total Supplier API requests',
  labelNames: ['route', 'status']
});

const suppliers = [
  { id: 1, name: 'Nordic Components', status: 'ACTIVE' },
  { id: 2, name: 'Baltic Logistics', status: 'REVIEW' }
];

app.get('/health/live', (_req, res) => res.json({ status: 'UP' }));
app.get('/health/ready', (_req, res) => res.json({ status: 'READY', version: serviceVersion }));

app.get('/api/suppliers', (req, res) => {
  const workMs = Math.min(Number(req.query.work || 0), 250);
  const until = Date.now() + workMs;
  while (Date.now() < until) Math.sqrt(Math.random());
  requestCounter.inc({ route: '/api/suppliers', status: '200' });
  res.json({ serviceVersion, count: suppliers.length, suppliers });
});

app.get('/metrics', async (_req, res) => {
  res.set('Content-Type', client.register.contentType);
  res.end(await client.register.metrics());
});

app.listen(port, '0.0.0.0', () => {
  console.log(`supplier-service listening on ${port}, version=${serviceVersion}`);
});
EOF

cat > test/health.test.js <<'EOF'
import test from 'node:test';
import assert from 'node:assert/strict';

test('basic arithmetic proves test runner works', () => {
  assert.equal(2 + 2, 4);
});
EOF

cat > Dockerfile <<'EOF'
FROM node:22.14.0-alpine AS dependencies
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev && npm cache clean --force

FROM node:22.14.0-alpine
ENV NODE_ENV=production
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY --from=dependencies --chown=app:app /app/node_modules ./node_modules
COPY --chown=app:app src ./src
COPY --chown=app:app package.json ./package.json
USER app
```

```
EXPOSE 8080
CMD ["node", "src/server.js"]
EOF

cat > .dockerignore <<'EOF'
node_modules
npm-debug.log
coverage
.git
EOF

npm install
npm test
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Tests pass	Source and dependency lock ready.	Build container.
npm not found	Node.js local CLI unavailable.	Use Docker build-only workflow or install Node LTS via nvm.
Dependency resolution error	Network or incompatible package version.	Inspect npm error; keep lock file after successful resolution.
Test fails	Do not build image.	Fix source/test until green.

Step 15.2 - Build, run, and push Supplier Service image

Previous evidence	Supplier unit test passes.
Why this step now	Container ko registry push se pehle local process, non-root user, health and metrics endpoints validate karne hain. Yeh image-level gate hai.
Action	Run the copy-paste block below.
Expected evidence	Health READY, supplier JSON, metric, non-root uid and successful push digest show hon.
Next connection	Registry now contains first real artifact; Helm can describe how Kubernetes runs it.

```
cd ~/projects/procureflow-devops-modernization/applications/supplier-service-nodejs

IMAGE="localhost:5001/procureflow/supplier-service:1.0.0"

docker build -t "$IMAGE" .
docker rm -f supplier-service-local 2>/dev/null || true
docker run -d --name supplier-service-local -p 18081:8080 "$IMAGE"

for i in {1..30}; do
  if curl -fsS http://127.0.0.1:18081/health/ready; then break; fi
  sleep 1
done

echo
curl -fsS http://127.0.0.1:18081/api/suppliers | jq .
curl -fsS http://127.0.0.1:18081/metrics | grep procureflow_supplier_http_requests_total | head

docker exec supplier-service-local id
docker logs supplier-service-local
docker rm -f supplier-service-local

docker push "$IMAGE"
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
All checks pass	Image runtime contract valid.	Create Kubernetes manifests/Helm chart.
Container exits	Application startup or module error.	docker logs; run image interactively if needed.
id shows uid 0 root	Dockerfile security contract failed.	Verify USER app and image layer.
Metrics empty	Request counter not invoked or endpoint error.	Call /api/suppliers first, then recheck metrics.
Push denied/connection refused	Registry path unhealthy.	Return to registry smoke; do not modify app image.

16. Java Order Service

Order Service Java/JVM operational ownership demonstrate karegi. Initial version database ke baghair memory mein order create karegi aur Actuator metrics expose karegi. PostgreSQL integration next phase mein add hogi; is separation se application runtime aur database dependency failures isolate rehte hain.

Step 16.1 - Spring Boot Order Service source create karein

Previous evidence	Supplier image successfully pushed.
Why this step now	Second language/runtime add karne se pipeline multi-stack capability prove hogi. Health, metrics and structured order endpoint pehle validate honge, phir database/event dependencies attach hongi.
Action	Run the copy-paste block below.
Expected evidence	Spring context test BUILD SUCCESS show kare.
Next connection	Java test green hone ke baad image runtime health validate hoga.

```
cd ~/projects/procureflow-devops-modernization/applications/order-service-java

mkdir -p src/main/java/com/procureflow/order \
  src/main/resources \
  src/test/java/com/procureflow/order

cat > pom.xml <<'EOF'
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.procureflow</groupId>
  <artifactId>order-service</artifactId>
  <version>1.0.0</version>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.0</version>
    <relativePath/>
  </parent>
  <properties>
    <java.version>21</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
EOF

cat > src/main/java/com/procureflow/order/OrderApplication.java <<'EOF'
package com.procureflow.order;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class OrderApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderApplication.class, args);
    }
}
EOF

cat > src/main/java/com/procureflow/order/OrderController.java <<'EOF'
package com.procureflow.order;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.time.Instant;
import java.util.Map;
import java.util.UUID;

@RestController
@RequestMapping("/api/orders")
public class OrderController {
    private final Counter createdCounter;

    public OrderController(MeterRegistry registry) {
        this.createdCounter = Counter.builder("procureflow.orders.created")
                .description("Orders created")
```

```
                .register(registry);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Map<String, Object> create(@RequestBody Map<String, Object> request) {
        createdCounter.increment();
        return Map.of(
                "orderId", UUID.randomUUID().toString(),
                "status", "CREATED",
                "supplierId", request.getOrDefault("supplierId", "unknown"),
                "amount", request.getOrDefault("amount", 0),
                "createdAt", Instant.now().toString()
        );
    }
}
EOF

cat > src/main/resources/application.yaml <<'EOF'
server:
  port: 8080
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
  endpoint:
    health:
      probes:
        enabled: true
  health:
    livenessstate:
      enabled: true
    readinessstate:
      enabled: true
info:
  app:
    name: order-service
    version: ${SERVICE_VERSION:dev}
EOF

cat > src/test/java/com/procureflow/order/OrderApplicationTests.java <<'EOF'
package com.procureflow.order;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class OrderApplicationTests {
    @Test
    void contextLoads() {}
}
EOF

cat > Dockerfile <<'EOF'
FROM maven:3.9.9-eclipse-temurin-21 AS build
WORKDIR /workspace
COPY pom.xml ./
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY --from=build --chown=app:app /workspace/target/order-service-1.0.0.jar app.jar
USER app
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=70.0", "-jar", "/app/app.jar"]
EOF

cat > .dockerignore <<'EOF'
target
.git
.idea
EOF

# Run tests through Maven container so local Maven installation is optional
docker run --rm \
  -v "$PWD:/workspace" \
  -w /workspace \
  maven:3.9.9-eclipse-temurin-21 \
  mvn -B test
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
BUILD SUCCESS	Java source/runtime baseline valid.	Build image.
Dependency download timeout	Network/Maven Central issue.	Retry after network validation; use local Maven cache mount if needed.
Context load failure	Spring configuration or code issue.	Read test stack trace; do not skip failing tests.
Java version mismatch	Container tag or pom property inconsistent.	Keep Java 21 across build/runtime/chart values.

Step 16.2 - Build, verify and push Order Service image

Previous evidence	Java tests pass.
Why this step now	Image ko Kubernetes se pehle local Docker par verify karna runtime layer isolate karta hai. Actuator readiness and Prometheus endpoints required evidence hain.
Action	Run the copy-paste block below.
Expected evidence	Readiness UP, HTTP 201 order JSON, metric value >=1, non-root user and image push digest show hon.
Next connection	Both Node.js and Java images available; now one reusable Helm chart can package them.

```
cd ~/projects/procureflow-devops-modernization/applications/order-service-java

IMAGE="localhost:5001/procureflow/order-service:1.0.0"

docker build -t "$IMAGE" .
docker rm -f order-service-local 2>/dev/null || true
docker run -d --name order-service-local -p 18082:8080 "$IMAGE"

for i in {1..60}; do
  if curl -fsS http://127.0.0.1:18082/actuator/health/readiness; then break; fi
  sleep 1
done

echo
curl -fsS -X POST http://127.0.0.1:18082/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"supplierId":"SUP-100","amount":1250}' | jq .

curl -fsS http://127.0.0.1:18082/actuator/prometheus \
  | grep 'procureflow_orders_created_total'

docker exec order-service-local id
docker logs --tail=50 order-service-local
docker rm -f order-service-local

docker push "$IMAGE"
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
All validation passes	Java artifact ready.	Create Helm chart.
Readiness timeout	Spring process failed or slow.	docker logs; check port and application.yaml.
HTTP 500 on POST	Controller serialization/request issue.	Inspect logs and response body.
Prometheus metric absent	Order call not successful or metric naming differs.	Search actuator output for procureflow_orders.
Root user	Runtime security issue.	Verify USER app and copied file permissions.

Part VI - Helm packaging and multi-environment deployment

17. Why Helm comes after validated images

Helm image ko fix nahi karta; Helm Kubernetes desired state package karta hai. Agar image independently run nahi karti, Helm debugging two variables mix kar dega. Ab dono images local Docker aur registry par validated hain, isliye deployment packaging meaningful hai.

```
Validated image
      |
      v
Helm values define runtime differences
      |
      v
Templates produce Kubernetes manifests
      |
      v
helm template validates rendering
      |
      v
server-side dry run validates API
      |
      v
helm upgrade --install changes live state
```

18. Reusable ProcureFlow Helm chart

Step 18.1 - Chart metadata and default values create karein

Previous evidence	Supplier and Order images are in local registry.
Why this step now	One chart multiple services ko values-driven pattern se deploy karega. Defaults safe honge; environment-specific values only differences override karengi.
Action	Run the copy-paste block below.
Expected evidence	helm lint zero failures return kare.
Next connection	Values contract ready; templates can convert this contract into Deployments, Services and Routes.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/helm/procureflow/Chart.yaml <<'EOF'
apiVersion: v2
name: procureflow
version: 0.1.0
appVersion: "1.0.0"
description: ProcureFlow Java and Node.js microservices
 type: application
EOF

# Remove accidental leading space before type if copied from formatted text
sed -i 's/^ type:/type:/' platform/helm/procureflow/Chart.yaml

cat > platform/helm/procureflow/values.yaml <<'EOF'
global:
  registry: localhost:5001/procureflow
  environment: dev
  gatewayName: procureflow-gateway

services:
  supplier:
    enabled: true
    name: supplier-service
    image:
      repository: supplier-service
      tag: "1.0.0"
      pullPolicy: IfNotPresent
    replicas: 1
    port: 8080
    routePath: /api/suppliers
    health:
      liveness: /health/live
      readiness: /health/ready
    metrics:
      enabled: true
      path: /metrics
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        cpu: 500m
        memory: 256Mi
    env:
      SERVICE_VERSION: "1.0.0"

  order:
    enabled: true
    name: order-service
    image:
      repository: order-service
      tag: "1.0.0"
      pullPolicy: IfNotPresent
    replicas: 1
    port: 8080
    routePath: /api/orders
    health:
      liveness: /actuator/health/liveness
      readiness: /actuator/health/readiness
    metrics:
      enabled: true
      path: /actuator/prometheus
    resources:
      requests:
        cpu: 150m
        memory: 256Mi
      limits:
        cpu: 700m
        memory: 512Mi
    env:
      SERVICE_VERSION: "1.0.0"

hpa:
  enabled: false
  minReplicas: 1
  maxReplicas: 4
  targetCPUUtilizationPercentage: 60

podDisruptionBudget:
  enabled: false
  minAvailable: 1
EOF

helm lint platform/helm/procureflow
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Lint success	Metadata and values syntax valid.	Create templates.
YAML parse error	Indentation or accidental formatting issue.	Use nl -ba file; fix exact line.
Chart.yaml missing type	Metadata malformed.	Verify sed correction and required fields.

Step 18.2 - Helper, Deployment and Service templates create karein

Previous evidence	Chart values lint successfully.
Why this step now	Templates service map ko iterate karengi. Same operational baseline - non-root, probes, resources and labels - every microservice par consistently apply hoga.
Action	Run the copy-paste block below.
Expected evidence	Two Deployments and two Services render hon; lint pass ho.
Next connection	Workloads and Services exist in rendered state; external traffic still needs routes.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/helm/procureflow/templates/_helpers.tpl <<'EOF'
{{- define "procureflow.labels" -}}
app.kubernetes.io/part-of: procureflow
app.kubernetes.io/managed-by: {{ .Release.Service }}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
environment: {{ .Values.global.environment | quote }}
{{- end }}
EOF

cat > platform/helm/procureflow/templates/deployments.yaml <<'EOF'
{{- range $key, $svc := .Values.services }}
{{- if $svc.enabled }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ $svc.name }}
  labels:
    {{- include "procureflow.labels" $ | nindent 4 }}
    app.kubernetes.io/name: {{ $svc.name }}
spec:
  replicas: {{ $svc.replicas }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ $svc.name }}
  template:
    metadata:
      labels:
        {{- include "procureflow.labels" $ | nindent 8 }}
        app.kubernetes.io/name: {{ $svc.name }}
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/path: {{ $svc.metrics.path | quote }}
        prometheus.io/port: {{ $svc.port | quote }}
    spec:
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: {{ $svc.name }}
          image: "{{ $.Values.global.registry }}/{{ $svc.image.repository }}:{{ $svc.image.tag }}"
          imagePullPolicy: {{ $svc.image.pullPolicy }}
          ports:
            - name: http
              containerPort: {{ $svc.port }}
          env:
            {{- range $name, $value := $svc.env }}
            - name: {{ $name }}
              value: {{ $value | quote }}
            {{- end }}
          readinessProbe:
            httpGet:
              path: {{ $svc.health.readiness }}
              port: http
            initialDelaySeconds: 5
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 12
          livenessProbe:
            httpGet:
              path: {{ $svc.health.liveness }}
              port: http
            initialDelaySeconds: 15
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          resources:
            {{- toYaml $svc.resources | nindent 12 }}
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
            readOnlyRootFilesystem: true
      terminationGracePeriodSeconds: 30
---
{{- end }}
{{- end }}
EOF

cat > platform/helm/procureflow/templates/services.yaml <<'EOF'
{{- range $key, $svc := .Values.services }}
{{- if $svc.enabled }}
apiVersion: v1
kind: Service
metadata:
```

```
  name: {{ $svc.name }}
  labels:
    {{- include "procureflow.labels" $ | nindent 4 }}
    app.kubernetes.io/name: {{ $svc.name }}
spec:
  selector:
    app.kubernetes.io/name: {{ $svc.name }}
  ports:
    - name: http
      port: 80
      targetPort: http
---
{{- end }}
{{- end }}
EOF

helm lint platform/helm/procureflow
helm template procureflow platform/helm/procureflow \
  --namespace procureflow-dev \
  --values platform/helm/procureflow/values.yaml \
  > /tmp/procureflow-rendered.yaml

grep '^kind:' /tmp/procureflow-rendered.yaml | sort | uniq -c
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Expected kinds/counts	Core templates render correctly.	Add HTTPRoutes and HPA.
nil pointer or map error	Values/template key mismatch.	Use helm template --debug and compare values hierarchy.
SecurityContext causes app write failure later	Read-only root filesystem conflicts with runtime temp path.	Mount emptyDir at /tmp rather than disabling security globally.

Step 18.3 - HTTPRoute, HPA and PDB templates add karein

Previous evidence	Deployments and Services render.
Why this step now	Gateway already provides infrastructure entry. Each application now owns its HTTPRoute. HPA and PDB are conditional because local dev and production-like profiles have different needs.
Action	Run the copy-paste block below.
Expected evidence	Rendered output includes two HTTPRoutes; server dry run reports created/configured dry run without schema errors.
Next connection	API-valid chart ready; environment files can now express only intentional differences.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/helm/procureflow/templates/routes.yaml <<'EOF'
{{- range $key, $svc := .Values.services }}
{{- if $svc.enabled }}
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: {{ $svc.name }}
  labels:
    {{- include "procureflow.labels" $ | nindent 4 }}
spec:
  parentRefs:
    - name: {{ $.Values.global.gatewayName }}
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: {{ $svc.routePath }}
      backendRefs:
        - name: {{ $svc.name }}
          port: 80
---
{{- end }}
{{- end }}
EOF

cat > platform/helm/procureflow/templates/hpa.yaml <<'EOF'
{{- if .Values.hpa.enabled }}
{{- range $key, $svc := .Values.services }}
{{- if $svc.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ $svc.name }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ $svc.name }}
  minReplicas: {{ $.Values.hpa.minReplicas }}
  maxReplicas: {{ $.Values.hpa.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ $.Values.hpa.targetCPUUtilizationPercentage }}
---
{{- end }}
{{- end }}
{{- end }}
EOF

cat > platform/helm/procureflow/templates/pdb.yaml <<'EOF'
{{- if .Values.podDisruptionBudget.enabled }}
{{- range $key, $svc := .Values.services }}
{{- if $svc.enabled }}
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: {{ $svc.name }}
spec:
  minAvailable: {{ $.Values.podDisruptionBudget.minAvailable }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ $svc.name }}
---
{{- end }}
{{- end }}
{{- end }}
EOF

helm template procureflow platform/helm/procureflow \
  --namespace procureflow-dev \
  --values platform/helm/procureflow/values.yaml \
  > /tmp/procureflow-rendered.yaml

grep '^kind:' /tmp/procureflow-rendered.yaml | sort | uniq -c
kubectl apply --dry-run=server -f /tmp/procureflow-rendered.yaml
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Server dry run succeeds	Templates match current cluster APIs.	Create environment values.

Observation	Meaning	Next decision
No matches for HTTPRoute	Gateway API CRDs missing/incompatible.	Return to Envoy installation.
PDB/HPA absent	Expected because disabled defaults.	Enable in prod-like values, not by editing templates.
route backend port invalid	Service port mismatch.	Keep backendRefs port equal Service port 80.

Step 18.4 - Dev, staging and production-like values create karein

Previous evidence	Base chart server-side dry run succeeds.
Why this step now	Environment separation ka purpose three copied charts nahi, one chart plus controlled overrides hai. Same image digest/tag promotion drift reduce karta hai.
Action	Run the copy-paste block below.
Expected evidence	Dev lacks HPA/PDB; staging includes HPA; prod includes HPA and PDB.
Next connection	Dev values are safe for first live release.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/helm/procureflow/values-dev.yaml <<'EOF'
global:
  environment: dev
services:
  supplier:
    replicas: 1
    env:
      SERVICE_VERSION: "1.0.0-dev"
  order:
    replicas: 1
    env:
      SERVICE_VERSION: "1.0.0-dev"
hpa:
  enabled: false
podDisruptionBudget:
  enabled: false
EOF

cat > platform/helm/procureflow/values-staging.yaml <<'EOF'
global:
  environment: staging
services:
  supplier:
    replicas: 1
    env:
      SERVICE_VERSION: "1.0.0-staging"
  order:
    replicas: 1
    env:
      SERVICE_VERSION: "1.0.0-staging"
hpa:
  enabled: true
  minReplicas: 1
  maxReplicas: 3
podDisruptionBudget:
  enabled: false
EOF

cat > platform/helm/procureflow/values-prod.yaml <<'EOF'
global:
  environment: prod
services:
  supplier:
    replicas: 2
    env:
      SERVICE_VERSION: "1.0.0"
  order:
    replicas: 2
    env:
      SERVICE_VERSION: "1.0.0"
hpa:
  enabled: true
  minReplicas: 2
  maxReplicas: 4
  targetCPUUtilizationPercentage: 60
podDisruptionBudget:
  enabled: true
  minAvailable: 1
EOF

for values in values-dev.yaml values-staging.yaml values-prod.yaml; do
  echo "== ${values} =="
  helm template procureflow platform/helm/procureflow \
    -f platform/helm/procureflow/values.yaml \
    -f "platform/helm/procureflow/${values}" \
    --namespace procureflow-dev \
    | grep '^kind:' | sort | uniq -c
done
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Expected differences visible	Overrides work without chart duplication.	Deploy dev release.
Service env map replaced unexpectedly	Helm map merge semantics produced missing keys.	Render full manifest; include all required nested override keys or restructure values.
Prod resource exceeds local quota	Production-like profile too heavy for laptop.	Use staging/dev live and render prod offline for evidence.

19. First live Helm deployment

Step 19.1 - Remove smoke route and install ProcureFlow dev release

Previous evidence	Chart is API-valid and dev overrides render correctly.
Why this step now	Smoke route has served its purpose. Real routes can now own /api paths. Helm install records release history and creates rollback point.
Action	Run the copy-paste block below.
Expected evidence	Both deployments Available; pods Running/Ready; services and routes present; Helm status deployed.
Next connection	Live workloads exist; next validate user path, service DNS, probes and resource evidence.

```
cd ~/projects/procureflow-devops-modernization

kubectl delete -f platform/kubernetes/gateway/smoke-route.yaml --ignore-not-found

helm upgrade --install procureflow-dev platform/helm/procureflow \
  --namespace procureflow-dev \
  --values platform/helm/procureflow/values.yaml \
  --values platform/helm/procureflow/values-dev.yaml \
  --wait \
  --timeout 10m \
  --atomic

helm status procureflow-dev -n procureflow-dev
kubectl get deploy,pod,svc,httproute -n procureflow-dev
kubectl rollout status deployment/supplier-service -n procureflow-dev --timeout=180s
kubectl rollout status deployment/order-service -n procureflow-dev --timeout=240s
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Release deployed	Kubernetes runtime contract works.	Validate traffic and internal DNS.
Atomic rollback occurred	One resource failed readiness/install.	helm history; kubectl get events; inspect pod logs and chart output.
Read-only filesystem error	Runtime writes to /tmp or work dir.	Add emptyDir mount for required writable path, keep root filesystem read-only.
ImagePullBackOff	Registry mirror/tag problem.	Registry catalog and node hosts.toml; do not change pullPolicy blindly.

Step 19.2 - End-to-end functional and platform validation

Previous evidence	Helm release status deployed.
Why this step now	A Deployment Available condition alone business function prove nahi karti. External route, internal service discovery, health, metrics and resource usage all validate honge.
Action	Run the copy-paste block below.
Expected evidence	Supplier JSON, order 201 JSON, both internal health responses, route Accepted, endpoints populated and metrics values shown.
Next connection	Functional evidence enables a known-good application checkpoint before databases are introduced.

```
printf '
== External supplier route ==
'
curl -fsS http://127.0.0.1:8080/api/suppliers | jq .

printf '
== External order route ==
'
curl -fsS -X POST http://127.0.0.1:8080/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"supplierId":"SUP-100","amount":1250}' | jq .

printf '
== Internal DNS and service connectivity ==
'
kubectl run tmp-curl \
  -n procureflow-dev \
  --rm -i --restart=Never \
  --image=curlimages/curl:8.12.1 \
  -- sh -c '
    curl -fsS http://supplier-service/health/ready && echo;
    curl -fsS http://order-service/actuator/health/readiness && echo
  '

printf '
== Routes and endpoints ==
'
kubectl get httproute -n procureflow-dev
kubectl get endpoints -n procureflow-dev

printf '
== Resource use ==
'
kubectl top pods -n procureflow-dev
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
All validations pass	First application slice complete.	Record evidence and commit.
External fails, internal works	Gateway/HTTPRoute problem.	Inspect route conditions and Envoy proxy logs.
Internal DNS fails	Service name/namespace or CoreDNS issue.	kubectl get svc/endpoints; nslookup from debug pod.
Endpoint empty	Service selector does not match pod labels or pods unready.	Compare selectors/labels and readiness events.
kubectl top missing	Metrics API issue.	Return to Metrics Server validation.

Step 19.3 - Capture evidence and commit application baseline

Previous evidence	External and internal functional validation passes.
Why this step now	Interview claims must be supported by reproducible evidence. Save non-sensitive outputs and create a Git checkpoint before adding stateful dependencies.
Action	Run the copy-paste block below.
Expected evidence	Commit visible and evidence file contains no secrets.
Next connection	Now state can be introduced from a clean stateless baseline.

```
cd ~/projects/procureflow-devops-modernization

{
  date -Is
  helm status procureflow-dev -n procureflow-dev
  kubectl get deploy,pod,svc,httproute -n procureflow-dev -o wide
  kubectl top pods -n procureflow-dev
} > docs/evidence/01-application-baseline.txt

git add applications platform/helm docs/evidence/01-application-baseline.txt
git commit -m "feat(app): deploy Java and Node.js services with Helm and Gateway API"
git log --oneline -4
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Commit created	Stateless baseline recoverable.	Add PostgreSQL dependency.

Observation	Meaning	Next decision
Evidence contains token/password	Unsafe artifact.	Redact/remove secret before git add; rotate exposed credential if committed.

Part VII - PostgreSQL integration and stateful workload foundations

20. Why state is introduced after a stable stateless slice

Previous evidence	Java and Node.js services deploy, route and expose health successfully without external state.
Why this step now	A database adds storage, credentials, DNS, schema and connection-pool failure modes. Adding it only after stateless validation keeps troubleshooting variables controlled.
Action	Deploy PostgreSQL with a PVC, validate database independently, then modify Order Service to depend on it.
Expected evidence	PostgreSQL pod Ready, PVC Bound, SQL query works, then Order Service readiness and persistence work.
Next connection	Once modern database dependency is understood, legacy MySQL migration can be demonstrated with clearer comparison.

21. PostgreSQL local stateful service

Step 21.1 - Generate PostgreSQL secret outside Git

Previous evidence	Stateless application baseline committed.
Why this step now	Credentials template file mein hardcode nahi karni. Local lab mein secret Kubernetes API mein create hoga; Git only secret name contract store karega.
Action	Run the copy-paste block below.
Expected evidence	Secret exists and only key names are printed; password value terminal output mein na aaye.
Next connection	Credentials available hone ke baad StatefulSet safely reference kar sakta hai.

```
cd ~/projects/procureflow-devops-modernization

POSTGRES_PASSWORD="$(openssl rand -base64 24 | tr -d '
' | tr '/+' '_-')"

kubectl create secret generic order-postgres-credentials \
  -n procureflow-dev \
  --from-literal=POSTGRES_DB=procureflow \
  --from-literal=POSTGRES_USER=procureflow \
  --from-literal=POSTGRES_PASSWORD="$POSTGRES_PASSWORD" \
  --dry-run=client -o yaml \
  | kubectl apply -f -

unset POSTGRES_PASSWORD
kubectl get secret order-postgres-credentials -n procureflow-dev
kubectl get secret order-postgres-credentials -n procureflow-dev -o json \
  | jq -r '.data | keys[]' | sort
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Three keys listed	Credential contract ready.	Deploy PostgreSQL.
Secret manifest printed with base64 values	Command was changed or -o output exposed.	Clear terminal history if necessary; rotate password and avoid saving output.
Namespace not found	Environment foundation missing.	Reapply namespaces before database.

Step 21.2 - PostgreSQL Service and StatefulSet deploy karein

Previous evidence	Credential secret exists.
Why this step now	Stable identity plus persistent volume database restart ke baad

	data preserve karte hain. Readiness pg_isready database acceptance prove karti hai.
Action	Run the copy-paste block below.
Expected evidence	StatefulSet ready 1/1; pod Running/Ready; PVC Bound.
Next connection	Database process ready is not enough; SQL create/read/restart persistence must be proven.

```
cd ~/projects/procureflow-devops-modernization

cat > databases/postgresql/postgresql.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: order-postgres
  namespace: procureflow-dev
spec:
  clusterIP: None
  selector:
    app.kubernetes.io/name: order-postgres
  ports:
    - name: postgres
      port: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: order-postgres
  namespace: procureflow-dev
spec:
  serviceName: order-postgres
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: order-postgres
  template:
    metadata:
      labels:
        app.kubernetes.io/name: order-postgres
        app.kubernetes.io/part-of: procureflow
    spec:
      terminationGracePeriodSeconds: 30
      securityContext:
        fsGroup: 999
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: postgres
          image: postgres:17.5-alpine
          ports:
            - name: postgres
              containerPort: 5432
          envFrom:
            - secretRef:
                name: order-postgres-credentials
          env:
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          readinessProbe:
            exec:
              command: ["sh", "-c", "pg_isready -U $POSTGRES_USER -d $POSTGRES_DB"]
            initialDelaySeconds: 5
            periodSeconds: 5
          livenessProbe:
            exec:
              command: ["sh", "-c", "pg_isready -U $POSTGRES_USER -d $POSTGRES_DB"]
            initialDelaySeconds: 20
            periodSeconds: 10
          resources:
            requests:
              cpu: 100m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 2Gi
EOF

kubectl apply -f databases/postgresql/postgresql.yaml
kubectl rollout status statefulset/order-postgres -n procureflow-dev --timeout=300s
```

```
kubectl get pod,pvc -n procureflow-dev -l app.kubernetes.io/name=order-postgres
kubectl describe pvc data-order-postgres-0 -n procureflow-dev
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Pod Ready and PVC Bound	Database runtime/storage ready.	Run independent SQL validation.
PVC Pending	Storage class/provisioner issue.	Return to storage smoke; inspect PVC events.
Permission denied on data directory	Volume ownership/security context mismatch.	Inspect pod logs; confirm fsGroup and image user behavior.
CrashLoopBackOff	Secret/env/init problem.	kubectl logs order-postgres-0 and describe pod.

Step 21.3 - SQL and restart persistence test

Previous evidence	PostgreSQL pod Ready and PVC Bound.
Why this step now	A health probe only accepts connections. Data durability prove karne ke liye table/row create karke pod restart ke baad read karenge.
Action	Run the copy-paste block below.
Expected evidence	Row id=1 message=PERSISTENCE_OK pod recreation ke baad bhi return ho.
Next connection	Independent database durability proven; application can now add a controlled dependency.

```
kubectl exec -n procureflow-dev order-postgres-0 -- sh -c '
  PGPASSWORD="$POSTGRES_PASSWORD" psql \
    -U "$POSTGRES_USER" \
    -d "$POSTGRES_DB" \
    -v ON_ERROR_STOP=1 \
    -c "CREATE TABLE IF NOT EXISTS platform_validation (id integer primary key, message text);" \
    -c "INSERT INTO platform_validation VALUES (1, '''PERSISTENCE_OK''') ON CONFLICT (id) DO UPDATE SET
message=EXCLUDED.message;" \
    -c "SELECT * FROM platform_validation;"
'

kubectl delete pod order-postgres-0 -n procureflow-dev
kubectl wait pod/order-postgres-0 -n procureflow-dev \
  --for=condition=Ready --timeout=300s

kubectl exec -n procureflow-dev order-postgres-0 -- sh -c '
  PGPASSWORD="$POSTGRES_PASSWORD" psql \
    -U "$POSTGRES_USER" \
    -d "$POSTGRES_DB" \
    -c "SELECT * FROM platform_validation;"
'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Row remains	PVC persistence proven.	Integrate Order Service.
Table missing after restart	Wrong PGDATA/mount path or ephemeral storage.	Inspect volumeMount and PVC attachment before proceeding.
Authentication failed	Secret/database credential mismatch.	Compare secret keys and pod env key names without printing password.
Pod name not ready	StatefulSet recreation issue.	Describe pod and PVC; verify node capacity.

22. Connect Order Service to PostgreSQL

Step 22.1 - Add JDBC, JPA and Flyway dependencies

Previous evidence	PostgreSQL SQL and persistence tests pass independently.
Why this step now	Application dependency only ab add hogi because database itself known-good hai. Flyway schema changes ko application

	release ke saath versioned and repeatable banata hai.
Action	Run the copy-paste block below.
Expected evidence	pom shows JPA/PostgreSQL/Flyway and YAML shows datasource configuration.
Next connection	Schema migration and connection config ready; business controller must persist through repository.

```
cd ~/projects/procureflow-devops-modernization/applications/order-service-java

cat > pom.xml <<'EOF'
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.procureflow</groupId>
  <artifactId>order-service</artifactId>
  <version>1.1.0</version>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.0</version>
    <relativePath/>
  </parent>
  <properties><java.version>21</java.version></properties>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</
artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-actuator</
artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</
artifactId></dependency>
    <dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</
scope></dependency>
    <dependency><groupId>org.flywaydb</groupId><artifactId>flyway-database-postgresql</artifactId></
dependency>
    <dependency><groupId>io.micrometer</groupId><artifactId>micrometer-registry-prometheus</artifactId></
dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</
artifactId><scope>test</scope></dependency>
  </dependencies>
  <build><plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-
plugin</artifactId></plugin></plugins></build>
</project>
EOF

mkdir -p src/main/resources/db/migration
cat > src/main/resources/db/migration/V1__create_purchase_orders.sql <<'EOF'
CREATE TABLE IF NOT EXISTS purchase_orders (
  id UUID PRIMARY KEY,
  supplier_id VARCHAR(100) NOT NULL,
  amount NUMERIC(14,2) NOT NULL,
  status VARCHAR(40) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL
);
EOF

cat > src/main/resources/application.yaml <<'EOF'
server:
  port: 8080
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
  endpoint:
    health:
      probes:
        enabled: true
  health:
    livenessstate:
      enabled: true
    readinessstate:
      enabled: true
spring:
  datasource:
    url: ${DATABASE_URL:jdbc:postgresql://localhost:5432/procureflow}
    username: ${DATABASE_USER:procureflow}
    password: ${DATABASE_PASSWORD:procureflow}
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
  flyway:
    enabled: true
info:
  app:
    name: order-service
    version: ${SERVICE_VERSION:dev}
EOF

grep -nE 'data-jpa|postgresql|flyway' pom.xml
tail -30 src/main/resources/application.yaml
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Dependencies and YAML present	Build configuration updated.	Create entity/repository/controller.
Duplicate spring key	YAML already contained old content and was appended instead of replaced.	Use the exact overwrite command above; do not append another spring block.
Maven model error	POM XML or dependency artifact is invalid.	Run mvn help:effective-pom and correct the reported line.

Step 22.2 - Replace in-memory order response with persisted entity

Previous evidence	JPA and Flyway dependencies configured.
Why this step now	A database is valuable only when business transaction commits and can be read again. Entity, repository and controller make persistence observable.
Action	Run the copy-paste block below.
Expected evidence	Image build succeeds and registry push digest appears.
Next connection	Artifact exists; Deployment now needs database endpoint and secret values without exposing credentials.

```
cd ~/projects/procureflow-devops-modernization/applications/order-service-java

cat > src/main/java/com/procureflow/order/PurchaseOrder.java <<'EOF'
package com.procureflow.order;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

@Entity
@Table(name = "purchase_orders")
public class PurchaseOrder {
    @Id
    private UUID id;
    @Column(name = "supplier_id", nullable = false)
    private String supplierId;
    @Column(nullable = false)
    private BigDecimal amount;
    @Column(nullable = false)
    private String status;
    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    protected PurchaseOrder() {}

    public PurchaseOrder(String supplierId, BigDecimal amount) {
        this.id = UUID.randomUUID();
        this.supplierId = supplierId;
        this.amount = amount;
        this.status = "CREATED";
        this.createdAt = Instant.now();
    }

    public UUID getId() { return id; }
    public String getSupplierId() { return supplierId; }
    public BigDecimal getAmount() { return amount; }
    public String getStatus() { return status; }
    public Instant getCreatedAt() { return createdAt; }
}
EOF

cat > src/main/java/com/procureflow/order/PurchaseOrderRepository.java <<'EOF'
package com.procureflow.order;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.UUID;

public interface PurchaseOrderRepository extends JpaRepository<PurchaseOrder, UUID> {}
EOF

cat > src/main/java/com/procureflow/order/OrderController.java <<'EOF'
package com.procureflow.order;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/orders")
public class OrderController {
    private final PurchaseOrderRepository repository;
    private final Counter createdCounter;

    public OrderController(PurchaseOrderRepository repository, MeterRegistry registry) {
        this.repository = repository;
        this.createdCounter = Counter.builder("procureflow.orders.created")
                .description("Orders created")
                .register(registry);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public PurchaseOrder create(@RequestBody Map<String, Object> request) {
        String supplierId = String.valueOf(request.getOrDefault("supplierId", "unknown"));
        BigDecimal amount = new BigDecimal(String.valueOf(request.getOrDefault("amount", "0")));
        PurchaseOrder saved = repository.save(new PurchaseOrder(supplierId, amount));
        createdCounter.increment();
        return saved;
    }

    @GetMapping
    public List<PurchaseOrder> list() {
```

```
        return repository.findAll();
    }
}
EOF

# Tests need a database. Build only now; Kubernetes integration test will validate real PostgreSQL.
docker build -t localhost:5001/procureflow/order-service:1.1.0 .
docker push localhost:5001/procureflow/order-service:1.1.0
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Build and push pass	Database-capable artifact ready.	Extend Helm secret env support.
Compile error	Entity/controller imports or method signatures wrong.	Read Maven compiler line and fix before pushing.
Spring test fails due localhost DB	Context now requires datasource.	Use Testcontainers/H2 profile later; for current controlled integration, run package with skipTests only after code compile and document gap.
Flyway dependency not found	Version/module mismatch.	Use flyway-core plus correct database module compatible with Spring Boot BOM.

Step 22.3 - Add secret-driven environment support to Helm

Previous evidence	Order image 1.1.0 exists.
Why this step now	Database password values.yaml mein nahi honi chahiye. Chart only secret name/key references store karega; Kubernetes resolves value at runtime.
Action	Run the copy-paste block below.
Expected evidence	Rendered Deployment shows secretKeyRef, but actual password value does not appear.
Next connection	Deployment manifest now knows how to connect without storing password; upgrade can test true dependency.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/helm/procureflow/templates/deployments.yaml <<'EOF'
{{- range $key, $svc := .Values.services }}
{{- if $svc.enabled }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ $svc.name }}
  labels:
    {{- include "procureflow.labels" $ | nindent 4 }}
    app.kubernetes.io/name: {{ $svc.name }}
spec:
  replicas: {{ $svc.replicas }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ $svc.name }}
  template:
    metadata:
      labels:
        {{- include "procureflow.labels" $ | nindent 8 }}
        app.kubernetes.io/name: {{ $svc.name }}
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/path: {{ $svc.metrics.path | quote }}
        prometheus.io/port: {{ $svc.port | quote }}
    spec:
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: {{ $svc.name }}
          image: "{{ $.Values.global.registry }}/{{ $svc.image.repository }}:{{ $svc.image.tag }}"
          imagePullPolicy: {{ $svc.image.pullPolicy }}
          ports:
            - name: http
              containerPort: {{ $svc.port }}
          env:
            {{- range $name, $value := $svc.env }}
            - name: {{ $name }}
              value: {{ $value | quote }}
            {{- end }}
            {{- range $item := $svc.secretEnv }}
            - name: {{ $item.name }}
              valueFrom:
                secretKeyRef:
                  name: {{ $item.secretName }}
                  key: {{ $item.secretKey }}
            {{- end }}
          readinessProbe:
            httpGet:
              path: {{ $svc.health.readiness }}
              port: http
            initialDelaySeconds: 5
            periodSeconds: 5
            failureThreshold: 12
          livenessProbe:
            httpGet:
              path: {{ $svc.health.liveness }}
              port: http
            initialDelaySeconds: 15
            periodSeconds: 10
          resources:
            {{- toYaml $svc.resources | nindent 12 }}
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
            readOnlyRootFilesystem: true
      terminationGracePeriodSeconds: 30
---
{{- end }}
{{- end }}
EOF

cat > platform/helm/procureflow/values.yaml <<'EOF'
global:
  registry: localhost:5001/procureflow
  environment: dev
  gatewayName: procureflow-gateway
services:
  supplier:
    enabled: true
    name: supplier-service
    image: {repository: supplier-service, tag: "1.0.0", pullPolicy: IfNotPresent}
    replicas: 1
```

```
    port: 8080
    routePath: /api/suppliers
    health: {liveness: /health/live, readiness: /health/ready}
    metrics: {enabled: true, path: /metrics}
    resources:
      requests: {cpu: 100m, memory: 128Mi}
      limits: {cpu: 500m, memory: 256Mi}
    env: {SERVICE_VERSION: "1.0.0"}
    secretEnv: []
  order:
    enabled: true
    name: order-service
    image: {repository: order-service, tag: "1.1.0", pullPolicy: IfNotPresent}
    replicas: 1
    port: 8080
    routePath: /api/orders
    health:
      liveness: /actuator/health/liveness
      readiness: /actuator/health/readiness
    metrics: {enabled: true, path: /actuator/prometheus}
    resources:
      requests: {cpu: 150m, memory: 256Mi}
      limits: {cpu: 700m, memory: 512Mi}
    env:
      SERVICE_VERSION: "1.1.0"
      DATABASE_URL: "jdbc:postgresql://order-postgres:5432/procureflow"
      DATABASE_USER: "procureflow"
    secretEnv:
      - name: DATABASE_PASSWORD
        secretName: order-postgres-credentials
        secretKey: POSTGRES_PASSWORD
hpa:
  enabled: false
  minReplicas: 1
  maxReplicas: 4
  targetCPUUtilizationPercentage: 60
podDisruptionBudget:
  enabled: false
  minAvailable: 1
EOF

helm lint platform/helm/procureflow
helm template procureflow platform/helm/procureflow \
  -f platform/helm/procureflow/values.yaml \
  -f platform/helm/procureflow/values-dev.yaml \
  -n procureflow-dev \
  > /tmp/procureflow-db-rendered.yaml

grep -A8 -B4 'DATABASE_PASSWORD' /tmp/procureflow-db-rendered.yaml
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
secretKeyRef rendered	Credential injection design correct.	Deploy upgrade.
Password plaintext visible	Secret handling failed.	Remove value, rotate secret, inspect Git history before continuing.
Template error on secretEnv	One service lacks empty list or YAML indentation wrong.	Set secretEnv: [] for services without secrets.

Step 22.4 - Upgrade Order Service and verify durable orders

Previous evidence	Chart renders database secret references safely.
Why this step now	Helm upgrade activates the dependency. Readiness should remain false until Flyway and DB connection succeed, preventing premature traffic.
Action	Run the copy-paste block below.
Expected evidence	Created order list mein pod restart ke baad bhi present ho; logs show successful Flyway migration and datasource startup.
Next connection	Modern stateful dependency works; now legacy MySQL source can be built for migration demonstration.

```
cd ~/projects/procureflow-devops-modernization

helm upgrade procureflow-dev platform/helm/procureflow \
  --namespace procureflow-dev \
  --values platform/helm/procureflow/values.yaml \
  --values platform/helm/procureflow/values-dev.yaml \
  --wait \
  --timeout 10m \
  --atomic

kubectl rollout status deployment/order-service -n procureflow-dev --timeout=300s
kubectl logs deployment/order-service -n procureflow-dev --tail=120

curl -fsS -X POST http://127.0.0.1:8080/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"supplierId":"SUP-DB-100","amount":2500.75}' | tee /tmp/order-created.json | jq .

curl -fsS http://127.0.0.1:8080/api/orders | jq .

kubectl delete pod -n procureflow-dev \
  -l app.kubernetes.io/name=order-service
kubectl rollout status deployment/order-service -n procureflow-dev --timeout=300s
curl -fsS http://127.0.0.1:8080/api/orders | jq .
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Order remains after app pod restart	Persistence and reconnect proven.	Capture DB checkpoint.
Readiness fails	DB DNS, credential, schema or connection issue.	Logs first; then kubectl get endpoints order-postgres; verify secret key names.
Flyway validation error	Schema history vs migration mismatch.	Do not delete history; fix migration/version strategy.
HTTP 502/503 during rollout only	Gateway had no ready backend briefly.	Check rolling strategy, replicas and readiness; production-like profile uses >=2 replicas/PDB.

Step 22.5 - Database integration checkpoint

Previous evidence	Order persistence survives application restart.
Why this step now	Before legacy migration complexity, save a known-good modern database state.
Action	Run the copy-paste block below.
Expected evidence	Commit created; secret manifest/password not staged.
Next connection	Next phase can compare legacy and target state with a stable modern service already operating.

```
cd ~/projects/procureflow-devops-modernization

{
  date -Is
  kubectl get statefulset,pod,pvc -n procureflow-dev \
    -l app.kubernetes.io/name=order-postgres -o wide
  kubectl get deployment/order-service -n procureflow-dev -o wide
  helm history procureflow-dev -n procureflow-dev
  curl -fsS http://127.0.0.1:8080/api/orders
} > docs/evidence/02-postgresql-integration.txt

git add applications/order-service-java platform/helm databases/postgresql docs/evidence/02-postgresql-
integration.txt
git commit -m "feat(data): persist orders in PostgreSQL with Flyway and secret references"
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Commit correct	Modern state baseline safe.	Build legacy MySQL source.
Secret staged	Credentials risk.	git reset secret file; add to .gitignore; rotate if exposed.

Part VIII - Legacy MySQL baseline and controlled migration to Kubernetes

23. Migration principle: prove source, backup, target, restore, validation, then cutover

```
Legacy MySQL source
      |
      | inventory + baseline row count/checksum
      v
Consistent logical backup
      |
      | restore rehearsal
      v
Kubernetes MySQL target + PVC
      |
      | schema/data validation
      v
Application connection test
      |
      | controlled write freeze + final delta
      v
Cutover
      |
      +---- success ---> observe and retire source later
      |
      +---- failure ---> rollback connection to source
```

Why not cut over immediately?
Backup file successfully ban jana migration success nahi. Recovery tab proven hoti hai jab target par restore, row counts, checksums, application queries, restart persistence aur rollback path all pass karein.

24. Legacy MySQL source on Docker Compose

Step 24.1 - Legacy source compose and seed data create karein

Previous evidence	Modern PostgreSQL integration is stable.
Why this step now	Legacy source ko separate Docker Compose boundary mein run karna VM-style ownership simulate karta hai: local volume, static port and manually managed backup.
Action	Run the copy-paste block below.
Expected evidence	Container healthy and three supplier rows show.
Next connection	Source running; migration baseline must capture count and deterministic content hash.

```
cd ~/projects/procureflow-devops-modernization

cat > legacy-platform/docker-compose/mysql-init.sql <<'EOF'
CREATE DATABASE IF NOT EXISTS legacy_procureflow;
USE legacy_procureflow;

CREATE TABLE IF NOT EXISTS suppliers (
  id INT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  status VARCHAR(30) NOT NULL,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO suppliers (id, name, status) VALUES
  (1, 'Nordic Components', 'ACTIVE'),
  (2, 'Baltic Logistics', 'REVIEW'),
  (3, 'Central Steel Works', 'ACTIVE')
ON DUPLICATE KEY UPDATE
  name=VALUES(name), status=VALUES(status);
EOF

cat > legacy-platform/docker-compose/compose.yaml <<'EOF'
services:
  legacy-mysql:
    image: mysql:8.4
    container_name: procureflow-legacy-mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: local-root-only
      MYSQL_DATABASE: legacy_procureflow
      MYSQL_USER: legacy_app
      MYSQL_PASSWORD: local-app-only
    ports:
      - "127.0.0.1:13306:3306"
    volumes:
      - legacy_mysql_data:/var/lib/mysql
      - ./mysql-init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-plocal-root-only"]
      interval: 5s
      timeout: 3s
      retries: 30

volumes:
  legacy_mysql_data:
EOF

cd legacy-platform/docker-compose
docker compose -p procureflow-legacy up -d

docker compose -p procureflow-legacy ps
for i in {1..60}; do
  STATUS=$(docker inspect procureflow-legacy-mysql --format '{{.State.Health.Status}}')
  echo "MySQL health: $STATUS"
  [ "$STATUS" = healthy ] && break
  sleep 2
done

docker exec procureflow-legacy-mysql \
  mysql -ulegacy_app -plocal-app-only legacy_procureflow \
  -e 'SELECT * FROM suppliers ORDER BY id;'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Healthy with 3 rows	Legacy source baseline exists.	Capture inventory/checksum.
Port 13306 occupied	Another local DB uses port.	Identify owner or change only documented compose port.
Health unhealthy	MySQL initialization/password/log issue.	docker logs procureflow-legacy-mysql.
Rows absent on reused volume	Init scripts only run on empty data directory.	Seed manually or remove disposable legacy volume after confirming no needed data.

Step 24.2 - Capture source inventory and checksum

Previous evidence	Legacy source contains known rows.
Why this step now	Before backup, create measurable source truth. Without baseline, restore can look successful while rows are missing or altered.
Action	Run the copy-paste block below.

Expected evidence	Row count 3 and a SHA-256 hash stored.
Next connection	Baseline fixed; backup can now be validated against known source.

```
cd ~/projects/procureflow-devops-modernization

mkdir -p migration/mysql-to-kubernetes/artifacts

docker exec procureflow-legacy-mysql \
  mysql -N -B -ulegacy_app -plocal-app-only legacy_procureflow \
  -e 'SELECT COUNT(*) FROM suppliers;' \
  | tee migration/mysql-to-kubernetes/artifacts/source-row-count.txt

docker exec procureflow-legacy-mysql \
  mysql -N -B -ulegacy_app -plocal-app-only legacy_procureflow \
  -e 'SELECT id,name,status FROM suppliers ORDER BY id;' \
  | tee migration/mysql-to-kubernetes/artifacts/source-data.tsv

sha256sum migration/mysql-to-kubernetes/artifacts/source-data.tsv \
  | tee migration/mysql-to-kubernetes/artifacts/source-data.sha256

cat migration/mysql-to-kubernetes/artifacts/source-row-count.txt
cat migration/mysql-to-kubernetes/artifacts/source-data.sha256
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Count/hash recorded	Source truth measurable.	Create logical backup.
Count unexpected	Source changed or seed incomplete.	Investigate before backup; baseline must reflect approved source.
Password warning	Expected local lab warning but credential hygiene still poor.	Next improvement: use MYSQL_PWD or option file; never use this pattern for real production.

Step 24.3 - Create and inspect logical backup

Previous evidence	Source count and checksum recorded.
Why this step now	mysqldump creates portable logical backup. We inspect non-sensitive structure and verify file non-empty before target deployment.
Action	Run the copy-paste block below.
Expected evidence	Backup file non-empty; CREATE TABLE and INSERT statements visible; checksum file created.
Next connection	Backup prepared; target now must be independently provisioned and healthy before restore.

```
cd ~/projects/procureflow-devops-modernization

BACKUP="migration/mysql-to-kubernetes/artifacts/legacy_procureflow_$(date +%Y%m%d_%H%M%S).sql"

docker exec procureflow-legacy-mysql \
  mysqldump \
    -uroot -plocal-root-only \
    --single-transaction \
    --routines \
    --triggers \
    --events \
    --set-gtid-purged=OFF \
    legacy_procureflow \
  > "$BACKUP"

test -s "$BACKUP"
sha256sum "$BACKUP" > "${BACKUP}.sha256"
wc -l -c "$BACKUP"
grep -E '^CREATE TABLE|^INSERT INTO' "$BACKUP" | head -20
printf '%s
' "$BACKUP" > migration/mysql-to-kubernetes/artifacts/latest-backup-path.txt
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Backup non-empty and statements present	Logical backup plausible.	Deploy target MySQL.

Observation	Meaning	Next decision
Zero-byte backup	Dump failed though redirect created file.	Check command stderr/credentials; delete invalid file.
Access denied	Root password mismatch.	Verify compose env/volume reuse and credentials.
Data contains sensitive values	Backup is secret material.	Keep artifacts ignored/private; never commit real production dump.

25. Kubernetes MySQL target

Step 25.1 - Target secret and StatefulSet deploy karein

Previous evidence	Logical backup exists and checksum captured.
Why this step now	Target database uses new credentials and persistent storage. Source remains untouched, enabling rollback throughout rehearsal.
Action	Run the copy-paste block below.
Expected evidence	Target pod Ready and PVC Bound.
Next connection	Empty target healthy; restore can now be rehearsed without source downtime.

```
cd ~/projects/procureflow-devops-modernization

TARGET_ROOT_PASSWORD="$(openssl rand -base64 24 | tr -d '
' | tr '/+' '_-')"
TARGET_APP_PASSWORD="$(openssl rand -base64 24 | tr -d '
' | tr '/+' '_-')"

kubectl create secret generic target-mysql-credentials \
  -n procureflow-dev \
  --from-literal=MYSQL_ROOT_PASSWORD="$TARGET_ROOT_PASSWORD" \
  --from-literal=MYSQL_DATABASE=legacy_procureflow \
  --from-literal=MYSQL_USER=legacy_app \
  --from-literal=MYSQL_PASSWORD="$TARGET_APP_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -

unset TARGET_ROOT_PASSWORD TARGET_APP_PASSWORD

cat > databases/mysql/target-mysql.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: target-mysql
  namespace: procureflow-dev
spec:
  clusterIP: None
  selector:
    app.kubernetes.io/name: target-mysql
  ports:
    - name: mysql
      port: 3306
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: target-mysql
  namespace: procureflow-dev
spec:
  serviceName: target-mysql
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: target-mysql
  template:
    metadata:
      labels:
        app.kubernetes.io/name: target-mysql
        app.kubernetes.io/part-of: procureflow
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: mysql
          image: mysql:8.4
          ports:
            - name: mysql
              containerPort: 3306
          envFrom:
            - secretRef:
                name: target-mysql-credentials
          readinessProbe:
            exec:
              command: ["sh", "-c", "mysqladmin ping -h 127.0.0.1 -uroot -p$MYSQL_ROOT_PASSWORD"]
            initialDelaySeconds: 15
            periodSeconds: 5
            failureThreshold: 24
          livenessProbe:
            exec:
              command: ["sh", "-c", "mysqladmin ping -h 127.0.0.1 -uroot -p$MYSQL_ROOT_PASSWORD"]
            initialDelaySeconds: 60
            periodSeconds: 10
          resources:
            requests:
              cpu: 150m
              memory: 512Mi
            limits:
              cpu: "1"
              memory: 1Gi
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 3Gi
```

```
EOF

kubectl apply -f databases/mysql/target-mysql.yaml
kubectl rollout status statefulset/target-mysql -n procureflow-dev --timeout=600s
kubectl get pod,pvc -n procureflow-dev -l app.kubernetes.io/name=target-mysql
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Ready/Bound	Target infrastructure ready.	Copy and restore backup.
Quota exceeded	Dev namespace memory quota too low for simultaneous databases.	Temporarily stop nonessential profile or adjust documented quota based on capacity.
MySQL initializing slowly	First startup can exceed normal app startup.	Watch logs and readiness; do not use aggressive liveness initial delay.
Permission/storage error	PVC mount/image user issue.	Inspect events and logs before restore.

Step 25.2 - Restore backup into target

Previous evidence	Target MySQL Ready and backup checksum exists.
Why this step now	Restore rehearsal proves backup usability. We stream file through kubectl exec so no persistent secret dump remains inside pod.
Action	Run the copy-paste block below.
Expected evidence	Checksum OK and target row count 3.
Next connection	Count equality is necessary but not sufficient; content checksum comparison comes next.

```
cd ~/projects/procureflow-devops-modernization

BACKUP="$(cat migration/mysql-to-kubernetes/artifacts/latest-backup-path.txt)"
sha256sum --check "${BACKUP}.sha256"

kubectl exec -i -n procureflow-dev target-mysql-0 -- sh -c '
  mysql -uroot -p"$MYSQL_ROOT_PASSWORD" legacy_procureflow
' < "$BACKUP"

kubectl exec -n procureflow-dev target-mysql-0 -- sh -c '
  mysql -N -B -ulegacy_app -p"$MYSQL_PASSWORD" legacy_procureflow \
    -e "SELECT COUNT(*) FROM suppliers;"
'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Restore and count pass	Backup is recoverable.	Compare deterministic data hash.
ERROR table exists	Target not empty from previous rehearsal.	For lab, recreate target database/PVC or use clean schema; in production plan idempotent restore carefully.
Access denied	Target credentials/privileges mismatch.	Verify secret keys and MySQL user creation logs.
Unknown database	Initialization incomplete or DB name mismatch.	Create DB or correct secret before restore.

Step 25.3 - Compare target data with source checksum

Previous evidence	Target row count equals source baseline.
Why this step now	Same count can still hide changed values. Deterministic sorted export and SHA-256 compare gives stronger validation.
Action	Run the copy-paste block below.
Expected evidence	diff produces no output and DATA_HASH_MATCH prints one hash.
Next connection	Data correctness proven; next target restart and application/client connectivity prove operational readiness.

```
cd ~/projects/procureflow-devops-modernization

kubectl exec -n procureflow-dev target-mysql-0 -- sh -c '
  mysql -N -B -ulegacy_app -p"$MYSQL_PASSWORD" legacy_procureflow \
    -e "SELECT id,name,status FROM suppliers ORDER BY id;"
' > migration/mysql-to-kubernetes/artifacts/target-data.tsv

sha256sum migration/mysql-to-kubernetes/artifacts/target-data.tsv \
  | tee migration/mysql-to-kubernetes/artifacts/target-data.sha256

diff -u \
  migration/mysql-to-kubernetes/artifacts/source-data.tsv \
  migration/mysql-to-kubernetes/artifacts/target-data.tsv

SOURCE_HASH=$(awk '{print $1}' migration/mysql-to-kubernetes/artifacts/source-data.sha256)
TARGET_HASH=$(awk '{print $1}' migration/mysql-to-kubernetes/artifacts/target-data.sha256)

[ "$SOURCE_HASH" = "$TARGET_HASH" ]
echo "DATA_HASH_MATCH=${SOURCE_HASH}"
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Hashes match	Restored business data is identical for selected scope.	Test target restart and client access.
Count same but diff exists	Value/order/encoding mismatch.	Inspect diff and collation/character-set handling before cutover.
Hash command fails	Files or baseline path missing.	Recreate deterministic exports; do not guess validation.

Step 25.4 - Target restart, DNS and rollback rehearsal

Previous evidence	Source and target data hashes match.
Why this step now	Cutover requires target survive restart and source remain reachable for rollback. We test both paths before changing any application configuration.
Action	Run the copy-paste block below.
Expected evidence	Both target and source return 3; target does so after pod recreation.
Next connection	Technical prerequisites pass; controlled cutover procedure can now be written and rehearsed.

```
kubectl delete pod target-mysql-0 -n procureflow-dev
kubectl wait pod/target-mysql-0 -n procureflow-dev \
  --for=condition=Ready --timeout=600s

kubectl run mysql-client \
  -n procureflow-dev \
  --rm -i --restart=Never \
  --image=mysql:8.4 \
  --env="MYSQL_PWD=$(kubectl get secret target-mysql-credentials -n procureflow-dev -o
jsonpath='{.data.MYSQL_PASSWORD}' | base64 -d)" \
  -- mysql -h target-mysql -ulegacy_app -N -B legacy_procureflow \
  -e 'SELECT COUNT(*) FROM suppliers;'

# Confirm source still works for rollback
cd ~/projects/procureflow-devops-modernization/legacy-platform/docker-compose
docker exec procureflow-legacy-mysql \
  mysql -N -B -ulegacy_app -plocal-app-only legacy_procureflow \
  -e 'SELECT COUNT(*) FROM suppliers;'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Both paths return expected count	Target ready and rollback source preserved.	Document cutover runbook.
Target fails after restart	Persistence/startup not reliable.	Stop cutover; inspect PVC and logs.
Source unavailable	Rollback path lost.	Restore source availability before any cutover.
Secret printed in process list concern	Inline env is visible locally.	For stricter lab, create temporary client pod secretRef; delete immediately.

26. Cutover and rollback runbook

Current sample Supplier Service is in-memory, so migration cutover is demonstrated at database connection/client contract level. A later service version can read suppliers from MySQL. The runbook remains the same: stop writes, final backup/delta, validate, change endpoint, observe, rollback if thresholds fail.

Step 26.1 - Write executable migration validation script

Previous evidence	Target and source are simultaneously healthy.
Why this step now	Human checklist alone can miss a step under pressure. Script repeatably checks source count, target count, target pod readiness and hash equality before cutover approval.
Action	Run the copy-paste block below.
Expected evidence	MIGRATION_VALIDATION=PASS.
Next connection	Automated validation now feeds an explicit go/no-go decision.

```
cd ~/projects/procureflow-devops-modernization

cat > migration/mysql-to-kubernetes/validate-migration.sh <<'EOF'
#!/usr/bin/env bash
set -Eeuo pipefail
ROOT=$(cd "$(dirname "$0")/../.." && pwd)
ART="$ROOT/migration/mysql-to-kubernetes/artifacts"

kubectl wait pod/target-mysql-0 -n procureflow-dev \
  --for=condition=Ready --timeout=120s >/dev/null

SOURCE_COUNT=$(docker exec procureflow-legacy-mysql \
  mysql -N -B -ulegacy_app -plocal-app-only legacy_procureflow \
  -e 'SELECT COUNT(*) FROM suppliers;')

TARGET_DATA=$(mktemp)
trap 'rm -f "$TARGET_DATA"' EXIT
kubectl exec -n procureflow-dev target-mysql-0 -- sh -c '
  mysql -N -B -ulegacy_app -p"$MYSQL_PASSWORD" legacy_procureflow \
  -e "SELECT id,name,status FROM suppliers ORDER BY id;"
' > "$TARGET_DATA"

TARGET_COUNT=$(wc -l < "$TARGET_DATA" | tr -d ' ')
SOURCE_HASH=$(awk '{print $1}' "$ART/source-data.sha256")
TARGET_HASH=$(sha256sum "$TARGET_DATA" | awk '{print $1}')

printf 'SOURCE_COUNT=%s
TARGET_COUNT=%s
' "$SOURCE_COUNT" "$TARGET_COUNT"
printf 'SOURCE_HASH=%s
TARGET_HASH=%s
' "$SOURCE_HASH" "$TARGET_HASH"

[ "$SOURCE_COUNT" = "$TARGET_COUNT" ]
[ "$SOURCE_HASH" = "$TARGET_HASH" ]
echo 'MIGRATION_VALIDATION=PASS'
EOF

chmod +x migration/mysql-to-kubernetes/validate-migration.sh
./migration/mysql-to-kubernetes/validate-migration.sh
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
PASS	Pre-cutover data gate satisfied.	Document cutover/rollback decisions.
Count mismatch	Data delta or incomplete restore.	Do not cut over; repeat backup/sync.
Hash mismatch	Content mismatch.	Do not cut over; inspect diff/collation.
Target not Ready	Operational gate failed.	Do not cut over.

Cutover decision table

Observation	Meaning	Next decision
Target Ready, validation PASS, error budget healthy	All technical gates pass.	Freeze writes, take final delta backup, restore delta, run validation, update

Observation	Meaning	Next decision
		application endpoint, observe.
Validation fails	Data correctness not proven.	NO-GO. Keep source active and troubleshoot.
Target latency/error rate exceeds threshold	Functional target exists but business reliability risk high.	Rollback endpoint to source and investigate.
Source changed during long restore	Initial backup became stale.	Use final sync/controlled write window before cutover.
Cutover works for 30-60 minutes	Early success only.	Keep source retained/read-only for agreed rollback window; do not destroy immediately.

Rollback sequence

6. Stop or pause new writes to prevent split-brain.
7. Change application database endpoint/secret reference back to legacy source.
8. Restart or roll out application safely and confirm readiness.
9. Run business smoke tests and monitor errors/latency.
10. Document target writes that occurred after cutover and reconcile before next attempt.

> **Migration explanation**
> I did not treat mysqldump success as migration success. I recorded source counts and a deterministic hash, restored into a PVC-backed MySQL StatefulSet, compared target content, recreated the target pod to prove persistence, validated both source and target paths, and only then produced a go/no-go cutover and rollback runbook.

Step 26.2 - Migration checkpoint commit without secret dump

Previous evidence	Validation script passes and rollback runbook is defined.
Why this step now	Migration logic and manifests belong in Git; database dump and credentials do not.
Action	Run the copy-paste block below.
Expected evidence	Commit created; dump/checksum/data exports untracked or ignored.
Next connection	Stateful migration story is complete; asynchronous platform components can now extend the business flow.

```
cd ~/projects/procureflow-devops-modernization

cat >> .gitignore <<'EOF'
migration/mysql-to-kubernetes/artifacts/*.sql
migration/mysql-to-kubernetes/artifacts/*.sha256
migration/mysql-to-kubernetes/artifacts/*.tsv
migration/mysql-to-kubernetes/artifacts/latest-backup-path.txt
EOF

git add .gitignore legacy-platform/docker-compose databases/mysql migration/mysql-to-kubernetes/validate-
migration.sh
git commit -m "feat(migration): add legacy MySQL backup restore validation and rollback flow"
git status --short
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Working tree safe	Migration implementation stored without data leak.	Proceed to messaging profile.
SQL dump staged	Sensitive artifact risk.	git reset; ensure ignore pattern; remove from Git history if committed.

Part IX - Kafka, RabbitMQ, and Elasticsearch operational profiles

27. Why Kafka and RabbitMQ are not interchangeable

Dono messaging tools hain, lekin problem model different hai. Kafka durable event log hai jahan multiple consumer groups same business event independently replay kar sakte hain. RabbitMQ work queue hai jahan ek task worker ko deliver hota hai, acknowledgement/retry/dead-letter behavior important hota hai.

```
OrderCreated business event
       |
       v
Kafka topic
       +---- Approval consumer group
       +---- Search-index consumer group
       +---- Audit consumer group

Notification task
       |
       v
RabbitMQ queue
       +---- Worker 1 OR Worker 2
       +---- retry exchange
       +---- dead-letter queue
```

28. Staged resource rule for messaging/search

Do not start everything blindly
16 GB machine par PostgreSQL, target MySQL, Kafka, RabbitMQ, Elasticsearch, Jenkins, SonarQube and monitoring simultaneously resource pressure create kar sakte hain. Before each profile, kubectl top nodes/pods and free -h check karein. Non-required profile scale down karein, data PVC retain karein.

Step 28.1 - Current capacity gate

Previous evidence	Core application and database workloads exist.
Why this step now	Messaging deploy karne se pehle available resources measure karna preventable Pending/OOM incidents avoid karta hai.
Action	Run the copy-paste block below.
Expected evidence	Enough free memory ho and no unexplained failed/pending pod.
Next connection	Capacity gate pass hone par one-node Kafka development profile deploy hoga.

```
printf '== WSL memory ==
'
free -h
printf '
== Kubernetes nodes ==
'
kubectl top nodes
printf '
== Top consumers ==
'
kubectl top pods -A --sort-by=memory | head -20
printf '
== Pending or failed pods ==
'
kubectl get pods -A --field-selector=status.phase!=Running,status.phase!=Succeeded
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Healthy headroom	Messaging profile can start.	Deploy Kafka.
Memory nearly exhausted	Starting brokers would create noise.	Scale down target-mysql if migration demo complete; stop Sonar/Jenkins; select one profile.
Existing Pending pods	Cluster capacity/config already unhealthy.	Resolve before adding components.
Metrics unavailable	Capacity decision lacks evidence.	Restore Metrics Server first.

29. Kafka single-node KRaft development profile

Step 29.1 - Deploy Kafka and create topic

Previous evidence	Capacity gate passes.
Why this step now	Single broker local profile event semantics and lag troubleshooting prove karta hai; production replication/availability claim nahi karta.
Action	Run the copy-paste block below.
Expected evidence	Kafka pod Ready and topic has 3 partitions, replication factor 1.
Next connection	Topic exists; now business event production and consumer-group offset can be demonstrated.

```
cd ~/projects/procureflow-devops-modernization

cat > messaging/kafka/kafka-dev.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: kafka
  namespace: procureflow-dev
spec:
  selector:
    app: kafka
  ports:
    - name: broker
      port: 9092
    - name: controller
      port: 9093
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: kafka
  namespace: procureflow-dev
spec:
  serviceName: kafka
  replicas: 1
  selector:
    matchLabels:
      app: kafka
  template:
    metadata:
      labels:
        app: kafka
    spec:
      containers:
        - name: kafka
          image: apache/kafka:3.9.1
          ports:
            - {name: broker, containerPort: 9092}
            - {name: controller, containerPort: 9093}
          env:
            - {name: KAFKA_NODE_ID, value: "1"}
            - {name: KAFKA_PROCESS_ROLES, value: "broker,controller"}
            - {name: KAFKA_LISTENERS, value: "PLAINTEXT://:9092,CONTROLLER://:9093"}
            - {name: KAFKA_ADVERTISED_LISTENERS, value: "PLAINTEXT://kafka:9092"}
            - {name: KAFKA_CONTROLLER_LISTENER_NAMES, value: "CONTROLLER"}
            - {name: KAFKA_LISTENER_SECURITY_PROTOCOL_MAP, value:
"CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT"}
            - {name: KAFKA_CONTROLLER_QUORUM_VOTERS, value: "1@kafka-0.kafka:9093"}
            - {name: KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR, value: "1"}
            - {name: KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR, value: "1"}
            - {name: KAFKA_TRANSACTION_STATE_LOG_MIN_ISR, value: "1"}
            - {name: KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS, value: "0"}
          resources:
            requests: {cpu: 200m, memory: 512Mi}
            limits: {cpu: "1", memory: 1Gi}
          volumeMounts:
            - {name: data, mountPath: /var/lib/kafka/data}
  volumeClaimTemplates:
    - metadata: {name: data}
      spec:
        accessModes: ["ReadWriteOnce"]
        resources: {requests: {storage: 2Gi}}
EOF

kubectl apply -f messaging/kafka/kafka-dev.yaml
kubectl rollout status statefulset/kafka -n procureflow-dev --timeout=600s
kubectl logs kafka-0 -n procureflow-dev --tail=80

kubectl exec -n procureflow-dev kafka-0 -- \
  /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka:9092 \
  --create --if-not-exists \
  --topic order-created \
  --partitions 3 \
  --replication-factor 1

kubectl exec -n procureflow-dev kafka-0 -- \
  /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka:9092 \
  --describe --topic order-created
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Topic described	Broker and metadata path work.	Produce/consume event.
Image env unsupported	Official image version configuration	Check image logs and official image

Observation	Meaning	Next decision
	changed.	environment names; keep one pinned version.
Broker cannot form quorum	Controller voter hostname/listener mismatch.	Verify StatefulSet DNS kafka-0.kafka and controller port.
Quota exceeded	Profile too heavy.	Scale down optional database/search workloads.

Step 29.2 - Produce, consume and inspect consumer lag

Previous evidence	Kafka topic exists.
Why this step now	A running broker is not enough. We need event delivery and consumer offset evidence, then intentionally create lag for troubleshooting demonstration.
Action	Run the copy-paste block below.
Expected evidence	Three JSON events consumed; group output shows CURRENT-OFFSET, LOG-END-OFFSET and LAG zero.
Next connection	Kafka event semantics proven; RabbitMQ can now demonstrate task and dead-letter semantics.

```
# Produce three events
kubectl exec -i -n procureflow-dev kafka-0 -- \
  /opt/kafka/bin/kafka-console-producer.sh \
  --bootstrap-server kafka:9092 \
  --topic order-created <<'EOF'
{"orderId":"ORD-1","status":"CREATED"}
{"orderId":"ORD-2","status":"CREATED"}
{"orderId":"ORD-3","status":"CREATED"}
EOF

# Consume using a named group so offsets are recorded
kubectl exec -n procureflow-dev kafka-0 -- \
  /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server kafka:9092 \
  --topic order-created \
  --group approval-service \
  --from-beginning \
  --max-messages 3

kubectl exec -n procureflow-dev kafka-0 -- \
  /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server kafka:9092 \
  --describe --group approval-service
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
LAG 0	Consumer caught up.	Document lag incident method.
LAG > 0	Messages remain unprocessed.	Check consumer health/rate; scale consumers only if partitions allow.
Group not found	Consumer exited before offset commit or group mismatch.	Repeat with named group and inspect group list.
Duplicate events	At-least-once semantics or repeated producer input.	Consumers need idempotency; do not assume exactly once.

30. RabbitMQ task queue and dead-letter profile

Step 30.1 - Deploy RabbitMQ with management UI

Previous evidence	Kafka profile is validated.
Why this step now	RabbitMQ will model one notification task being owned by one worker, with acknowledgements and dead-letter routing separate from Kafka event replay.
Action	Run the copy-paste block below.
Expected evidence	Deployment Available; diagnostics Ping succeeded; UI can open

	locally.
Next connection	Broker is healthy; queue topology now encodes retry/failure behavior.

```
cd ~/projects/procureflow-devops-modernization

kubectl create secret generic rabbitmq-credentials \
  -n procureflow-dev \
  --from-literal=RABBITMQ_DEFAULT_USER=procureflow \
  --from-literal=RABBITMQ_DEFAULT_PASS="$(openssl rand -base64 24 | tr -d '
')" \
  --dry-run=client -o yaml | kubectl apply -f -

cat > messaging/rabbitmq/rabbitmq.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: rabbitmq
  namespace: procureflow-dev
spec:
  selector: {app: rabbitmq}
  ports:
    - {name: amqp, port: 5672}
    - {name: management, port: 15672}
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rabbitmq
  namespace: procureflow-dev
spec:
  replicas: 1
  selector: {matchLabels: {app: rabbitmq}}
  template:
    metadata: {labels: {app: rabbitmq}}
    spec:
      containers:
        - name: rabbitmq
          image: rabbitmq:4.1-management
          ports:
            - {name: amqp, containerPort: 5672}
            - {name: management, containerPort: 15672}
          envFrom:
            - secretRef: {name: rabbitmq-credentials}
          readinessProbe:
            exec: {command: ["rabbitmq-diagnostics", "-q", "ping"]}
            initialDelaySeconds: 10
            periodSeconds: 5
          resources:
            requests: {cpu: 100m, memory: 256Mi}
            limits: {cpu: 500m, memory: 512Mi}
EOF

kubectl apply -f messaging/rabbitmq/rabbitmq.yaml
kubectl rollout status deployment/rabbitmq -n procureflow-dev --timeout=300s
kubectl exec deployment/rabbitmq -n procureflow-dev -- rabbitmq-diagnostics -q ping
kubectl port-forward -n procureflow-dev service/rabbitmq 15672:15672 >/tmp/rabbit-port-forward.log 2>&1 &
echo $! > /tmp/rabbit-port-forward.pid
echo 'Management UI: http://127.0.0.1:15672'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Ping succeeded	Broker ready.	Create DLX/queues through rabbitmqadmin.
Deployment OOM	Resource limit/profile pressure.	Stop Kafka temporarily or increase local profile capacity.
Port-forward already in use	Old process exists.	kill process from /tmp pid or select another local port.
Credential lost	Generated password not printed.	Retrieve locally from secret only when logging in; do not commit.

Step 30.2 - Configure notification queue and dead-letter queue

Previous evidence	RabbitMQ diagnostics pass.
Why this step now	Poison messages must not retry forever. Main queue routes rejected messages to dead-letter exchange/queue for

	controlled investigation.
Action	Run the copy-paste block below.
Expected evidence	notification.tasks shows one ready message and notification.dlq zero.
Next connection	RabbitMQ queue semantics proven. Search profile can be run separately when capacity permits.

```
RABBIT_USER=$(kubectl get secret rabbitmq-credentials -n procureflow-dev -o
jsonpath='{.data.RABBITMQ_DEFAULT_USER}' | base64 -d)
RABBIT_PASS=$(kubectl get secret rabbitmq-credentials -n procureflow-dev -o
jsonpath='{.data.RABBITMQ_DEFAULT_PASS}' | base64 -d)

kubectl exec deployment/rabbitmq -n procureflow-dev -- rabbitmqadmin \
  -u "$RABBIT_USER" -p "$RABBIT_PASS" \
  declare exchange name=notification.dlx type=direct durable=true

kubectl exec deployment/rabbitmq -n procureflow-dev -- rabbitmqadmin \
  -u "$RABBIT_USER" -p "$RABBIT_PASS" \
  declare queue name=notification.dlq durable=true

kubectl exec deployment/rabbitmq -n procureflow-dev -- rabbitmqadmin \
  -u "$RABBIT_USER" -p "$RABBIT_PASS" \
  declare binding source=notification.dlx destination=notification.dlq routing_key=failed

kubectl exec deployment/rabbitmq -n procureflow-dev -- rabbitmqadmin \
  -u "$RABBIT_USER" -p "$RABBIT_PASS" \
  declare queue name=notification.tasks durable=true \
  arguments='{"x-dead-letter-exchange":"notification.dlx","x-dead-letter-routing-key":"failed"}'

kubectl exec deployment/rabbitmq -n procureflow-dev -- rabbitmqadmin \
  -u "$RABBIT_USER" -p "$RABBIT_PASS" \
  publish routing_key=notification.tasks \
  payload='{"orderId":"ORD-1","type":"EMAIL"}'

kubectl exec deployment/rabbitmq -n procureflow-dev -- rabbitmqctl list_queues name messages_ready
messages_unacknowledged
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Main queue has 1	Task publication works.	Reject/nack via worker simulation and inspect DLQ.
Queue absent	Declaration command/auth failed.	Check rabbitmqadmin output and credentials.
Message in wrong vhost	Default vhost assumption mismatch.	Specify vhost consistently.
DLQ receives task only after reject/expiry	Expected behavior.	Use this to explain poison-message isolation.

31. Elasticsearch optional search profile

Elasticsearch memory intensive hai. 16 GB laptop par Kafka/RabbitMQ ko scale down before this profile. Local single-node mode search/index operations prove karta hai; production cluster sizing, snapshots and shard design separate decisions hain.

Step 31.1 - Deploy single-node Elasticsearch and verify index

Previous evidence	Messaging validation captured and optional components can be scaled down.
Why this step now	Procurement search use case supplier/order documents index karega. First prove cluster health, index creation, document write and search.
Action	Run the copy-paste block below.
Expected evidence	Cluster health yellow/green in single-node mode and search returns ORD-1.
Next connection	Search operations proven; now infrastructure automation and delivery automation can manage repeatability.

```
kubectl scale statefulset/kafka -n procureflow-dev --replicas=0
kubectl scale deployment/rabbitmq -n procureflow-dev --replicas=0

cd ~/projects/procureflow-devops-modernization
cat > databases/elasticsearch/elasticsearch-dev.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: elasticsearch
  namespace: procureflow-dev
spec:
  selector: {app: elasticsearch}
  ports: [{name: http, port: 9200}]
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: elasticsearch
  namespace: procureflow-dev
spec:
  replicas: 1
  selector: {matchLabels: {app: elasticsearch}}
  template:
    metadata: {labels: {app: elasticsearch}}
    spec:
      containers:
        - name: elasticsearch
          image: docker.elastic.co/elasticsearch/elasticsearch:8.17.0
          env:
            - {name: discovery.type, value: single-node}
            - {name: xpack.security.enabled, value: "false"}
            - {name: ES_JAVA_OPTS, value: "-Xms512m -Xmx512m"}
          ports: [{name: http, containerPort: 9200}]
          readinessProbe:
            httpGet: {path: /_cluster/health, port: http}
            initialDelaySeconds: 30
            periodSeconds: 10
          resources:
            requests: {cpu: 250m, memory: 1Gi}
            limits: {cpu: "1", memory: 1500Mi}
EOF

kubectl apply -f databases/elasticsearch/elasticsearch-dev.yaml
kubectl rollout status deployment/elasticsearch -n procureflow-dev --timeout=600s
kubectl port-forward -n procureflow-dev service/elasticsearch 19200:9200 >/tmp/es-port-forward.log 2>&1 &
echo $! > /tmp/es-port-forward.pid
sleep 3

curl -fsS http://127.0.0.1:19200/_cluster/health | jq .
curl -fsS -X PUT http://127.0.0.1:19200/procureflow-orders
curl -fsS -X POST http://127.0.0.1:19200/procureflow-orders/_doc/ORD-1 \
  -H 'Content-Type: application/json' \
  -d '{"supplier":"Nordic Components","status":"CREATED","amount":1250}'
curl -fsS -X POST 'http://127.0.0.1:19200/procureflow-orders/_refresh'
curl -fsS 'http://127.0.0.1:19200/procureflow-orders/_search?q=status:CREATED' | jq '.hits.hits'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Yellow with all primaries active	Expected single-node replicas cannot allocate.	For local lab accept/document; production requires replica nodes.
Red health	Primary shards unavailable.	Inspect logs, disk and memory; do not index more data.
OOMKilled	JVM/container profile too large.	Stop other profiles or lower heap with sufficient overhead.
vm.max_map_count error	Kernel requirement unmet.	Set appropriate WSL sysctl and document host prerequisite.

Part X - Terraform and Ansible: provisioning versus configuration

32. The responsibility boundary

```
Terraform
  | creates lifecycle-managed infrastructure objects
  | network, volume, container, cluster/cloud resources
  v
Provisioned target
  |
  | Ansible configures operating system and application state
  v
Configured legacy server
  | Java, users, directories, systemd, monitoring agent
```

> **Why both?**
> Terraform resource graph aur lifecycle manage karta hai. Ansible target ke andar packages/files/services configure karta hai. Ek tool ko doosre ki job dene se state, idempotency aur troubleshooting unclear ho jati hai.

33. Terraform local Docker lab

Step 33.1 - Install Terraform and create provider contract

Previous evidence	Core platform already works manually, so automation can be compared against known behavior.
Why this step now	Automation before understanding would hide mechanics. Ab registry/network/container behavior known hai, Terraform same type ke resources declaratively manage karna teach karega.
Action	Run the copy-paste block below.
Expected evidence	Terraform version prints and provider initialization succeeds with lock file.
Next connection	Provider can talk to Docker; now resource graph will create a disposable test environment.

```
cd /tmp
source ~/projects/procureflow-devops-modernization/.tool-versions.env

curl -fsSLO "https://releases.hashicorp.com/terraform/${TERRAFORM_VERSION}/terraform_${TERRAFORM_VERSION}
_linux_amd64.zip"
unzip -o "terraform_${TERRAFORM_VERSION}_linux_amd64.zip"
sudo install -m 0755 terraform /usr/local/bin/terraform
terraform version

cd ~/projects/procureflow-devops-modernization/platform/terraform/local
cat > versions.tf <<'EOF'
terraform {
  required_version = ">= 1.8.0"
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.6"
    }
  }
}
EOF

cat > providers.tf <<'EOF'
provider "docker" {}
EOF

terraform fmt
terraform init
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Init successful	Provider contract ready.	Declare disposable automation resources.
Checksum/provider download failure	Network/proxy or registry issue.	Verify releases.hashicorp.com and registry.terraform.io access.
Docker socket permission	Provider cannot reach daemon.	docker info must work for same user; do

Observation	Meaning	Next decision
		not chmod 777 socket.

Step 33.2 - Manage a disposable platform sandbox with Terraform

Previous evidence	Terraform Docker provider initialized.
Why this step now	Existing hand-created registry remains untouched. A separate sandbox proves plan/apply/state/destroy lifecycle without risking working cluster.
Action	Run the copy-paste block below.
Expected evidence	Plan shows 3 resources, apply succeeds, HTTP 200, state lists network/image/container.
Next connection	Created state enables drift/change/destroy demonstration.

```
cd ~/projects/procureflow-devops-modernization/platform/terraform/local

cat > main.tf <<'EOF'
resource "docker_network" "platform_sandbox" {
  name = "procureflow-platform-sandbox"
}

resource "docker_image" "nginx" {
  name         = "nginx:1.27-alpine"
  keep_locally = true
}

resource "docker_container" "platform_probe" {
  name  = "procureflow-terraform-probe"
  image = docker_image.nginx.image_id

  networks_advanced {
    name = docker_network.platform_sandbox.name
  }

  ports {
    internal = 80
    external = 18090
    ip       = "127.0.0.1"
  }

  healthcheck {
    test     = ["CMD", "wget", "-q", "-O", "-", "http://127.0.0.1/"]
    interval = "5s"
    timeout  = "2s"
    retries  = 10
  }
}

output "probe_url" {
  value = "http://127.0.0.1:18090"
}
EOF

terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
terraform output
curl -I http://127.0.0.1:18090
terraform state list
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Apply and HTTP 200	Terraform lifecycle proven.	Change configuration and observe plan.
Port collision	18090 already used.	Identify owner or choose documented alternate variable.
State lists resources	State tracks actual objects.	Never manually delete unless testing drift.
Apply partial failure	State may contain successful resources.	terraform state list and plan; do not rerun blindly.

Step 33.3 - Demonstrate change plan, drift and destroy

Previous evidence	Terraform owns the sandbox resources.
Why this step now	Senior Terraform usage includes reading plan and reconciling drift, not only apply. A controlled label change shows in-place update; manual deletion demonstrates refresh.
Action	Run the copy-paste block below.
Expected evidence	After manual delete, plan proposes recreate. Final state list empty after destroy.
Next connection	Terraform lifecycle understood; Ansible can now configure a target from an inventory.

```
cd ~/projects/procureflow-devops-modernization/platform/terraform/local

# Insert a labels block after the image attribute.
sed -i '/image = docker_image.nginx.image_id/a\
\  labels {\
\    label = "managed-by"\
\    value = "terraform"\
\  }' main.tf

terraform fmt
terraform plan
terraform apply -auto-approve

docker rm -f procureflow-terraform-probe
terraform plan

# Recreate from desired state, then destroy cleanly
terraform apply -auto-approve
terraform destroy -auto-approve
terraform state list
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Recreate proposed	Drift detection works.	Record ADR for local vs Azure modules.
No drift detected	State refresh disabled or wrong daemon context.	Run terraform refresh/plan and verify provider endpoint.
Destroy leaves resources	Resource not in state or dependency issue.	Compare docker ps/network with terraform state; clean only understood leftovers.

34. Ansible legacy Linux configuration lab

Step 34.1 - Install Ansible and start a disposable SSH target

Previous evidence	Terraform provisioning semantics are proven.
Why this step now	Ansible needs a reachable target. A disposable Ubuntu SSH container simulates a legacy Linux node without requiring another full VM.
Action	Run the copy-paste block below.
Expected evidence	SSH returns container hostname and devops non-root user.
Next connection	Target connectivity proven; Ansible can enforce desired configuration idempotently.

```
sudo apt-get update
sudo apt-get install -y ansible
ansible --version

cd ~/projects/procureflow-devops-modernization
mkdir -p legacy-platform/ansible/files

# Create a lab-only SSH key
if [ ! -f legacy-platform/ansible/lab_key ]; then
  ssh-keygen -t ed25519 -N '' -f legacy-platform/ansible/lab_key
fi

cat > legacy-platform/ansible/Dockerfile.legacy-node <<'EOF'
FROM ubuntu:24.04
RUN apt-get update && DEBIAN_FRONTEND=noninteractive apt-get install -y \
    openssh-server python3 sudo curl ca-certificates && \
    mkdir -p /run/sshd && \
    useradd -m -s /bin/bash devops && \
    echo 'devops ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/devops
COPY lab_key.pub /home/devops/.ssh/authorized_keys
RUN chown -R devops:devops /home/devops/.ssh && chmod 700 /home/devops/.ssh && chmod 600
/home/devops/.ssh/authorized_keys
EXPOSE 22
CMD ["/usr/sbin/sshd", "-D", "-e"]
EOF

docker build -t procureflow/legacy-node:1.0 \
  -f legacy-platform/ansible/Dockerfile.legacy-node \
  legacy-platform/ansible

docker rm -f procureflow-legacy-node 2>/dev/null || true
docker run -d --name procureflow-legacy-node -p 127.0.0.1:12222:22 procureflow/legacy-node:1.0

ssh -o StrictHostKeyChecking=no \
  -i legacy-platform/ansible/lab_key \
  -p 12222 devops@127.0.0.1 'hostname && id'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
SSH works	Target reachable and Python installed.	Create inventory/playbook.
Permission denied publickey	Key ownership/copy issue.	Inspect authorized_keys permissions and public key content.
Connection refused	sshd/container/port mapping issue.	docker logs and docker port inspect.
Host key changed	Disposable target recreated.	Remove only matching localhost:12222 known_hosts entry.

Step 34.2 - Inventory and idempotent playbook

Previous evidence	SSH target works manually.
Why this step now	Ansible troubleshooting starts by proving transport, then applying package/user/file/service state. Second run should report no changes.
Action	Run the copy-paste block below.
Expected evidence	Ping pong; first play changes resources; second play changed=0; Java 21 and marker file visible.
Next connection	Configuration automation proven; now delivery automation can build and promote application artifacts.

```
cd ~/projects/procureflow-devops-modernization

cat > legacy-platform/ansible/inventory.ini <<'EOF'
[legacy]
legacy-node ansible_host=127.0.0.1 ansible_port=12222 ansible_user=devops
ansible_ssh_private_key_file=./lab_key ansible_python_interpreter=/usr/bin/python3
EOF

cat > legacy-platform/ansible/site.yaml <<'EOF'
- name: Configure ProcureFlow legacy Linux node
  hosts: legacy
  become: true
  tasks:
    - name: Install Java and operations tools
      ansible.builtin.apt:
        name:
          - openjdk-21-jre-headless
          - curl
          - jq
        state: present
        update_cache: true

    - name: Create application group
      ansible.builtin.group:
        name: procureflow
        state: present

    - name: Create application user
      ansible.builtin.user:
        name: procureflow
        group: procureflow
        shell: /usr/sbin/nologin
        create_home: false

    - name: Create application directory
      ansible.builtin.file:
        path: /opt/procureflow
        state: directory
        owner: procureflow
        group: procureflow
        mode: '0750'

    - name: Write managed environment marker
      ansible.builtin.copy:
        dest: /opt/procureflow/managed-by-ansible.txt
        content: "environment=legacy-lab
managed_by=ansible
"
        owner: procureflow
        group: procureflow
        mode: '0640'
EOF

cd legacy-platform/ansible
ansible all -i inventory.ini -m ping
ansible-playbook -i inventory.ini site.yaml
ansible-playbook -i inventory.ini site.yaml
ansible legacy -i inventory.ini -b -a 'java -version'
ansible legacy -i inventory.ini -b -a 'cat /opt/procureflow/managed-by-ansible.txt'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Second run changed=0	Idempotency proven.	Commit automation code without private key.
Second run changes apt cache only	Cache update can report changes depending module behavior.	Use cache_valid_time and evaluate task-specific idempotency.
Python interpreter error	Target lacks correct Python path.	Verify ansible_python_interpreter.
Become failure	sudo policy missing.	Fix target privilege model, not Ansible command flags randomly.

Step 34.3 - Protect lab key and commit IaC code

Previous evidence	Terraform and Ansible labs pass.
Why this step now	Private keys and Terraform state must not enter Git. Code and provider lock file may be committed; runtime state and secrets remain local/remote backend.
Action	Run the copy-paste block below.

Expected evidence	Commit contains .tf/.yaml/Dockerfile but not private key or state.
Next connection	Provisioning and configuration layers documented; continuous integration is next.

```
cd ~/projects/procureflow-devops-modernization

cat >> .gitignore <<'EOF'
legacy-platform/ansible/lab_key
legacy-platform/ansible/lab_key.pub
platform/terraform/local/tfplan
EOF

git add .gitignore platform/terraform/local legacy-platform/ansible
git reset legacy-platform/ansible/lab_key legacy-platform/ansible/lab_key.pub 2>/dev/null || true
git reset platform/terraform/local/terraform.tfstate* 2>/dev/null || true
git commit -m "feat(iac): add Terraform lifecycle and Ansible legacy-node configuration"
git status --short
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Safe commit	IaC evidence ready.	Build Jenkins CI stack.
Key/state tracked	Security issue.	Remove from index/history and rotate key before pushing remote.

Part XI - Jenkins Continuous Integration with Groovy

35. CI responsibility and boundary

```
Git commit
   |
   v
Jenkins CI
   +-- checkout
   +-- unit/integration tests
   +-- build
   +-- code quality
   +-- image build
   +-- vulnerability scan
   +-- push immutable image
   |
   v
GitOps change / pull request
   |
   v
Argo CD owns deployment
```

Why Jenkins does not run kubectl apply to production
Agar Jenkins build bhi kare aur directly cluster mutate bhi kare, CI credential blast radius large hota hai aur deployed desired state Git se drift kar sakti hai. Jenkins artifact produce karega; GitOps repository image version approve karegi; Argo CD reconcile karega.

36. Jenkins and SonarQube local stack

Step 36.1 - Prepare host kernel and Jenkins custom image

Previous evidence	Application, registry and Kubernetes platform work manually.
Why this step now	CI automation ab known-good commands ko repeat karegi. Jenkins agent ko Git, Node.js, Maven and Docker CLI chahiye; Docker daemon host socket se use hoga.
Action	Run the copy-paste block below.
Expected evidence	Custom Jenkins image builds successfully.
Next connection	Jenkins image exists; compose can connect it to SonarQube, PostgreSQL and Docker socket.

```
cd ~/projects/procureflow-devops-modernization

# SonarQube/Elasticsearch-based components need enough virtual memory maps.
sudo sysctl -w vm.max_map_count=524288

cat > ci/jenkins/Dockerfile <<'EOF'
FROM jenkins/jenkins:lts-jdk21
USER root
RUN apt-get update && DEBIAN_FRONTEND=noninteractive apt-get install -y \
    ca-certificates curl git jq docker.io nodejs npm maven && \
    rm -rf /var/lib/apt/lists/*
RUN jenkins-plugin-cli --plugins \
    workflow-aggregator \
    git \
    credentials-binding \
    pipeline-stage-view \
    junit \
    warnings-ng
RUN groupadd -f docker && usermod -aG docker jenkins
USER jenkins
EOF

docker build -t procureflow/jenkins:local ci/jenkins
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Build successful	CI runtime dependencies packaged.	Create Compose stack.
Plugin download failure	Update center/network issue.	Retry after network check; keep plugin list pinned/recorded.
vm.max_map_count permission/WSL reset	Kernel setting may reset after WSL restart.	Add documented startup check; do not

Observation	Meaning	Next decision
		assume persistence.
nodejs version older than app target	Distribution package differs from Node 22 image.	Use containerized Node test or NodeSource/nvm in image; record version contract.

Step 36.2 - Start Jenkins, SonarQube and Sonar PostgreSQL

Previous evidence	Custom Jenkins image exists.
Why this step now	CI and code-quality services need stable named volumes and network. Compose manages this support stack separately from Kubernetes application workloads.
Action	Run the copy-paste block below.
Expected evidence	All three containers Up; Sonar API status UP; Jenkins initial password available.
Next connection	Support services available; one-time credentials and Jenkins global tools now configure pipeline execution.

```
cd ~/projects/procureflow-devops-modernization

cat > ci/jenkins/compose.yaml <<'EOF'
services:
  jenkins:
    image: procureflow/jenkins:local
    container_name: procureflow-jenkins
    restart: unless-stopped
    user: root
    ports:
      - "127.0.0.1:18080:8080"
      - "127.0.0.1:50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    extra_hosts:
      - "host.docker.internal:host-gateway"
    networks: [ci]

  sonar-db:
    image: postgres:17.5-alpine
    container_name: procureflow-sonar-db
    restart: unless-stopped
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar-local-only
      POSTGRES_DB: sonar
    volumes:
      - sonar_db:/var/lib/postgresql/data
    networks: [ci]

  sonarqube:
    image: sonarqube:community
    container_name: procureflow-sonarqube
    restart: unless-stopped
    depends_on: [sonar-db]
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar-local-only
    ports:
      - "127.0.0.1:19000:9000"
    volumes:
      - sonar_data:/opt/sonarqube/data
      - sonar_extensions:/opt/sonarqube/extensions
      - sonar_logs:/opt/sonarqube/logs
    networks: [ci]

networks:
  ci:
    name: procureflow-ci

volumes:
  jenkins_home:
  sonar_db:
  sonar_data:
  sonar_extensions:
  sonar_logs:
EOF

docker compose -p procureflow-ci -f ci/jenkins/compose.yaml up -d
docker compose -p procureflow-ci -f ci/jenkins/compose.yaml ps

for i in {1..90}; do
  if curl -fsS http://127.0.0.1:19000/api/system/status | jq -e '.status == "UP"' >/dev/null; then
    echo 'SonarQube is UP'
    break
  fi
  sleep 5
done

printf '
Jenkins initial password:
'
docker exec procureflow-jenkins cat /var/jenkins_home/secrets/initialAdminPassword
printf '
Jenkins URL: http://127.0.0.1:18080
SonarQube URL: http://127.0.0.1:19000
'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Services healthy	CI support platform ready.	Complete Jenkins UI setup.
Sonar exits with bootstrap checks	vm.max_map_count/memory issue.	Inspect docker logs; apply sysctl and

Observation	Meaning	Next decision
		increase profile capacity.
Jenkins docker permission denied	Socket group ID mismatch inside container.	Inspect host socket GID and create image/group mapping, or run local lab as root with documented risk.
Ports occupied	Previous services conflict.	Identify listener and select documented local ports.

One-time UI setup checklist

11. Open Jenkins at http://127.0.0.1:18080 and enter the initial admin password.
12. Install suggested plugins; create a local administrator user. Do not reuse a real production password.
13. In Manage Jenkins -> Credentials, later add GitHub token as Secret text with ID github-token only if automated Git push is enabled.
14. Open SonarQube at http://127.0.0.1:19000, change default admin password, create project procureflow and generate a token.
15. Store Sonar token in Jenkins credentials with ID sonar-token. Never put it in Jenkinsfile or Git.
16. Configure SonarQube server URL as http://sonarqube:9000 because Jenkins and SonarQube share Docker network procureflow-ci.

37. Build scripts and Jenkinsfile

Step 37.1 - Add deterministic service build script

Previous evidence	Jenkins and SonarQube services are reachable.
Why this step now	Pipeline stages should call version-controlled scripts rather than contain every shell detail. This allows the same script to run locally and in CI.
Action	Run the copy-paste block below.
Expected evidence	Script syntax passes, image scans without blocking findings, and push succeeds.
Next connection	Build/push logic reusable; pipeline can orchestrate tests, quality and parallel service builds.

```
cd ~/projects/procureflow-devops-modernization

cat > ci/scripts/build-and-push.sh <<'EOF'
#!/usr/bin/env bash
set -Eeuo pipefail

SERVICE=${1:?Usage: build-and-push.sh <supplier|order> <tag>}
TAG=${2:?Usage: build-and-push.sh <supplier|order> <tag>}
REGISTRY=${REGISTRY:-localhost:5001/procureflow}

case "$SERVICE" in
  supplier)
    CONTEXT=applications/supplier-service-nodejs
    IMAGE_NAME=supplier-service
    ;;
  order)
    CONTEXT=applications/order-service-java
    IMAGE_NAME=order-service
    ;;
  *)
    echo "Unsupported service: $SERVICE" >&2
    exit 1
    ;;
esac

IMAGE="${REGISTRY}/${IMAGE_NAME}:${TAG}"
echo "Building ${IMAGE} from ${CONTEXT}"
docker build --pull -t "$IMAGE" "$CONTEXT"

echo "Scanning ${IMAGE}"
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:0.67.2 image \
  --severity HIGH,CRITICAL \
  --exit-code 1 \
  --ignore-unfixed \
  "$IMAGE"

echo "Pushing ${IMAGE}"
docker push "$IMAGE"
printf '%s
' "$IMAGE"
EOF

chmod +x ci/scripts/build-and-push.sh
bash -n ci/scripts/build-and-push.sh
REGISTRY=localhost:5001/procureflow \
  ci/scripts/build-and-push.sh supplier manual-script-test
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Scan and push pass	Reusable CI unit works.	Create test database script.
Trivy blocks HIGH/CRITICAL	Security gate found actionable issue.	Review CVEs, base image update, exception policy; do not blindly use --exit-code 0.
Registry push fails	Artifact path unavailable.	Return to registry health; CI logic is not root cause.
Dockerfile test not run	Image build alone may miss unit tests.	Pipeline test stages remain mandatory before this script.

Step 37.2 - Create Jenkinsfile with gates and immutable tag

Previous evidence	Build script works manually.
Why this step now	Git short SHA makes artifact traceable to source. Tests run before image build, and parallel service builds reduce pipeline duration without mixing artifacts.
Action	Run the copy-paste block below.
Expected evidence	Jenkinsfile committed; no token/password literal exists.
Next connection	Pipeline as code ready; Jenkins job can now produce traceable images.

```
cd ~/projects/procureflow-devops-modernization

cat > ci/scripts/ci-preflight.sh <<'EOF'
#!/usr/bin/env bash
set -Eeuo pipefail
docker version
curl -fsS http://host.docker.internal:5001/v2/
node --version
npm --version
java -version
mvn -version
EOF

cat > ci/scripts/test-node.sh <<'EOF'
#!/usr/bin/env bash
set -Eeuo pipefail
cd applications/supplier-service-nodejs
npm ci
npm test
EOF

cat > ci/scripts/test-java.sh <<'EOF'
#!/usr/bin/env bash
set -Eeuo pipefail
CI_NETWORK=${CI_NETWORK:-procureflow-ci}
docker rm -f order-ci-postgres 2>/dev/null || true
docker run -d --name order-ci-postgres \
  --network "$CI_NETWORK" \
  -e POSTGRES_DB=procureflow \
  -e POSTGRES_USER=procureflow \
  -e POSTGRES_PASSWORD=ci-only \
  postgres:17.5-alpine >/dev/null
trap 'docker rm -f order-ci-postgres >/dev/null 2>&1 || true' EXIT
for i in $(seq 1 60); do
  docker exec order-ci-postgres pg_isready -U procureflow && break
  sleep 2
done
cd applications/order-service-java
DATABASE_URL=jdbc:postgresql://order-ci-postgres:5432/procureflow \
DATABASE_USER=procureflow \
DATABASE_PASSWORD=ci-only \
mvn -B test
EOF

chmod +x ci/scripts/*.sh

cat > Jenkinsfile <<'EOF'
pipeline {
  agent any
  options {
    timestamps()
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '20'))
    timeout(time: 40, unit: 'MINUTES')
  }
  environment {
    REGISTRY = 'localhost:5001/procureflow'
    CI_NETWORK = 'procureflow-ci'
  }
  stages {
    stage('Checkout') {
      steps {
        checkout scm
        script {
          env.IMAGE_TAG = sh(script: 'git rev-parse --short=12 HEAD', returnStdout: true).trim()
        }
      }
    }
    stage('Preflight') {
      steps { sh 'ci/scripts/ci-preflight.sh' }
    }
    stage('Tests') {
      parallel {
        stage('Node.js tests') {
          steps { sh 'ci/scripts/test-node.sh' }
        }
        stage('Java integration test') {
          steps { sh 'CI_NETWORK=$CI_NETWORK ci/scripts/test-java.sh' }
        }
      }
    }
    stage('Build, scan and push') {
      parallel {
        stage('Supplier image') {
          steps { sh 'REGISTRY=$REGISTRY ci/scripts/build-and-push.sh supplier $IMAGE_TAG' }
        }
        stage('Order image') {
```

```
          steps { sh 'REGISTRY=$REGISTRY ci/scripts/build-and-push.sh order $IMAGE_TAG' }
        }
      }
    }
    stage('Publish metadata') {
      steps {
        script {
          def metadata = "commit=${GIT_COMMIT}\ntag=${IMAGE_TAG}\nsupplier=${REGISTRY}/supplier-service:$
{IMAGE_TAG}\norder=${REGISTRY}/order-service:${IMAGE_TAG}\n"
          writeFile file: 'artifact-metadata.txt', text: metadata
        }
        archiveArtifacts artifacts: 'artifact-metadata.txt', fingerprint: true
      }
    }
  }
  post {
    always {
      junit allowEmptyResults: true, testResults: '**/target/surefire-reports/*.xml'
      sh 'docker rm -f order-ci-postgres 2>/dev/null || true'
    }
    success { echo "Immutable images published with tag ${IMAGE_TAG}" }
    failure { echo 'Pipeline failed. Do not promote an artifact.' }
  }
}
EOF

git add Jenkinsfile ci/scripts ci/jenkins
git commit -m "feat(ci): add Jenkins Groovy pipeline with tests scans and immutable images"
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Pipeline file committed	SCM pipeline can be created.	Create Multibranch/Pipeline job.
Credentials found in file	Security gate failed.	Remove/rotate and use Jenkins credentials store.
Java test needs database network	CI temporary DB must share procureflow-ci network.	Verify docker network inspect procureflow-ci.
Shell script syntax fails	Pipeline orchestration should not start.	Run bash -n ci/scripts/*.sh locally and fix first.

Jenkins job creation

17. Create a Pipeline job named procureflow-ci.
18. Select Pipeline script from SCM, Git, and enter the repository URL.
19. For a private repository, choose the Git credential; never embed credentials in URL.
20. Set Script Path to Jenkinsfile and branch to main or a controlled feature branch.
21. Run Build Now. Capture Stage View, artifact-metadata.txt and console summary as evidence.

Pipeline output decision chain

Observation	Meaning	Next decision
Node test fails	Source behavior is not trustworthy.	Stop. Fix test/code. Do not build/push.
Java DB test fails	Schema, credentials or persistence contract broken.	Inspect temporary PostgreSQL and test stack trace.
Trivy blocks image	Artifact violates vulnerability policy.	Update dependency/base image or document approved exception.
Push succeeds	Immutable artifact exists.	Create GitOps image-tag change; do not deploy via Jenkins.
One parallel build fails	Release set incomplete.	Do not promote either service unless release policy permits independent promotion.

Part XII - Argo CD GitOps continuous delivery

38. Desired state and reconciliation

```
Git main branch
  | chart + environment values
  v
Argo CD desired state
  |
  | compare
  v
Live Kubernetes state
  |
  +---- same ------> Synced / Healthy
  |
  +---- different -> OutOfSync -> reconcile or alert
```

39. Install and access Argo CD

Step 39.1 - Install Argo CD in dedicated namespace

Previous evidence	Jenkins can produce images; deployment ownership is still manual Helm.
Why this step now	Argo CD must exist before handoff from manual Helm to GitOps. Dedicated namespace isolates platform controller from application namespaces.
Action	Run the copy-paste block below.
Expected evidence	Argo CD core pods Running and server accessible.
Next connection	Controller ready; CLI and repository/application configuration can be added.

```
cd ~/projects/procureflow-devops-modernization

kubectl create namespace argocd --dry-run=client -o yaml | kubectl apply -f -
kubectl apply -n argocd \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

kubectl wait --for=condition=Available deployment/argocd-server \
  -n argocd --timeout=600s
kubectl wait --for=condition=Available deployment/argocd-repo-server \
  -n argocd --timeout=600s
kubectl get pods -n argocd

kubectl port-forward service/argocd-server -n argocd 18443:443 \
  >/tmp/argocd-port-forward.log 2>&1 &
echo $! > /tmp/argocd-port-forward.pid

ARGOCD_PASSWORD=$(kubectl get secret argocd-initial-admin-secret \
  -n argocd \
  -o jsonpath='{.data.password}' | base64 -d)

echo 'Argo CD URL: https://127.0.0.1:18443'
echo "Initial admin password: ${ARGOCD_PASSWORD}"
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Pods healthy	GitOps controller ready.	Install CLI and register application.
Image pull/network failure	GitHub registry access issue.	Inspect pod events and network.
Server port-forward exits	Service/pod not ready or port used.	Check log file/pid and select free local port.
Stable URL changed unexpectedly	Floating stable manifest changed.	For long-term reproducibility pin a tested release tag and record it in tool versions.

Step 39.2 - Install Argo CD CLI and authenticate

Previous evidence	Argo CD server is reachable by port-forward.
Why this step now	CLI gives scriptable app status, diff, sync and rollback evidence needed for interview/runbooks.
Action	Run the copy-paste block below.

Expected evidence	CLI version and logged-in admin user information shown.
Next connection	CLI authenticated; repository URL and desired Helm path can now define an Application.

```
cd /tmp
ARGOCD_VERSION=$(curl -fsSL https://api.github.com/repos/argoproj/argo-cd/releases/latest | jq
-r .tag_name)
curl -fsSLo argocd "https://github.com/argoproj/argo-cd/releases/download/${ARGOCD_VERSION}/argocd-linux-
amd64"
chmod +x argocd
sudo install -m 0755 argocd /usr/local/bin/argocd

ARGOCD_PASSWORD=$(kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath='{.data.password}'
| base64 -d)
argocd login 127.0.0.1:18443 \
  --username admin \
  --password "$ARGOCD_PASSWORD" \
  --insecure
argocd version --client
argocd account get-user-info
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Login succeeds	CLI path ready.	Create Application manifest.
x509 error	Local self-signed certificate.	Use --insecure only for local lab; production uses trusted TLS.
Connection refused	Port-forward unavailable.	Restart port-forward and verify service.
Latest API rate limit	GitHub unauthenticated API limit.	Use pinned version in .tool-versions.env.

40. Handoff from manual Helm to Argo CD

Step 40.1 - Push repository and create Argo Application template

Previous evidence	Argo CD is healthy and CLI works.
Why this step now	Argo repo-server cannot read your WSL working directory directly. A reachable Git remote is the source of truth. Compute remains local; Git remote provides collaboration/audit.
Action	Run the copy-paste block below.
Expected evidence	Git push succeeds and Application server dry run passes.
Next connection	Desired state is remotely reachable; ownership can transition from Helm CLI to Argo CD.

```
cd ~/projects/procureflow-devops-modernization

# Set your actual Git repository URL once.
read -r -p 'Enter Git repository URL: ' REPO_URL
[ -n "$REPO_URL" ]

git remote get-url origin >/dev/null 2>&1 \
  && git remote set-url origin "$REPO_URL" \
  || git remote add origin "$REPO_URL"

git push -u origin main

cat > platform/argocd/apps/procureflow-dev.yaml.template <<'EOF'
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: procureflow-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: __REPO_URL__
    targetRevision: main
    path: platform/helm/procureflow
    helm:
      valueFiles:
        - values.yaml
        - values-dev.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: procureflow-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false
EOF

sed "s|__REPO_URL__|${REPO_URL}|g" \
  platform/argocd/apps/procureflow-dev.yaml.template \
  > /tmp/procureflow-dev-application.yaml

kubectl apply --dry-run=server -f /tmp/procureflow-dev-application.yaml
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Remote push and dry run pass	Repo reachable contract likely valid.	Uninstall manual Helm release and apply Argo app.
Authentication failure on push	Git credential not configured.	Use SSH key or credential manager; never put token in script/URL.
Private repo inaccessible to Argo	Argo lacks repository credentials.	Add repo credentials through argocd repo add using secret/SSH key.
Template still contains placeholder	REPO_URL substitution failed.	grep __REPO_URL__ before apply.

Step 40.2 - Remove manual release and let Argo CD reconcile

Previous evidence	Argo Application manifest is valid and repository pushed.
Why this step now	Two controllers must not own same resources. Manual Helm release is removed, database manifests remain, then Argo recreates application resources from Git.
Action	Run the copy-paste block below.
Expected evidence	Argo app Synced/Healthy; application endpoints work again.
Next connection	Git now controls app state; drift and rollback can be demonstrated safely.

```
cd ~/projects/procureflow-devops-modernization

# Keep Gateway and databases; remove only ProcureFlow Helm-managed app resources.
helm uninstall procureflow-dev -n procureflow-dev
kubectl get deploy,svc,httproute -n procureflow-dev

kubectl apply -f /tmp/procureflow-dev-application.yaml
argocd app wait procureflow-dev \
  --sync \
  --health \
  --timeout 600

argocd app get procureflow-dev
kubectl get deploy,pod,svc,httproute -n procureflow-dev
curl -fsS http://127.0.0.1:8080/api/suppliers | jq .
curl -fsS http://127.0.0.1:8080/api/orders | jq .
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Synced and Healthy	GitOps ownership established.	Test self-heal and Git rollback.
ComparisonError repository	Repo URL/credentials/path issue.	argocd app get and repo-server logs.
Degraded due missing secret	Secret intentionally not in Git.	Pre-provision secret or use sealed/external secret solution.
OutOfSync generated defaults	Live fields differ from desired.	Inspect argocd app diff; configure ignoreDifferences only for justified controller fields.

Step 40.3 - Demonstrate self-heal, version promotion and rollback

Previous evidence	Argo app is Synced and Healthy.
Why this step now	GitOps value appears when manual drift is corrected and Git history drives release/rollback. We first create safe drift, then commit a tag change, then revert.
Action	Run the copy-paste block below.
Expected evidence	Manual scale returns to Git replica count; promoted image appears; git revert restores previous desired tag.
Next connection	Delivery chain complete: CI produces artifact, Git approves desired version, Argo deploys and self-heals.

```
# Safe drift: manually change replica count
kubectl scale deployment/supplier-service -n procureflow-dev --replicas=3
kubectl get deployment/supplier-service -n procureflow-dev -w &
WATCH_PID=$!
sleep 30
kill "$WATCH_PID" 2>/dev/null || true

argocd app get procureflow-dev

# Promotion example: replace tag with the immutable Jenkins SHA.
cd ~/projects/procureflow-devops-modernization
read -r -p 'Enter successful Jenkins IMAGE_TAG: ' IMAGE_TAG

sed -i "0,/tag: "1.0.0"/s//tag: "${IMAGE_TAG}"/" platform/helm/procureflow/values.yaml
git add platform/helm/procureflow/values.yaml
git commit -m "chore(gitops): promote supplier image ${IMAGE_TAG} to dev"
git push origin main

argocd app wait procureflow-dev --sync --health --timeout 600
kubectl get deployment/supplier-service -n procureflow-dev \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"
"}'

# Rollback by reverting the Git commit, not by hiding desired state.
git revert --no-edit HEAD
git push origin main
argocd app wait procureflow-dev --sync --health --timeout 600
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Self-heal and promotion work	GitOps reconciliation proven.	Add monitoring.
Manual replicas remain 3	SelfHeal disabled or app not tracking	Check syncPolicy and resource ownership.

Observation	Meaning	Next decision
	resource.	
New image ImagePullBackOff	Jenkins tag not present in registry or mirror issue.	Registry catalog/tag check; revert Git commit.
Revert sync healthy	Rollback is auditable in Git.	Capture Argo history/screenshots.

Part XIII - Prometheus, Grafana, service metrics, and alerting

41. Why observability comes after a working transaction

Previous evidence	Supplier and Order transactions work, persist data, and deploy through GitOps.
Why this step now	Monitoring before defining the business path produces many infrastructure graphs but weak operational meaning. Now we know which request, dependency and failure matters.
Action	Install Prometheus/Grafana, discover application metrics, build queries, define SLI/SLO and alerts.
Expected evidence	Targets Up, application metrics queryable, dashboards show traffic/resource behavior, alerts fire during controlled failure.
Next connection	These signals become the evidence used by scaling and incident decisions.

42. Install kube-prometheus-stack

Step 42.1 - Install monitoring stack with explicit selectors

Previous evidence	GitOps application is healthy.
Why this step now	Prometheus Operator manages ServiceMonitor and PrometheusRule resources. Selector settings must allow our application monitors rather than silently ignoring them.
Action	Run the copy-paste block below.
Expected evidence	Prometheus, Grafana, Alertmanager and operator pods Running/Ready.
Next connection	Prometheus exists but does not yet know application endpoints; ServiceMonitor provides discovery contract.

```
cd ~/projects/procureflow-devops-modernization

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

cat > observability/prometheus/values.yaml <<'EOF'
grafana:
  service:
    type: ClusterIP
  persistence:
    enabled: false
  resources:
    requests: {cpu: 100m, memory: 256Mi}
    limits: {cpu: 500m, memory: 512Mi}

prometheus:
  prometheusSpec:
    retention: 3d
    serviceMonitorSelectorNilUsesHelmValues: false
    podMonitorSelectorNilUsesHelmValues: false
    ruleSelectorNilUsesHelmValues: false
    resources:
      requests: {cpu: 200m, memory: 512Mi}
      limits: {cpu: "1", memory: 1Gi}

alertmanager:
  alertmanagerSpec:
    resources:
      requests: {cpu: 50m, memory: 128Mi}
      limits: {cpu: 300m, memory: 256Mi}
EOF

helm upgrade --install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  --values observability/prometheus/values.yaml \
  --wait \
  --timeout 15m

kubectl get pods -n monitoring
kubectl get prometheus,alertmanager -n monitoring
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
All core pods Ready	Monitoring control plane ready.	Create ServiceMonitors.
Pending due resources	Current profile exceeds capacity.	Scale down Kafka/Elasticsearch/SonarQube before monitoring.
CRD ownership conflict	Previous monitoring install exists.	Inspect Helm releases/CRDs; avoid deleting shared CRDs blindly.
Prometheus OOM	Retention/resource profile too high.	Reduce retention and optional components; preserve evidence of capacity decision.

Step 42.2 - Add ServiceMonitor to Helm chart

Previous evidence	Prometheus Operator is available.
Why this step now	Application chart knows each metrics path and service port. Therefore ServiceMonitor should be generated from the same values to avoid duplicated configuration drift.
Action	Run the copy-paste block below.
Expected evidence	Two ServiceMonitors created with release=monitoring and correct paths.
Next connection	Discovery resources exist; next query proves Prometheus actually scraped metrics.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/helm/procureflow/templates/servicemonitors.yaml <<'EOF'
{{- range $key, $svc := .Values.services }}
{{- if and $svc.enabled $svc.metrics.enabled }}
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: {{ $svc.name }}
  labels:
    release: monitoring
    {{- include "procureflow.labels" $ | nindent 4 }}
spec:
  namespaceSelector:
    matchNames:
      - {{ $.Release.Namespace }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ $svc.name }}
  endpoints:
    - port: http
      path: {{ $svc.metrics.path }}
      interval: 15s
      scrapeTimeout: 10s
---
{{- end }}
{{- end }}
EOF

helm lint platform/helm/procureflow
git add platform/helm/procureflow/templates/servicemonitors.yaml
git commit -m "feat(observability): add application ServiceMonitors"
git push origin main
argocd app wait procureflow-dev --sync --health --timeout 600

kubectl get servicemonitor -n procureflow-dev
kubectl get servicemonitor supplier-service -n procureflow-dev -o yaml
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
ServiceMonitors present	Discovery objects ready.	Verify Prometheus targets/queries.
No matches for ServiceMonitor	Monitoring CRDs missing.	Recheck kube-prometheus-stack install.
Argo Degraded	Template API/selector issue.	argocd app diff and Helm render.
Only one monitor	One service metrics.enabled false or rendering issue.	Inspect values and rendered resources.

Step 42.3 - Access Prometheus/Grafana and verify targets

Previous evidence	ServiceMonitors are live.
Why this step now	Kubernetes object existence is not scrape success. Prometheus targets and metric query must prove end-to-end collection.
Action	Run the copy-paste block below.
Expected evidence	Application target up value 1 and supplier request metric result visible.
Next connection	Metric pipeline proven; we can define useful signals instead of decorative dashboards.

```
kubectl port-forward -n monitoring service/monitoring-kube-prometheus-prometheus 19090:9090 \
  >/tmp/prometheus-port-forward.log 2>&1 &
echo $! > /tmp/prometheus-port-forward.pid

kubectl port-forward -n monitoring service/monitoring-grafana 13000:80 \
  >/tmp/grafana-port-forward.log 2>&1 &
echo $! > /tmp/grafana-port-forward.pid

GRAFANA_PASSWORD=$(kubectl get secret monitoring-grafana -n monitoring \
  -o jsonpath='{.data.admin-password}' | base64 -d)

echo 'Prometheus: http://127.0.0.1:19090'
echo 'Grafana: http://127.0.0.1:13000'
echo "Grafana user=admin password=${GRAFANA_PASSWORD}"

sleep 20
curl -fsSG http://127.0.0.1:19090/api/v1/query \
  --data-urlencode 'query=up{namespace="procureflow-dev"}' \
  | jq '.data.result[] | {metric: .metric, value: .value[1]}'

curl -fsSG http://127.0.0.1:19090/api/v1/query \
  --data-urlencode 'query=procureflow_supplier_http_requests_total' \
  | jq '.data.result'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
up=1 and metric result	Scraping works.	Create operational queries and alerts.
No target result	ServiceMonitor not selected or target labels differ.	Inspect Prometheus serviceMonitorSelector and target page.
up=0	Prometheus discovered but cannot scrape.	Check service endpoints, port name, metrics path, NetworkPolicy.
Metric empty but up=1	No request yet or metric name differs.	Generate supplier traffic; inspect raw /metrics.

43. Signals, SLI, SLO, and dashboard queries

Golden signals mapped to ProcureFlow

Signal	Question answered	Example query/evidence
Traffic	Request rate	rate(procureflow_supplier_http_requests_total[5m])
Errors	Non-success responses	sum(rate(procureflow_supplier_http_requests_total{status!="200"}[5m]))
Latency	Request duration histogram when instrumented	histogram_quantile(0.95, sum by (le) (rate(http_server_requests_seconds_bucket[5m])))
Saturation	CPU/memory vs requests/limits	container_cpu_usage_seconds_total and working_set_bytes
Database	Readiness and pool utilization	up plus HikariCP active/pending metrics
Kafka	Consumer lag	consumer group lag exporter metric or CLI evidence
RabbitMQ	Queue depth/unacked	rabbitmq queue metrics/exporter
Business	Orders created per minute	rate(procureflow_orders_created_total[5m])

Suggested SLI/SLO

SLI is measured behavior. SLO is target. SLA is external agreement and consequences. Local project SLO is engineering target, not customer contract.

Observation	Meaning	Next decision
Availability SLI	Successful requests / total requests	99.5% over rolling 30 days for production design
Latency SLI	Proportion below 500 ms	95% supplier reads below 500 ms
Order durability SLI	Created orders readable after commit	99.99% successful committed writes
Delivery freshness SLI	Time from Git approval to Healthy	95% under 10 minutes

Observation	Meaning	Next decision
Recovery SLI	Time from incident declaration to restoration	RTO target depends on service criticality

44. Prometheus alerts

Step 44.1 - Add application health and restart alerts

Previous evidence	Application metrics are queryable.
Why this step now	Alerts should represent actionable risk. Pod restart and scrape-down signals are useful only with a sustained duration to avoid transient rollout noise.
Action	Run the copy-paste block below.
Expected evidence	Rules appear in Prometheus API with inactive/pending/firing state.
Next connection	Alert is loaded; controlled failure now proves notification signal behavior.

```
cd ~/projects/procureflow-devops-modernization

cat > observability/prometheus/procureflow-rules.yaml <<'EOF'
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: procureflow-alerts
  namespace: monitoring
  labels:
    release: monitoring
spec:
  groups:
    - name: procureflow.availability
      rules:
        - alert: ProcureFlowTargetDown
          expr: up{namespace="procureflow-dev",service=~"supplier-service|order-service"} == 0
          for: 2m
          labels:
            severity: critical
            service: procureflow
          annotations:
            summary: "ProcureFlow metrics target is down"
            description: "Prometheus cannot scrape {{ $labels.service }} for more than 2 minutes."

        - alert: ProcureFlowContainerRestarting
          expr: increase(kube_pod_container_status_restarts_total{namespace="procureflow-
dev",container=~"supplier-service|order-service"}[10m]) > 2
          for: 5m
          labels:
            severity: warning
            service: procureflow
          annotations:
            summary: "ProcureFlow container is repeatedly restarting"
            description: "Pod {{ $labels.pod }} restarted more than twice in ten minutes."
EOF

kubectl apply -f observability/prometheus/procureflow-rules.yaml
kubectl get prometheusrule procureflow-alerts -n monitoring -o yaml

curl -fsS http://127.0.0.1:19090/api/v1/rules \
  | jq '.data.groups[] | select(.name=="procureflow.availability") | .rules[] | {name, state}'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Rules loaded	Alert evaluation active.	Inject controlled target failure.
Rule absent	Prometheus rule selector mismatch.	Check release label and ruleSelector settings.
Parse error	PromQL/YAML invalid.	kubectl describe PrometheusRule and operator logs.
Alert immediately firing unexpectedly	Label selector matches wrong/missing target.	Inspect query result labels before changing thresholds.

Step 44.2 - Controlled alert validation

Previous evidence	Target-down alert is loaded.
Why this step now	An untested alert is only configuration. We cause a reversible scrape failure, observe Pending/Firing, then restore and verify resolution.
Action	Run the copy-paste block below.
Expected evidence	Alert transitions Pending/Firing after duration, then resolves after service restoration.
Next connection	Monitoring now produces tested evidence for incident triage and SLO discussion.

```
# Save current replica count
CURRENT_REPLICAS=$(kubectl get deployment/supplier-service -n procureflow-dev -o
jsonpath='{.spec.replicas}')

# Pause Argo self-heal for the test to prevent immediate restoration.
argocd app set procureflow-dev --self-heal=false
kubectl scale deployment/supplier-service -n procureflow-dev --replicas=0

for i in {1..18}; do
  date -Is
  curl -fsS http://127.0.0.1:19090/api/v1/alerts \
    | jq '.data.alerts[] | select(.labels.alertname=="ProcureFlowTargetDown") | {state,activeAt,labels}'
  sleep 20
done

kubectl scale deployment/supplier-service -n procureflow-dev --replicas="$CURRENT_REPLICAS"
kubectl rollout status deployment/supplier-service -n procureflow-dev --timeout=180s
argocd app set procureflow-dev --self-heal=true
argocd app sync procureflow-dev

curl -fsS http://127.0.0.1:8080/api/suppliers | jq .
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Alert fires and resolves	Monitoring loop validated.	Document incident response link.
No alert	Query labels do not match actual target.	Use Prometheus query UI to inspect up labels and adjust selector.
Argo restores replicas during test	Self-heal remained enabled.	Use controlled maintenance annotation/Argo setting; restore immediately afterward.
Service not restored	Rollback step incomplete.	Scale to Git desired value, enable self-heal, sync and test endpoint.

45. Centralized logs - staged implementation

First operational log source is kubectl logs because it is always available. Centralized log profile can add Loki with Grafana Alloy after metrics baseline is stable. Since chart values change across releases, validate current chart values with helm show values and pin tested chart versions before committing.

```
# Immediate log triage contract
kubectl logs deployment/order-service -n procureflow-dev --tail=200
kubectl logs deployment/order-service -n procureflow-dev --previous --tail=200
kubectl logs -n procureflow-dev -l app.kubernetes.io/name=order-service --since=15m --prefix

# Verify current official chart values before installing the optional log profile
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
helm search repo grafana/loki --versions | head
helm show values grafana/loki > /tmp/loki-current-values.yaml
helm show values grafana/alloy > /tmp/alloy-current-values.yaml
```

Observability explanation

I first proved the transaction, then instrumented the services, installed Prometheus Operator, created ServiceMonitors from the same Helm values, verified live targets, defined SLIs/SLOs, and tested an alert with controlled failure and recovery. I avoided claiming a dashboard was useful until its data path and alert action were validated.

Part XIV - Security, maintainability, and controlled access

46. Layered security model

```
Source security
  | branch review, secret scanning, dependency checks
  v
Build security
  | immutable tags, Trivy, SBOM
  v
Registry security
  | controlled access, no latest promotion
  v
Kubernetes security
  | non-root, dropped capabilities, RBAC, quotas
  v
Runtime security
  | probes, network controls, audit/evidence
  v
Operational security
  | least privilege, approved runbooks, rollback
```

47. Source and image security gates

Step 47.1 - Scan repository, IaC and images with Trivy

Previous evidence	CI and application artifacts exist.
Why this step now	Security scanning at multiple layers catches different risks: secrets/misconfiguration in repository and vulnerabilities in final runtime image.
Action	Run the copy-paste block below.
Expected evidence	Scans complete; blocking findings are reviewed; SBOM contains component count.
Next connection	Artifact security evidence exists; runtime identity should now avoid default broad access.

```
cd ~/projects/procureflow-devops-modernization

# Repository filesystem scan
docker run --rm -v "$PWD:/workspace" -w /workspace \
  aquasec/trivy:0.67.2 fs \
  --scanners vuln,secret,misconfig \
  --severity HIGH,CRITICAL \
  --ignore-unfixed \
  .

# Kubernetes/Helm rendered configuration scan
helm template procureflow platform/helm/procureflow \
  -f platform/helm/procureflow/values.yaml \
  -f platform/helm/procureflow/values-dev.yaml \
  -n procureflow-dev > /tmp/procureflow-security-render.yaml

docker run --rm -v /tmp:/scan aquasec/trivy:0.67.2 config \
  --severity HIGH,CRITICAL \
  /scan/procureflow-security-render.yaml

# SBOM generation for one immutable image
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /tmp:/reports \
  aquasec/trivy:0.67.2 image \
  --format cyclonedx \
  --output /reports/supplier-sbom.json \
  localhost:5001/procureflow/supplier-service:1.0.0

test -s /tmp/supplier-sbom.json
jq '.components | length' /tmp/supplier-sbom.json
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
No blocking findings	Artifact meets current local policy.	Add RBAC/service account.
Secret detected	Potential credential in repo.	Remove, rotate, and clean Git history; never only add ignore rule.

Observation	Meaning	Next decision
Misconfiguration finding	Manifest weakens runtime security.	Evaluate fix and operational compatibility.
Unfixed CVE	No upstream fix available.	Risk-assess exposure, compensating controls, and exception expiry.

48. Service account and least privilege

Step 48.1 - Add dedicated service account with no unnecessary API permissions

Previous evidence	Images and manifests scanned.
Why this step now	Application does not need Kubernetes API access. Dedicated service account with token automount disabled reduces credential exposure inside compromised pod.
Action	Run the copy-paste block below.
Expected evidence	Deployment Healthy and TOKEN_NOT_MOUNTED prints.
Next connection	Runtime credential exposure reduced; network control can be designed next.

```
cd ~/projects/procureflow-devops-modernization

cat > platform/helm/procureflow/templates/serviceaccount.yaml <<'EOF'
apiVersion: v1
kind: ServiceAccount
metadata:
  name: procureflow-runtime
  labels:
    {{- include "procureflow.labels" . | nindent 4 }}
automountServiceAccountToken: false
EOF

# Insert serviceAccountName and token policy into pod spec if not already present.
sed -i '/    spec:/a\      serviceAccountName: procureflow-runtime
      automountServiceAccountToken: false' \
  platform/helm/procureflow/templates/deployments.yaml

helm lint platform/helm/procureflow
helm template procureflow platform/helm/procureflow \
  -f platform/helm/procureflow/values.yaml \
  -f platform/helm/procureflow/values-dev.yaml \
  -n procureflow-dev \
  | grep -A3 -B3 'automountServiceAccountToken'

git add platform/helm/procureflow
git commit -m "feat(security): use tokenless dedicated runtime service account"
git push origin main
argocd app wait procureflow-dev --sync --health --timeout 600

kubectl exec deployment/supplier-service -n procureflow-dev -- \
  sh -c 'test ! -e /var/run/secrets/kubernetes.io/serviceaccount/token && echo TOKEN_NOT_MOUNTED'
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Token not mounted	Least-privilege runtime identity improved.	Review other security contexts.
Application fails	It unexpectedly depended on Kubernetes API token.	Identify legitimate need; create explicit minimal Role/RoleBinding rather than restoring default token.
Duplicate serviceAccountName	sed was run twice.	Clean template and rely on idempotent source edits.
Argo diff repeats	Generated/default field disagreement.	Inspect exact desired/live values; avoid blanket ignore.

49. NetworkPolicy reality on kind

> **CNI enforcement caveat**
> A NetworkPolicy object can be accepted by Kubernetes while the default kind networking does not enforce it. Do not claim enforcement without a policy-capable CNI such as Calico or Cilium and a connectivity test. In this main lab, policies

are design artifacts; a separate cluster profile should be used for enforcement testing.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: order-service-ingress
  namespace: procureflow-dev
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: order-service
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: envoy-gateway-system
      ports:
        - protocol: TCP
          port: 8080
```

Security decisions to explain

Observation	Meaning	Next decision
Non-root container	Reduces impact of process compromise.	Verify docker/kubectl exec id.
Read-only root filesystem	Prevents arbitrary persistent writes to image layer.	Mount explicit emptyDir for legitimate temp writes.
Drop ALL capabilities	Minimizes Linux privilege surface.	Add only a capability with documented requirement.
No service-account token	App cannot silently call Kubernetes API.	Create minimum Role only if business need exists.
Immutable Git SHA tag	Traceable artifact and predictable rollback.	Never promote mutable latest tag.
Secret reference	Password absent from values/Git.	Use External Secrets/Key Vault in Azure target.
Scan gate	Known high-risk artifacts blocked before promotion.	Exceptions need owner, reason and expiry.

Part XV - Performance baseline, load test, and autoscaling

50. Why baseline precedes scaling

Previous evidence	Application is observable and resource requests exist.
Why this step now	Autoscaling without baseline may scale around inefficient code, wrong probe behavior or tracking errors.
Action	Record low-load latency/error/CPU, run controlled k6 load, observe saturation, enable HPA, then compare.
Expected evidence	Before/after evidence shows whether scaling improves service behavior and what new bottleneck appears.
Next connection	Result drives right-sizing and cost decisions.

Step 50.1 - Create k6 baseline test

Previous evidence	Prometheus and application endpoints work.
Why this step now	A repeatable test script defines the same traffic, thresholds and duration each time. Supplier work query intentionally burns limited CPU so HPA signal is observable.
Action	Run the copy-paste block below.
Expected evidence	k6 summary shows request rate, error rate and p95; baseline saved.
Next connection	Baseline creates comparison point; HPA can now be tested against a known saturation pattern.

```
cd ~/projects/procureflow-devops-modernization

cat > load-testing/k6/supplier-load.js <<'EOF'
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 5 },
    { duration: '90s', target: 25 },
    { duration: '30s', target: 0 }
  ],
  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<800']
  }
};

const base = __ENV.BASE_URL || 'http://host.docker.internal:8080';

export default function () {
  const response = http.get(`${base}/api/suppliers?work=40`);
  check(response, {
    'status is 200': (r) => r.status === 200,
    'response has suppliers': (r) => r.body.includes('suppliers')
  });
  sleep(0.1);
}
EOF

kubectl top pods -n procureflow-dev

docker run --rm \
  -i \
  -e BASE_URL=http://host.docker.internal:8080 \
  -v "$PWD/load-testing/k6:/scripts" \
  grafana/k6:0.57.0 run /scripts/supplier-load.js \
  | tee docs/evidence/03-k6-baseline.txt

kubectl top pods -n procureflow-dev
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Thresholds pass	Current single replica handles baseline.	Enable HPA and increase load.
p95 fails with low CPU	Latency bottleneck may be app/IO, not CPU.	Inspect code, DB, network and traces; HPA may not help.
CPU saturates and errors rise	Scale candidate exists.	Enable HPA and compare.
Connection failures	Gateway/application availability issue.	Stop load and troubleshoot before scaling.

Step 50.2 - Enable HPA through GitOps and observe scaling

Previous evidence	Baseline metrics and CPU behavior are recorded.
Why this step now	Autoscaling setting should be desired state, not temporary kubectl autoscale. Git change enables repeatable review and rollback.
Action	Run the copy-paste block below.
Expected evidence	HPA increases supplier replicas under CPU load and later scales down; k6 comparison captured.
Next connection	Scaling evidence now supports reliability and cost trade-off discussion.

```
cd ~/projects/procureflow-devops-modernization

sed -i 's/^hpa:
  enabled: false/hpa:
  enabled: true/' platform/helm/procureflow/values.yaml || true
# If the multi-line sed did not match, edit only the first exact enabled line under hpa.
awk '
  /^hpa:/ {in_hpa=1}
  in_hpa && /enabled: false/ && !done {$0="  enabled: true"; done=1}
  {print}
' platform/helm/procureflow/values.yaml > /tmp/values.yaml
mv /tmp/values.yaml platform/helm/procureflow/values.yaml

git add platform/helm/procureflow/values.yaml
git commit -m "feat(scaling): enable HPA for local load validation"
git push origin main
argocd app wait procureflow-dev --sync --health --timeout 600

kubectl get hpa -n procureflow-dev
kubectl get hpa supplier-service -n procureflow-dev -w > /tmp/hpa-watch.txt &
HPA_WATCH_PID=$!

docker run --rm -i \
  -e BASE_URL=http://host.docker.internal:8080 \
  -v "$PWD/load-testing/k6:/scripts" \
  grafana/k6:0.57.0 run /scripts/supplier-load.js \
  | tee docs/evidence/04-k6-hpa.txt

sleep 60
kill "$HPA_WATCH_PID" 2>/dev/null || true
cat /tmp/hpa-watch.txt
kubectl get deployment,hpa -n procureflow-dev
kubectl top pods -n procureflow-dev
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
Replicas increase and p95 improves/stabilizes	CPU-based scaling helps this workload.	Right-size min/max and requests.
HPA unknown CPU	Metrics or CPU request missing.	kubectl describe hpa; verify requests.cpu and Metrics Server.
No scale despite high latency	CPU metric not saturated or bottleneck elsewhere.	Use correct metric; do not force low threshold blindly.
Scale creates DB/downstream bottleneck	Application tier no longer limiting.	Protect downstream with connection pools, queueing and rate limits.

Performance decision chain

Observation	Meaning	Next decision
CPU high, p95 high, error rate rises	Compute saturation likely.	HPA/right-size; then re-test downstream.
CPU low, p95 high	Waiting on database/network/lock/external dependency.	Trace and inspect dependency metrics; HPA likely wasteful.
More pods but no improvement	Shared bottleneck or load-balancer/session issue.	Inspect DB pool, Kafka partitions, locks and routing.
Scale improves latency but cost rises greatly	Reliability-cost trade-off.	Tune requests, target utilization, min replicas and code efficiency.
Errors only during rollout	Readiness/termination/replica strategy issue.	Tune rolling update, preStop and grace period.

Part XVI - Production-style incidents and command-level runbooks

51. Universal ten-minute triage flow

```
Minute 0-1: confirm impact and freeze changes
      |
Minute 1-2: compare last known good and recent change
      |
Minute 2-4: health, rollout, events, resource saturation
      |
Minute 4-6: logs and dependency checks
      |
Minute 6-8: decide mitigate, rollback, scale, or isolate
      |
Minute 8-10: verify recovery and communicate next update
```

Universal command block

```
NS=procureflow-dev

kubectl get deploy,pod,svc,httproute,hpa -n "$NS" -o wide
kubectl get events -n "$NS" --sort-by=.lastTimestamp | tail -50
kubectl top pods -n "$NS" --sort-by=memory
kubectl rollout history deployment/order-service -n "$NS"
kubectl logs deployment/order-service -n "$NS" --tail=200
kubectl logs deployment/order-service -n "$NS" --previous --tail=200 || true
argocd app get procureflow-dev
argocd app diff procureflow-dev || true
curl -sS -o /dev/null -w 'status=%{http_code} time=%{time_total}
' http://127.0.0.1:8080/api/suppliers
```

52. Incident A - New release causes CrashLoopBackOff

Previous evidence	Alert or user reports application unavailable immediately after release.
Why this step now	Temporal correlation makes recent GitOps/image/config change the first hypothesis, but we still verify pod reason and previous logs.
Action	Inspect rollout, pod state, events, current/previous logs and image/config diff.
Expected evidence	Identify startup error, secret missing, bad command, OOM or probe failure.
Next connection	Rollback Git commit or fix configuration, then verify endpoint and alert recovery.

```
NS=procureflow-dev
APP=order-service

kubectl get deployment "$APP" -n "$NS" -o wide
kubectl get pods -n "$NS" -l app.kubernetes.io/name="$APP" -o wide
POD=$(kubectl get pod -n "$NS" -l app.kubernetes.io/name="$APP" -o jsonpath='{.items[0].metadata.name}')

kubectl describe pod "$POD" -n "$NS"
kubectl logs "$POD" -n "$NS" --all-containers --tail=200
kubectl logs "$POD" -n "$NS" --all-containers --previous --tail=200 || true
kubectl get pod "$POD" -n "$NS" -o jsonpath='{range .status.containerStatuses[*]}{.name}{" reason="}
{.state.waiting.reason}{" lastReason="}{.lastState.terminated.reason}{" exit="}
{.lastState.terminated.exitCode}{"
"}{end}'

argocd app diff procureflow-dev
git log --oneline -5

# Mitigation: revert the bad desired-state commit
git revert --no-edit <BAD_COMMIT_SHA>
git push origin main
argocd app wait procureflow-dev --sync --health --timeout 600
curl -fsS http://127.0.0.1:8080/api/orders | jq .
```

Observation	Meaning	Next decision
Exit code 1 and application stack trace	App/config startup error.	Fix/revert code or configuration.
Last reason OOMKilled / exit 137	Memory limit or leak.	Rollback, right-size temporarily, collect JVM evidence.

Observation	Meaning	Next decision
CreateContainerConfigError	Secret/ConfigMap reference missing.	Restore dependency; do not rebuild image.
Readiness failed but process running	Dependency or probe contract issue.	Test health endpoint from pod and inspect DB/DNS.
ImagePullBackOff	Artifact availability/auth, not app startup.	Use Incident B flow.

53. Incident B - ImagePullBackOff after promotion

```
NS=procureflow-dev
APP=supplier-service
POD=$(kubectl get pod -n "$NS" -l app.kubernetes.io/name="$APP" -o jsonpath='{.items[0].metadata.name}')

kubectl describe pod "$POD" -n "$NS" | sed -n '/Events:/,$p'
kubectl get deployment "$APP" -n "$NS" -o jsonpath='{.spec.template.spec.containers[0].image}{"
"}'

curl -fsS http://127.0.0.1:5001/v2/_catalog | jq .
curl -fsS http://127.0.0.1:5001/v2/procureflow/supplier-service/tags/list | jq .

for node in $(kind get nodes --name procureflow); do
  echo "== $node =="
  docker exec "$node" cat /etc/containerd/certs.d/localhost:5001/hosts.toml
  docker exec "$node" crictl pull localhost:5001/procureflow/supplier-service:<TAG> || true
done

# Fast safe mitigation: revert GitOps tag to last known good
git revert --no-edit <PROMOTION_COMMIT>
git push origin main
argocd app wait procureflow-dev --sync --health --timeout 600
```

Observation	Meaning	Next decision
Tag absent in registry	CI/promotion ordering failure.	Publish image or revert tag; enforce artifact-exists gate.
Tag exists but node connection refused	Registry mirror/network issue.	Restore registry and hosts.toml/network.
Manifest unknown	Repository/tag spelling mismatch.	Compare exact image string and registry path.
Unauthorized	Private registry credentials missing.	Use imagePullSecret/identity; do not use public fallback.

54. Incident C - Order Service unavailable because PostgreSQL is down

```
NS=procureflow-dev

kubectl get pod,endpoints,pvc -n "$NS" -l app.kubernetes.io/name=order-postgres
kubectl describe pod order-postgres-0 -n "$NS"
kubectl logs order-postgres-0 -n "$NS" --tail=200
kubectl logs deployment/order-service -n "$NS" --tail=200 | grep -Ei 'postgres|connection|hikari|flyway|
timeout' || true

kubectl run db-dns-check -n "$NS" --rm -i --restart=Never   --image=postgres:17.5-alpine --   sh -c
'getent hosts order-postgres; pg_isready -h order-postgres -p 5432'

kubectl get secret order-postgres-credentials -n "$NS" -o json   | jq -r '.data | keys[]'
kubectl get events -n "$NS" --sort-by=.lastTimestamp | tail -30
```

Observation	Meaning	Next decision
PVC Pending/Lost	Storage path failure.	Restore volume/provisioner; protect data before recreating.
DB pod Ready but app auth fails	Credential rotation/mismatch.	Align secret and DB user; coordinate restart.
DNS fails	Service/CoreDNS issue.	Inspect Service/endpoints and DNS pods.
Connection pool exhausted	DB reachable but saturated/leaked.	Inspect active connections, pool metrics, slow queries; scale carefully.
Flyway error after release	Schema migration incompatibility.	Rollback app/schema using approved migration strategy.

55. Incident D - Java OOMKilled

```
NS=procureflow-dev
APP=order-service
POD=$(kubectl get pod -n "$NS" -l app.kubernetes.io/name="$APP" -o jsonpath='{.items[0].metadata.name}')

kubectl describe pod "$POD" -n "$NS" | sed -n '/Containers:/,/Conditions:/p'
kubectl get pod "$POD" -n "$NS" -o jsonpath='{range .status.containerStatuses[*]}{.name}{" lastReason="}
{.lastState.terminated.reason}{" exit="}{.lastState.terminated.exitCode}{" restarts="}{.restartCount}{"
"}{end}'
kubectl top pod "$POD" -n "$NS" --containers
kubectl logs "$POD" -n "$NS" --previous --tail=300

# Confirm requested/limited memory and JVM MaxRAMPercentage
kubectl get deployment "$APP" -n "$NS" -o yaml | grep -A8 -B4 -E 'resources:|MaxRAMPercentage'

# Immediate mitigation through Git: revert release or temporarily raise tested limit.
argocd app diff procureflow-dev
git log --oneline -5
```

Senior answer: pehle recovery, phir evidence. Temporary memory increase business restore kar sakti hai, lekin leak prove/fix ke baghair permanent solution nahi. JVM heap container limit se comfortably below hona chahiye taa ke metaspace, threads, native buffers and JVM overhead ke liye headroom rahe.

56. Incident E - Kafka consumer lag increases

```
kubectl exec -n procureflow-dev kafka-0 --   /opt/kafka/bin/kafka-consumer-groups.sh   --bootstrap-server
kafka:9092   --describe --group approval-service

kubectl get pods -n procureflow-dev -l app=approval-service -o wide
kubectl top pods -n procureflow-dev -l app=approval-service
kubectl logs -n procureflow-dev -l app=approval-service --since=15m --prefix

kubectl exec -n procureflow-dev kafka-0 --   /opt/kafka/bin/kafka-topics.sh   --bootstrap-server
kafka:9092   --describe --topic order-created
```

Observation	Meaning	Next decision
Lag grows, consumer errors	Consumer unhealthy or poison event.	Fix errors, isolate event, restart safely.
Lag grows, CPU saturated	Processing capacity too low.	Scale consumers up to useful partition count.
Consumers > partitions	Extra replicas cannot receive partitions.	Increase partitions only with ordering/key implications understood.
Lag spike then falls	Burst handled within recovery objective.	Tune alert duration; no incident if SLO protected.
Broker unavailable	Producer/consumer symptom is broker-side.	Check broker quorum, disk and network first.

57. Incident F - Argo CD OutOfSync or Degraded

```
argocd app get procureflow-dev
argocd app diff procureflow-dev
argocd app history procureflow-dev
argocd app resources procureflow-dev

kubectl get events -n procureflow-dev --sort-by=.lastTimestamp | tail -50
kubectl logs deployment/argocd-application-controller -n argocd --tail=200
kubectl logs deployment/argocd-repo-server -n argocd --tail=200

# Only after understanding the diff:
argocd app sync procureflow-dev
argocd app wait procureflow-dev --sync --health --timeout 600
```

Observation	Meaning	Next decision
ComparisonError	Repo/path/render/credentials failure.	Fix source access or Helm render.
OutOfSync with manual replicas	Live drift.	Allow self-heal or remove manual mutation.
Degraded Deployment	Desired manifest applied but runtime unhealthy.	Troubleshoot pod; sync alone will not fix bad image/config.
Prune required	Resource removed from Git remains live.	Review blast radius, then sync with prune.
Healthy but wrong behavior	Kubernetes health does not equal business health.	Run business smoke tests and metrics.

58. Incident documentation template

```
Title:
Start time / detection time:
Customer or business impact:
Affected service and environment:
Current mitigation:

Timeline:
- HH:MM Alert/user report
- HH:MM Change correlation checked
- HH:MM Evidence collected
- HH:MM Mitigation selected
- HH:MM Service restored

Root cause:
Contributing factors:
Why detection did/did not work:
Corrective actions with owner and due date:
- Prevention
- Detection
- Mitigation
- Documentation/testing

Evidence links:
Git commit, Argo history, Jenkins build, dashboard, logs, commands.
```

Part XVII - Daily operations, startup, shutdown, cleanup, and recovery

59. Daily status script

Step 59.1 - Create one-command platform status

Previous evidence	All platform layers now exist.
Why this step now	Demo/interview se pehle one status command missing service quickly identify kare, without manually remembering every namespace and container.
Action	Run the copy-paste block below.
Expected evidence	Status groups show healthy Docker, nodes, app, controllers, GitOps and HTTP checks.
Next connection	One-command visibility reduces demo risk and supports fast triage.

```
cd ~/projects/procureflow-devops-modernization

cat > scripts/90-status.sh <<'EOF'
#!/usr/bin/env bash
set -u

echo '== Host and Docker =='
free -h
docker info --format 'Docker={{.ServerVersion}} CPUs={{.NCPU}} Memory={{.MemTotal}}' 2>&1 || true
docker ps --format 'table {{.Names}}  {{.Status}}     {{.Ports}}'

echo
echo '== kind =='
kind get clusters
kubectl config current-context
kubectl get nodes -o wide

echo
echo '== ProcureFlow =='
kubectl get deploy,statefulset,pod,svc,httproute,hpa -n procureflow-dev -o wide

echo
echo '== Platform controllers =='
kubectl get pods -n envoy-gateway-system
kubectl get pods -n argocd
kubectl get pods -n monitoring

echo
echo '== GitOps =='
argocd app get procureflow-dev 2>&1 | sed -n '1,35p' || true

echo
echo '== Endpoints =='
curl -sS -o /dev/null -w 'supplier status=%{http_code} time=%{time_total}
' http://127.0.0.1:8080/api/suppliers || true
curl -sS -o /dev/null -w 'orders status=%{http_code} time=%{time_total}
' http://127.0.0.1:8080/api/orders || true
EOF

chmod +x scripts/90-status.sh
./scripts/90-status.sh
```

Output ko kaise read karna hai

Observation	Meaning	Next decision
All healthy	Demo-ready.	Capture screenshots/evidence.
One namespace unhealthy	Troubleshoot only that layer.	Use corresponding incident runbook.
Port-forward URLs fail but pods healthy	Local access tunnel absent.	Restart port-forward, not workloads.
Argo CLI auth expired	Status script still shows Kubernetes state.	Re-login to local Argo server.

60. Safe shutdown and restart

```
# Stop optional Docker CI stack while preserving volumes
docker compose -p procureflow-ci -f ci/jenkins/compose.yaml stop

# Scale optional heavy Kubernetes profiles down while keeping PVCs
kubectl scale deployment/elasticsearch -n procureflow-dev --replicas=0
kubectl scale deployment/rabbitmq -n procureflow-dev --replicas=0
kubectl scale statefulset/kafka -n procureflow-dev --replicas=0
kubectl scale statefulset/target-mysql -n procureflow-dev --replicas=0

# Full cluster stop for Docker Desktop restart: kind nodes stop with Docker.
# After Docker Desktop restarts:
docker start procureflow-registry || true
./scripts/20-kind-up.sh
kubectl get nodes

# Restore selected profile
kubectl scale statefulset/kafka -n procureflow-dev --replicas=1
kubectl rollout status statefulset/kafka -n procureflow-dev --timeout=600s
```

61. Destructive cleanup boundaries

Destructive commands
kind delete cluster, docker compose down -v, kubectl delete PVC and terraform destroy can permanently remove local data. Run only after confirming which evidence/data must be retained. Never paste these into a shared or production context.

```
# Non-destructive: stop CI containers, keep volumes
docker compose -p procureflow-ci -f ci/jenkins/compose.yaml stop

# Destructive CI reset: deletes Jenkins/Sonar data volumes
docker compose -p procureflow-ci -f ci/jenkins/compose.yaml down -v

# Destructive cluster reset: deletes all kind cluster state and PVC data
kind delete cluster --name procureflow

# Registry cleanup only after confirming images can be rebuilt
# docker rm -f procureflow-registry

# Review before pruning Docker resources
docker system df
docker image ls
docker volume ls
# Avoid docker system prune -a --volumes unless you fully understand the impact.
```

Part XVIII - Interview presentation, demonstration flow, and answers

62. Ninety-second project introduction

Say this naturally

ProcureFlow is my local enterprise modernization project for a procurement platform. I started with measurable Java and Node.js services and a legacy MySQL workload, then built a local Kubernetes platform with kind, a private local registry, Gateway API and Helm. Jenkins runs tests, builds and scans immutable images, while Argo CD owns deployment through GitOps. I integrated PostgreSQL with Flyway, demonstrated MySQL backup, restore, checksum validation, cutover and rollback, and added Kafka, RabbitMQ, Elasticsearch and Prometheus-based observability as staged profiles. The important part is that every next step depends on evidence from the previous step, so I can isolate failures and explain reliability, security, performance and cost trade-offs instead of only listing tools.

63. Ten-minute interview presentation

Time	Section	What to say/show
0:00-1:00	Business problem	Legacy procurement platform has manual deployment, stateful risk, weak observability and inconsistent environments.
1:00-2:00	Target architecture	Node.js + Java, PostgreSQL, messaging/search, kind Kubernetes, Gateway API.
2:00-3:00	Platform foundation	WSL/Docker preflight, local registry path, multi-node kind, storage and metrics validation.
3:00-4:00	Application packaging	Non-root images, health/metrics, Helm values, dev/staging/prod-like differences.
4:00-5:00	CI/CD	Jenkins tests/scans/pushes Git SHA images; Argo CD reconciles Git desired state.
5:00-6:30	Stateful migration	Baseline counts/hash, dump, target StatefulSet/PVC, restore, checksum, restart, go/no-go, rollback.
6:30-7:30	Observability/SRE	ServiceMonitors, Prometheus queries, SLI/SLO, tested alerts, 10-minute triage.
7:30-8:30	Scale/security/cost	HPA evidence, resource requests, tokenless service account, Trivy/SBOM, staged profiles.
8:30-9:30	Incident example	Bad image or DB outage: detect, correlate, inspect, mitigate/revert, verify, postmortem.
9:30-10:00	Boundary and next step	Local patterns validated; Azure target maps registry to ACR, kind to AKS, secrets to Key Vault, DB to managed service decision.

64. Live demonstration order

22. Run ./scripts/90-status.sh and show green platform state.
23. Create an order through Gateway and show it remains after application pod restart.
24. Show Jenkins artifact metadata with Git SHA image tag.
25. Show Argo CD Synced/Healthy and one Git-driven image history entry.
26. Show Prometheus target/query and Grafana resource/application panels.
27. Run migration validation script and show MIGRATION_VALIDATION=PASS.
28. Show one incident runbook and explain the decision tree; do not deliberately break the full demo unless time allows.

65. JD-to-project evidence map

JD requirement	Project implementation	Evidence to show
Linux administration	WSL Ubuntu, SSH legacy node, packages, users, filesystem, sysctl	Ansible second run changed=0
Ansible	Legacy node configuration	inventory.ini, site.yaml, idempotency output
Terraform	Docker resource lifecycle	plan/apply/state/drift/destroy
Azure understanding	Documented local-to-AKS mapping	ADR and architecture boundary
Bash/Python/Groovy	Scripts, manifest mutations/validation, Jenkinsfile	Git history and CI stages
Kubernetes/Helm/GitOps	kind, Helm chart, Gateway API, Argo CD	Synced/Healthy and self-heal
Jenkins/container builds	Tests, Trivy, local registry, SHA tags	artifact-metadata.txt
Prometheus	ServiceMonitor, rules, tested alert	Prometheus API/Grafana
Node.js and Java	Supplier and Order services	health, metrics, tests, images
MySQL/PostgreSQL	Legacy migration and modern persistence	checksum PASS and durable order
Kafka/RabbitMQ	Events vs work queues	topic/group lag and queue/DLQ
Elasticsearch	Search profile	cluster health, index and query
Security	non-root, read-only, RBAC/token, scans/SBOM	Trivy output and pod verification
Performance/cost	k6, HPA, quotas, staged profiles	before/after evidence
Incident ownership	Command-level runbooks and rollback	timeline/evidence/postmortem template

66. High-probability interview questions and connected answers

Why did you choose kind instead of Minikube?

Kind uses Docker containers as Kubernetes nodes and is easy to recreate in CI/local environments. The key decision was not brand preference; it was that I already used Docker Desktop with WSL 2 and needed a repeatable multi-node cluster plus documented local registry integration. I would still evaluate Minikube or k3d if their driver/networking behavior better matched a specific test.

Why not let Jenkins deploy directly?

Jenkins is trusted to build and publish artifacts. Argo CD is trusted to reconcile cluster state. This reduces CI cluster credentials, keeps deployment history in Git, detects drift and makes rollback an auditable Git revert. The handoff artifact is an immutable image tag plus a reviewed desired-state change.

Why use both Kafka and RabbitMQ?

OrderCreated is a domain event that multiple independent consumers may need and may replay, so Kafka fits. A notification is a task that one worker should complete with acknowledgement, retry and dead-letter behavior, so RabbitMQ fits. I selected them by delivery semantics, not because both appeared in the JD.

Would you really run MySQL in Kubernetes in production?

Not automatically. The lab validates StatefulSet, PVC, backup and restore mechanics. In Azure I would compare AKS-hosted MySQL with Azure Database for MySQL based on availability, RPO/RTO, patching, backup, operational skill, latency, compliance and cost. The project demonstrates the decision process and migration controls, not a universal recommendation.

How did you avoid downtime during deployment?

Readiness prevents traffic before dependencies are ready, rolling Deployment strategy keeps old pods until new pods become ready, production-like values use at least two replicas and a PodDisruptionBudget, and Argo health gates plus business smoke

tests verify the release. Database schema changes require backward-compatible expand/migrate/contract sequencing rather than assuming application rollback can undo data changes.

How do you troubleshoot a 500 spike?

First confirm whether the metric is real and identify affected route/version. Then freeze changes, compare canary/current release against last known good, inspect 5xx rate, latency, pods, events and saturation, read current and previous logs, test downstream database/messaging dependencies, and select mitigation. If impact correlates with release and rollback is safe, revert Git desired state; then verify business requests and alerts recover before root-cause analysis.

What is the biggest limitation of this project?

It validates engineering patterns on one workstation, not cloud availability zones, managed identities, cloud load balancers, managed database SLA or enterprise-scale data. I documented those gaps and designed direct mappings to AKS, ACR, Key Vault, Azure networking and managed data services.

67. Resume bullet examples

- Designed and implemented a local enterprise procurement-platform modernization lab using Java 21, Node.js, Docker, kind Kubernetes, Helm, Gateway API, Jenkins and Argo CD.
- Built evidence-driven CI/CD with automated tests, immutable Git SHA image tags, Trivy security gates, local registry delivery, GitOps self-healing and auditable rollback.
- Executed a stateful MySQL migration rehearsal using source baselines, logical backup, PVC-backed target restore, deterministic checksum validation, restart testing and go/no-go rollback procedures.
- Implemented Prometheus/Grafana observability, application ServiceMonitors, SLI/SLO queries, tested alerts, k6 load tests, HPA validation and production-style incident runbooks.
- Demonstrated Terraform resource lifecycle, Ansible idempotent Linux configuration, Kafka consumer-lag analysis, RabbitMQ dead-letter design and Elasticsearch search operations.

68. Evidence checklist before interview

Evidence	Minimum proof	Your status
Architecture diagram	Current and target flow with arrows and ownership	Ready / missing
Git history	Clear milestone commits, no secrets	Ready / missing
Jenkins	One green build and artifact metadata	Ready / missing
Argo CD	Synced/Healthy, history and self-heal proof	Ready / missing
Application	Supplier response, order creation and persistence	Ready / missing
Migration	MIGRATION_VALIDATION=PASS	Ready / missing
Prometheus/Grafana	Targets, query and alert evidence	Ready / missing
Load test	Baseline and HPA comparison	Ready / missing
Incident	One completed timeline/postmortem	Ready / missing
Security	Trivy report, SBOM, non-root/tokenless proof	Ready / missing

Appendix A - Command-reading rules and common failure patterns

A.1 How to read kubectl get pods

Observation	Meaning	Next decision
Running 1/1	Container running and Ready.	Continue to service/business validation.
Running 0/1	Process running but readiness false.	Inspect readiness endpoint and dependencies.
CrashLoopBackOff	Repeated startup failure.	Describe, current logs, previous logs, exit reason.
ImagePullBackOff	Runtime cannot obtain image.	Registry/tag/auth/network.
Pending	Scheduling, PVC, quota or image initialization.	Describe pod and inspect events.
Completed	One-shot process exited successfully.	Expected for Job; unexpected for service Deployment.

A.2 How to read curl results

Observation	Meaning	Next decision
200	Request succeeded.	Validate response content and latency, not status only.
201	Resource created.	Read ID and verify durable retrieval.
400	Client/request validation failure.	Check payload/schema.
401/403	Authentication/authorization.	Check identity/role/token, not network.
404	Path/route/resource mismatch.	Check Gateway match and application mapping.
502/503	Gateway has bad/unavailable backend.	Check endpoints, readiness and upstream logs.
Timeout	Network, saturation, deadlock or dependency wait.	Compare internal/external path and resource signals.

A.3 How to read Helm and Argo results

Observation	Meaning	Next decision
helm lint passes	Chart syntax/basic semantics valid.	Still render and server dry-run.
helm template passes	Local rendering works.	Still validate current cluster API.
server dry-run passes	API accepts resources.	Still observe runtime readiness.
Argo Synced	Desired/live manifests match.	Does not guarantee business health.
Argo Healthy	Resource health checks pass.	Run business smoke test.
Argo Degraded	One or more resources unhealthy.	Inspect resource tree and pod evidence.
ComparisonError	Argo cannot calculate desired state.	Repo/auth/path/Helm render problem.

Appendix B - Local-to-Azure mapping

Local lab	Azure target	Additional production concern
kind cluster	Azure Kubernetes Service	Managed control plane, zones, node pools, identity
Local registry	Azure Container Registry	Private endpoint, managed identity, replication
Docker/Kind network	Azure VNet/subnets/NSGs/private DNS	Address planning and egress controls
Kubernetes Secret	Key Vault + Workload Identity/CSI	Rotation and no static cloud credentials

Local lab	Azure target	Additional production concern
Local PVC	Azure Disk/File or managed database	Zones, snapshots, performance tiers
Gateway API/Envoy	AKS Gateway implementation/Application Gateway	Public/private load balancer, WAF, DNS/TLS
Local Prometheus	Managed Prometheus/Azure Monitor or self-managed	Retention, scale and alert routing
Docker Compose legacy source	Actual Linux/Windows VM estate	Discovery, dependency mapping, maintenance window
Terraform Docker provider	azurerm/azapi modules	Remote state, RBAC, policy, subscriptions
Local Jenkins	Managed/hosted agents or Azure-integrated CI	Autoscaling agents, secure secrets, network access

Appendix C - Glossary and full forms

Term	Full form	Meaning in this project
AKS	Azure Kubernetes Service	Microsoft-managed Kubernetes service.
ACR	Azure Container Registry	Managed private container registry.
CI	Continuous Integration	Automated validation and artifact creation.
CD	Continuous Delivery/Deployment	Controlled promotion and deployment.
GitOps	Git-operated desired state	Git is reviewed source of deployment truth.
HPA	Horizontal Pod Autoscaler	Changes pod replica count from metrics.
PVC	PersistentVolumeClaim	Workload request for persistent storage.
PDB	PodDisruptionBudget	Controls voluntary disruption availability.
RPO	Recovery Point Objective	Acceptable data-loss window.
RTO	Recovery Time Objective	Target restoration time.
SLI	Service Level Indicator	Measured service behavior.
SLO	Service Level Objective	Target for an SLI.
SLA	Service Level Agreement	External agreement often with consequences.
SBOM	Software Bill of Materials	Inventory of software components.
DLQ	Dead-Letter Queue	Failed messages isolated for investigation.
KRaft	Kafka Raft metadata mode	Kafka metadata quorum without ZooKeeper.
WAF	Web Application Firewall	HTTP-layer threat filtering.
RBAC	Role-Based Access Control	Permissions granted through roles.
IaC	Infrastructure as Code	Version-controlled infrastructure definitions.
JVM	Java Virtual Machine	Runtime executing Java bytecode.
OOM	Out Of Memory	Process/container exceeded available memory.

Appendix D - Official documentation references

Always verify version-specific commands against official documentation before a fresh rebuild. The following sources informed the architecture and current local choices:

- Docker Desktop WSL 2 backend and integration: https://docs.docker.com/desktop/features/wsl/
- Docker development with WSL 2: https://docs.docker.com/desktop/features/wsl/use-wsl/
- kind quick start and configuration: https://kind.sigs.k8s.io/docs/user/quick-start/ and https://kind.sigs.k8s.io/docs/user/configuration/
- kind local registry guide: https://kind.sigs.k8s.io/docs/user/local-registry/
- Kubernetes Gateway API: https://gateway-api.sigs.k8s.io/

- Gateway API migration from Ingress: https://gateway-api.sigs.k8s.io/guides/getting-started/migrating-from-ingress/
- Envoy Gateway Helm installation and quick start: https://gateway.envoyproxy.io/docs/install/install-helm/ and https://gateway.envoyproxy.io/v1.8/tasks/quickstart/
- Helm documentation: https://helm.sh/docs/
- Argo CD documentation: https://argo-cd.readthedocs.io/
- Prometheus Operator/kube-prometheus-stack chart: https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack
- Terraform documentation: https://developer.hashicorp.com/terraform/docs
- Ansible documentation: https://docs.ansible.com/
- Jenkins Pipeline documentation: https://www.jenkins.io/doc/book/pipeline/
- Trivy documentation: https://trivy.dev/latest/docs/
- k6 documentation: https://grafana.com/docs/k6/latest/

Appendix E - Final connected project chain

```
1. Business problem and transaction defined
        |
2. WSL/Docker/resource preflight proved
        |
3. Tool versions pinned and Git baseline created
        |
4. Local registry path proved end-to-end
        |
5. kind cluster, storage, Metrics API and Gateway proved
        |
6. Node.js and Java images independently tested and pushed
        |
7. Helm rendered, server dry-run validated, dev release deployed
        |
8. PostgreSQL persistence and Flyway dependency validated
        |
9. Legacy MySQL source measured, backed up, restored and compared
        |
10. Kafka/RabbitMQ/Elasticsearch staged operational profiles tested
        |
11. Terraform lifecycle and Ansible idempotency demonstrated
        |
12. Jenkins tests, scans and publishes immutable Git SHA images
        |
13. Argo CD reconciles Git state, self-heals and rolls back by revert
        |
14. Prometheus/Grafana collect signals and tested alerts
        |
15. Trivy, SBOM, non-root, tokenless identity improve security
        |
16. k6 baseline and HPA show performance/cost trade-off
        |
17. Incident runbooks connect evidence to mitigation and recovery
        |
18. Interview presentation shows decisions, proof and limitations
```

Every later layer depends on verified evidence from the previous layer

Final message

The project is strongest when you can explain not only what you installed, but why that component came next, what evidence allowed the decision, what failure would stop progression, how you would roll back, and what changes when moving from a local lab to Azure production.